BP101T COMPANION TEXT

Python for Pharmacists

A Complete Course in Python Programming

for the Pharmaceutical Sciences

Mr. Tajuddin Shaik

Co-Founder, BioDockify

Associate Professor, Department of Pharmacology

Malla Reddy College of Pharmacy

Course BP101T · B.Pharm Semester I · 30 Teaching Hours

18 Chapters · 5 Units · 355 Machine-Verified Code Examples

BioDockify

Building the Future of AI-Driven Pharmaceutical Research

learn.biodockify.com

Python for Pharmacists

A Complete Course in Python Programming for the Pharmaceutical Sciences

Mr. Tajuddin Shaik

Co-Founder, BioDockify

Associate Professor, Department of Pharmacology

Malla Reddy College of Pharmacy

Companion text for BP101T · B.Pharm Semester I

Based on the PCI-specified syllabus · 2026 edition

Published by the BioDockify Team

learn.biodockify.com

Copyright © 2026 Mr. Tajuddin Shaik · BioDockify Team

Published on learn.biodockify.com

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission.

This is a companion text for BP101T — Basics of Python Programming for Pharmaceutical Sciences (Theory), B.Pharm Semester I, as specified by the Pharmacy Council of India.

Every code example in this book has been machine-verified by execution.

Preface

This book was written for a specific student: the first-semester B.Pharm undergraduate who has never written a line of code, who may be intimidated by the word “programming”, and who deserves a textbook that speaks plainly, uses examples from their own field, and never skips the step they are stuck on.

Every concept arrives with a pharmacy anchor — a dose calculation before a loop, a batch ID before a string method, a concentration-time curve before a Matplotlib call. The mathematics is always secondary to the pharmacy; the code is always secondary to the question the pharmacist is trying to answer.

The book follows the PCI-specified BP101T syllabus unit by unit, chapter by chapter, bullet by bullet. Nothing in the syllabus is missing; nothing outside it is required. Each chapter opens with theory, proceeds through four to five worked examples per concept, and closes with quick-check questions. Each unit ends with a twenty-question exercise bank, an examination-points summary, and a syllabus coverage map.

A note on verification. Every code example in this book was executed by machine before publication; every printed output in the text is the actual output of the code above it. There are no invented examples and no approximations. Where a formula is used — Mosteller BSA, Cockcroft–Gault, Clark and Young rules — the clinical result was computed and checked, not asserted.

A note on originality. The prose in this book is original. Topic coverage was cross-checked against several teaching resources (credited on the Sources page), and an automated audit confirmed zero seven-word prose overlaps with every consulted source.

How to use this book. Read each chapter with a Jupyter notebook open. Type every example — do not copy-paste. Change the numbers and re-run. The moment a program prints an answer you did not expect is the moment you are learning.

— The Author

November 2026

Table of Contents

Preface....4

Chapter 1 — Python and Your Toolkit....6

1.1 What is a program?....6

1.2 How Python runs your code....6

1.3 Why should a pharmacy student learn Python?....7

1.4 Features of Python (syllabus point)....8

1.5 A short history of Python....9

1.6 Installing Python....9

1.7 Anaconda — the all-in-one option....10

1.8 IDEs — where you will type code....10

1.9 Why we never write Python in a plain text editor....12

1.10 Your first program....12

1.11 Troubleshooting your installation....13

Chapter 2 — Variables, Data Types and Type Casting....13

2.1 What is a variable?....14

2.2 Rules for naming variables....15

2.3 The four basic data types — theory....16

2.4 Checking a type: type()....19

2.5 The classic trap: 500 vs "500"....19

2.6 Type casting — converting on purpose....20

2.7 Comments — notes to the reader (and to yourself)....22

2.8 Worked example — paediatric syrup calculation....23

Chapter 3 — Operators, Input and Output....24

3.1 The seven arithmetic operators — theory....24

3.2 Operator precedence — who goes first?....25

3.3 Comparison operators — theory....26

3.4 Logical operators — and, or, not....26

3.5 Assignment shortcuts....27

3.6 input() — asking the user....28
3.7 print() — more than it looks....28
3.8 f-strings — printing like a human....28
3.9 Four complete pharmacy programs....29
Chapter 4 — String Operations and Manipulation....31
4.1 The bead-on-a-thread model — theory....31
4.2 Indexing — every bead has a number....31
4.3 Slicing — cutting out a piece....32
4.4 Immutability in action....33
4.5 Case and cleaning methods — upper, lower, title, strip....33
4.6 replace — swapping text....34
4.7 Searching — find, count, in....35
4.8 Taking apart and joining — split, join....35
4.9 Joining and repeating — +, *, f-strings....36
4.10 Worked programs....36
Chapter 5 — Libraries: Standard, Third-Party and pip....37
5.1 What is a library? — theory....37
5.2 Three ways to import....38
5.3 The math module....38
5.4 The datetime module — dates that calculate....39
5.5 Two more for the road....40
5.6 pip — install, list, uninstall (syllabus point)....41
5.7 Worked program — expiry report, complete....42
Unit I — Consolidation....42
Syllabus coverage map....42
Twenty-question exercise bank....42
Answers to selected exercises....45
Important points for examination....46
Chapter 6 — Conditional Statements....48
6.1 Why programs must decide....48

6.2 The if statement — theory and anatomy....48
6.3 if-else — two roads....50
6.4 if-elif-else — many roads, first True wins....51
6.5 Nested conditions — a question inside a question....53
Chapter 7 — Loops....55
7.1 Why loops exist....55
7.2 The for loop — anatomy and the loop variable....55
7.3 range() — counting without a list....56
7.4 The accumulator and counter patterns....57
7.5 The while loop — repeat while a condition holds....58
7.6 break and continue — the syllabus pair....59
7.7 enumerate() — looping with the index....61
7.8 zip() — looping two lists together....62
7.9 Nested loops — a loop inside a loop....63
Chapter 8 — Functions....64
8.1 Why functions exist....64
8.2 Anatomy of a function....65
8.3 Parameters and arguments....65
8.4 return — sending the answer back....65
8.5 Local scope — a function's variables are private....67
8.6 Modular program 1 — dosage calculation (syllabus)....67
8.7 Modular program 2 — BMI calculation (syllabus)....68
8.8 The dispensary toolkit — a third assembler....68
8.9 Lambda functions — one-line anonymous functions....69
8.10 Error handling — try, except, else, finally....71
8.11 Variable arguments — args and *kwargs....73
Unit II — Consolidation....75
Syllabus coverage map....75
Twenty-question exercise bank....75
Answers to selected exercises....78

Important points for examination....79
Chapter 9 — Lists and Tuples....81
9.1 Why collections exist....81
9.2 Creating, indexing and slicing lists....81
9.3 List methods — adding, removing, sorting....82
9.4 List comprehensions — one-line list building....83
9.5 Nested lists — a table in a variable....84
9.6 Tuples — the sealed record....85
Chapter 10 — Dictionaries and String Tools....87
10.1 Dictionaries — data with names on it....87
10.2 Adding, updating and removing....87
10.3 Looping through a dictionary....88
10.4 Nested dictionaries — a small patient database....89
10.5 Dictionary comprehension....89
10.6 String manipulation techniques (syllabus point)....89
10.7 Sets — the collection of unique values....90
Chapter 11 — NumPy Arrays....93
11.1 What NumPy is and why it exists....93
11.2 Creating arrays....94
11.3 Element-wise arithmetic — the vectorized advantage....95
11.4 Aggregation functions....95
11.5 Slicing and boolean filtering....96
11.6 Two-dimensional arrays — tables of numbers....97
11.7 reshape() — changing the shape without changing the data....99
11.8 Broadcasting — operating on mismatched shapes....100
Chapter 12 — CSV Files and Healthcare Datasets....101
12.1 What CSV is....101
12.2 Reading a CSV file — the csv module....102
12.3 Writing a CSV file....102
12.4 DictReader — rows as dictionaries....103

12.5 Handling a missing file safely....103
12.6 Structured healthcare datasets (syllabus point)....104
12.7 Importing a pharmaceutical dataset — the complete workflow....104
Unit III — Consolidation....105
Syllabus coverage map....105
Twenty-question exercise bank....105
Answers to selected exercises....108
Important points for examination....109
Chapter 13 — Pandas Fundamentals....111
13.1 What Pandas is and why it exists....111
13.2 The Series — one labelled column....112
13.3 The DataFrame — a full table....114
13.4 Reading CSV and Excel files (syllabus point)....115
13.5 Inspecting datasets (syllabus point)....116
13.6 Sorting and ranking....117
13.7 Adding, removing and renaming columns....118
Chapter 14 — Advanced DataFrame Operations....119
14.1 Why more operations matter....119
14.2 String methods on columns....119
14.3 Applying functions to columns — apply() and map()....121
14.4 Merging and joining DataFrames....122
14.5 Datetime handling in Pandas....123
Chapter 15 — Cleaning, Filtering and Grouping....124
15.1 Missing values — the NaN problem (syllabus point)....124
15.2 Filtering and selecting data (syllabus point)....126
15.3 Grouping and aggregation (syllabus point)....128
15.4 The ADR case study — the complete workflow....129
Unit IV — Consolidation....130
Syllabus coverage map....130
Twenty-question exercise bank....130

Answers to selected exercises....133
Important points for examination....133
Chapter 16 — Matplotlib Fundamentals....136
16.1 What Matplotlib is and why pharmacists plot....136
16.2 The plotting workflow....136
16.3 Line plots — the workhorse....137
16.4 Histograms — the distribution view....139
16.5 Scatter plots — the relationship view....140
16.6 Box plots — the five-number summary....141
16.7 Labelling axes, titles and legends (syllabus point)....142
16.8 Figure sizing and saving — producing publication-quality files....143
16.9 Subplots — multiple charts on one figure....146
Chapter 17 — Pharmaceutical Plots (Syllabus Point)....148
17.1 Concentration-time curves — oral vs IV administration....148
17.2 ADR reporting rates — the bar chart....149
17.3 Dissolution profiles — comparing formulations....150
17.4 The mini PK study — a complete workflow....150
17.5 Annotations — labelling points on the plot....151
17.6 Advanced bar charts — stacked and grouped....152
17.7 Error bars — showing uncertainty....153
Chapter 18 — Scientific Interpretation of Plots (Syllabus Point)....154
18.1 The figure is only step one....154
18.2 The interpretation reference table....154
18.3 Correlation is not the same as causation....154
18.4 Common interpretation mistakes....155
18.5 Pie charts — proportions of a whole....155
18.6 Colour maps and colour bars....156
Unit V — Consolidation....157
Syllabus coverage map....157
Twenty-question exercise bank....157

Answers to selected exercises....160
Important points for examination....161
Recommended References....162
Appendix A — Quick Reference....163
Appendix B — Practice Datasets....163
Sources and Acknowledgements....165
About the Publisher....166

Right-click the table of contents and choose “Update Field” to refresh page numbers.

UNIT I

Introduction to Python Programming

Chapters 1–5 · 6 teaching hours

Chapter 1 — Python and Your Toolkit

1.1 What is a program?

A **computer program** is a list of instructions that a computer follows, one after another. It is exactly like a recipe: ingredients go in (data), steps are followed in order (instructions), and a dish comes out (results). Everything a computer does — from printing your college hall ticket to analysing a clinical trial — is a program.

Python is the language we use to write those instructions. It is called a *high-level* language because it hides the electronics and lets you write steps that read almost like English: the same task usually takes fewer lines in Python than in older languages such as C++ or Java. Python is also *general-purpose* — the same language powers websites, hospital dashboards and drug-discovery research — so what you learn here travels anywhere.

1.2 How Python runs your code

Two facts explain almost everything you will see in this book.

1. **Python reads your program line by line, from top to bottom**, translating each line into action as it goes (this is what *interpreted* means). There is no separate "build" step: the moment you run the code, it runs. When an error exists on line 10, lines 1–9 have already executed — which is why partial output sometimes appears before an error message.
2. **Python can be used in two modes.** In *interactive mode* (the `>>>` prompt of a plain Python session, or a Jupyter cell) you type a line and see its result instantly — ideal for experimenting. In *script mode* you save a `.py` file and run the whole file at once — ideal for finished

programs. Jupyter gives you both worlds: cells behave interactively, but the notebook is a saved document.

```
# script mode: nothing appears unless you print it
dose = 500
print(dose)
```

```
OUTPUT
500
```

1.3 Why should a pharmacy student learn Python?

Modern pharmacy runs on data. A single pharmacokinetic (PK) study produces many concentration readings per patient; pharmacovigilance centres receive thousands of adverse drug reaction (ADR) reports every month; every dissolution test adds batch after batch of time-point measurements. Spreadsheets struggle as this data grows. Python handles it comfortably — and never makes a copy-paste mistake.

Python is also:

- **Free and open** — no licence, no cost, runs on Windows, macOS and Linux.
- **The most popular programming language in the world** (first place on the TIOBE index for several years running), which means every question you will ever have already has an answer online.
- **The language of science** — the data-analysis tools used in pharmaceutical research (NumPy, Pandas, Matplotlib, RDKit) are all Python libraries, and Units III–V of this course are built on them.

1.4 Features of Python (syllabus point)

Seven properties explain why Python became the default language of science:

1. **Readable** — programs read close to everyday English, which shortens the learning curve for newcomers.
2. **Interpreted** — instructions take effect immediately, one line at a time, so mistakes surface early.
3. **Free and open** — anybody may use, share and study it; a large community maintains it.
4. **Portable** — one program behaves the same across Windows, macOS and Linux.
5. **Batteries included** — a rich standard library plus a vast ecosystem of community packages (NumPy, Pandas, Matplotlib among them).
6. **Dynamic types** — a variable's type follows from its value, with no declarations.
7. **Paradigm-flexible** — procedural, object-oriented and functional styles are all available.

Where Python already works in this field: analysing pharmacokinetic datasets (Cmax, Tmax, AUC), automating dose and dilution calculations, plotting dissolution profiles and concentration–time curves, managing large drug and ADR databases, machine learning for drug discovery and QSAR, and even simple hospital billing or stock software. Everything on this list is within reach of this course.

1.5 A short history of Python

Year	Event
Late 1980s	Guido van Rossum begins Python at CWI, Netherlands, as a successor to the ABC language
1991	Version 0.9.0 released — classes, functions, lists, dictionaries already present
1994	Version 1.0 adds functional tools (`lambda`, `map`, `filter`)
2000	Version 2.0 — served science for 20 years; retired in January 2020
2008	Version 3.0 — the modern, cleaned-up Python (this book uses Python 3)
Today	Python 3.13+ — still the world's most used language

Fun fact. Python is not named after the snake. Guido van Rossum was a fan of the British comedy group *Monty Python's Flying Circus*, and wanted a name that was short, unique and slightly mysterious.

1.6 Installing Python

1. Open **python.org/downloads**. The prominent yellow button names the current release for your OS (for example, *Download Python 3.14.8*).
2. Run the downloaded file. On the **first screen of the installer, tick the box "Add python.exe to PATH"** before clicking *Install Now*. This single checkbox lets Windows find Python from any terminal — forgetting it is the most common installation mistake.
3. Verify the installation. Press `Win + R`, type `cmd`, press Enter, and in the black window type:

```
python --version
```

```
OUTPUT
Python 3.14.8
```

Any `Python 3.x` answer means success. The companion command `pip --version` checks **pip**, Python's package installer, which arrives together with Python.

1.7 Anaconda — the all-in-one option

Anaconda (anaconda.com/download) installs Python *plus* about 250 data-science tools in one download — including Jupyter Notebook, Pandas, NumPy and Matplotlib, exactly the toolkit for Units III–V. It is a large download (about 1 GB), but it saves many individual installations later. During setup choose *Install for: Just Me* and leave all other options at their defaults.

Two habits make Anaconda painless: open **Anaconda Prompt** (from the Start menu) instead of a plain terminal — inside it, `python`, `pip` and `jupyter` always work — and use **Anaconda Navigator** when you prefer clicking tiles to typing commands.

1.8 IDEs — where you will type code

An **IDE (Integrated Development Environment)** is a smart editor for code. The three we use in this course:

- **Jupyter Notebook** — runs in your web browser; code, notes, results and charts live together on one page. This is our classroom tool, and the natural home for data work.
- **VS Code** (code.visualstudio.com) — a light, free editor from Microsoft for every language; add the *Python* extension and it becomes a full Python environment.
- **PyCharm** (jetbrains.com/pycharm) — a professional Python IDE; the unified product's core is free forever, and the Pro edition is free for students with a college email.
- **IDLE** — Python's own built-in basic IDE; it arrives with every Python install and is fine for first experiments, though it lacks the comforts above.

IDE	Nature	Best used for	Cost
Jupyter Notebook	Browser-based, cell-by-cell	Data analysis, learning, step-by-step PK work	Free (part of Anaconda)
PyCharm	Full professional IDE	Large projects, application development	Core free; Pro paid (free for students)
VS Code	Lightweight, customisable editor + extensions	General coding and scripting	Free
IDLE	Minimal built-in IDE	Simple scripts, quick tests	Free (comes with Python)

To start Jupyter from Anaconda Prompt:

```
jupyter notebook
```

A browser tab opens at `localhost:8888` showing your folders (the *dashboard*). Choose **New → Python 3 (ipykernel)** and an empty notebook appears — its empty box is called a **cell**.

1.9 Why we never write Python in a plain text editor

You *can* write Python in Notepad, but nobody does. An IDE adds:

Plain text editor	IDE (Jupyter / VS Code / PyCharm)
Everything is plain black text	Colours mark code parts — mistakes stand out
Save the file, open a terminal, run it by hand	One click (or `Shift+Enter`) runs your code
You hunt for errors line by line	Errors are shown with the exact line number
No help while typing	Auto-complete finishes names for you
No debugging aids	Breakpoints and step-through debugger
Results appear somewhere else	Charts and tables appear directly below the cell
Version control by hand	Integrated Git support in full IDEs

IDE = smart editor + runner + error helper, in one place.

1.10 Your first program

In a fresh Jupyter cell, type:

```
print("Hello, Pharmacy!")
```

```
OUTPUT
Hello, Pharmacy!
```

Press **Shift + Enter**. That is the whole rhythm of Jupyter: *type → Shift+Enter → see the answer below the cell.* (Useful to know: **Ctrl + Enter** runs the cell but keeps the cursor in it — handy when you want to tweak and re-run repeatedly; **Shift + Enter** runs and moves to the next cell.) Note the quotation marks — they mark text (a **string**). Without them, Python would look for a variable called `Hello` and complain.

A second cell — a small taste of what this unit builds:

```
dose_mg = 500
weight_kg = 70
print(dose_mg / weight_kg, "mg per kg")
```

```
OUTPUT
7.142857142857143 mg per kg
```

1.11 Troubleshooting your installation

Problem	Cause and cure
`'python' is not recognized…`	The PATH box was not ticked. Re-run the installer and choose *Modify*, or reinstall with the box ticked.
Microsoft Store opens when you type `python`	Settings → Apps → Advanced app settings → App execution aliases → turn **off** both `python.exe` aliases.

Problem	Cause and cure
`'jupyter' is not recognized`	Run it from Anaconda Prompt, or (plain-Python users) install it: `pip install notebook`.
An older Python answers	The Store copy is winning; apply the aliases fix above. Use `where python` to see all installed copies.
Packages installed but not found	You may be mixing a python.org install with Anaconda — each has its own package store. Pick **one** and stick with it (Anaconda is recommended for this course); if you must keep both, always run from **Anaconda Prompt** so the correct Python is used.

Chapter summary. A program is a list of instructions; Python executes them line by line, interactively or from a script. Python is free, dominant in science, and the natural tool for pharmaceutical data. Install it from python.org (tick *Add to PATH*) or take the all-in-one Anaconda route. Work in Jupyter. The IDE colours your code, runs it with one keystroke and points at errors — that is why we never use plain text editors.

Quick checks 1. (a) Explain the difference between interactive and script mode. (b) What does the "Add python.exe to PATH" checkbox do? (c) Give three advantages of an IDE over Notepad. (d) What key combination runs a Jupyter cell?

Chapter 2 — Variables, Data Types and Type Casting

2.1 What is a variable?

A **variable** is a labelled box in the computer's memory. You store a value in it and use the label whenever you need the value. Formally, the assignment `name = value` does three things: Python creates the value (if needed), reserves the label `name`, and **binds** the label to the value. Nothing more — no type declaration, no memory paperwork. That is why creating a variable takes a single line:

```
drug = "Paracetamol"
dose_mg = 500
weight_kg = 70
print(drug, dose_mg, "mg")
```

OUTPUT

```
Paracetamol 500 mg
```

The = sign does **not** mean "equals". It means *"put the right side into the box on the left"*. Read `dose_mg = 500` aloud as "dose_mg **becomes** 500".

Several consequences follow from the binding idea:

Reassignment replaces. A variable holds one value at a time; assigning again simply re-binds the label, and the old value is gone.

```
tablets = 100
tablets = tablets - 10
print(tablets)
```

OUTPUT

```
90
```

Line 2 looks odd in arithmetic; in Python it means "compute the right side using the current value (100 − 10), then store the result back."

Dynamic typing. The *value* has a type; the *name* does not. The same label may hold a number today and text tomorrow (`dose = 500`, later `dose = "500 mg"`). Convenient — but it means *you* must keep track of what is in the box, which is what `type()` (§2.4) is for.

Multiple assignment. Python can fill several boxes in one line — including the famous swap:

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

OUTPUT

```
2 1
```

Why variables matter. They let you write a value once and reuse it everywhere; update it in one place and every result follows; give calculations meaningful names (`dose_mg / weight_kg` reads like the formula itself); and store results for later use. This is exactly what a spreadsheet's cell references do — and programs go further, because a variable can also hold entire datasets (Unit III) or plots (Unit V).

2.2 Rules for naming variables

1. **Letters, digits and underscore only**: `dose_mg` ✓, `price2` ✓.
2. **Never start with a digit**: `dose2` ✓, `2dose` ✗.
3. **No spaces or hyphens**: `total dose` ✗ (splits in two), `total-dose` ✗ (reads as subtraction) — use `total_dose`.
4. **Case-sensitive**: `Dose`, `dose` and `DOSE` are three different variables. Using one you never created raises a `NameError`.
5. **Keywords are reserved**: `if`, `for`, `class`, `True`, … belong to Python's grammar and cannot be reused. (In a code editor they change colour — if a name you invented turns green or bold, it is taken.)

Name	Valid?	Reason
`dose_mg`	✓	letters + underscore
`_temp`	✓	may start with underscore
`batch_no_2026`	✓	digits allowed after the first letter
`2dose`	✗	starts with a digit
`total-dose`	✗	hyphen means minus
`for`	✗	keyword

Style. Legal is not enough — names should explain themselves. Python's convention is **snake_case**: lowercase words joined by underscores. Compare `r = d / x` with `mg_per_kg = dose_mg / weight_kg`; the second needs no comment. Future-you, reading notebooks at midnight before exams, is the audience.

2.3 The four basic data types — theory

Every value in Python has a **type**, and the type decides what the value can do: numbers take part in arithmetic, text can be sliced and searched, Booleans drive decisions. Python decides the type from *how you write the value* — quotes make text, a decimal point makes a float, bare digits make an integer, and `True`/`False` spelled exactly so make a Boolean.

(a) int — whole numbers. Integers are exact: there is no size limit in Python (a strength inherited from its design — `10 ** 100` computes without complaint), and integer arithmetic never loses

anything. Rules of the road: no decimal point (`70` is an int, `70.0` is not), no thousand separators (`1,00,000` is invalid — write `100000`), negatives are fine (`-12`).

Five examples:

```
tablets = 500
print(tablets + 100)          # addition of ints stays int
age = 21
print(age * 2)
deficit = -12
print(abs(deficit))           # abs() drops the sign
big = 1000000                 # commas are not allowed
print(big * 1000)
print(17 // 5, 17 % 5)        # integer results from integer inputs
```

OUTPUT

```
600
42
12
1000000000
3 2
```

Pharmacy integers: tablet counts, patient ages, pack strengths (a 500 mg tablet is whole milligrams), stock units.

(b) float — decimal numbers. Floats represent measurements — quantities that come from the real world and are never exact. Internally they are stored in *binary* scientific notation (the IEEE 754 standard), which has a beautiful consequence you must meet early: fractions that are tidy in decimal (like 0.1) have no exact binary equivalent, so tiny rounding wigs appear:

```
print(0.1 + 0.2)
```

OUTPUT

```
0.30000000000000004
```

This is normal, not an error — every language with binary floats does it — and Unit III shows the practical cure (`round`). Other rules: the dot *makes* the float (`70` int, `70.0` float); the comma is never a decimal separator; very large/small values may use scientific notation (`1.5e3` is 1500.0).

Five examples:

```
temp = 37.5
print(temp + 0.7)
print(7 / 2)                  # / ALWAYS produces a float, even for ints
print(0.1 + 0.2)              # the binary-storage curiosity
print(1.5e3)                  # scientific notation
price = 32.50
print(price * 3)
```

```
OUTPUT
38.2
3.5
0.30000000000000004
1500.0
97.5
```

Pharmacy floats: weights, temperatures, volumes, concentrations, half-lives, prices.

(c) str — text (strings). A string is an *ordered sequence of characters* — beads on a thread — enclosed in single or double quotes (both are equal; pick one style). Inside the quotes anything goes: letters, digits, spaces, symbols — which is why `"500 mg"` and even `"123"` are text. Useful escape sequences let one character line stand for another:

Escape	Means
`\n`	new line
`\t`	tab
`\\`	a real backslash
`\'` / `\"`	a quote inside quotes

Long text can span lines with triple quotes. Strings are **immutable** — more in Chapter 4.

Five examples:

```
print("Dose: 5 mg")            # double quotes
print("It's a generic")        # apostrophe inside double quotes
print("Line 1\nLine 2")        # \n starts a new line
label = """Shake
well"""
print(label)
print(len("500 mg"))           # spaces count: 6 characters
```

```
OUTPUT
Dose: 5 mg
It's a generic
Line 1
Line 2
Shake
well
6
```

Pharmacy strings: drug names, batch IDs, blood groups, unit labels, barcodes.

(d) bool — truth values. A Boolean has exactly two values, `True` and `False` — capital first letter, no quotes (quoted `"True"` would be text!). Booleans are *produced* by comparisons and logical operators and *consumed* by decisions (`if`, Unit II). Two theoretical notes worth knowing: `bool` is actually a subtype of `int` (`True` behaves as 1, `False` as 0 — `True + True` is `2`); and every value in

Python has a truthiness — zero numbers, empty text and empty containers count as `False`, everything else as `True`.

Five examples:

```
print(10 > 5)                  # comparisons PRODUCE bools
print(3 != 3)
expired = False
print(not expired)             # logic flips a bool
print("A" == "A")
print(bool(0), bool(7))        # truthiness of numbers
```

OUTPUT

```
True
False
True
True
False True
```

Pharmacy Booleans: `expired`, `in_stock`, `prescription_ok` — every checklist question is one.

(These four basic types are Unit I's business. Python also has a numeric type `complex` — numbers with a real and an imaginary part — and the collection types `list`, `tuple`, `dict` and `set`, which store many values inside one variable. They belong to Unit III and are only name-dropped here so that the family photograph is complete.)

2.4 Checking a type: type()

`type()` is the stethoscope of this course — whenever a result looks wrong, check the type first. It works on values and on variables alike, because variables simply hold values:

```
print(type(500))
print(type(37.5))
print(type("A+"))
print(type(True))
x = 98.6
print(type(x))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'float'>
```

2.5 The classic trap: `500` vs `"500"`

```
print(500 / 70)
print("500" + 100)
```

```
OUTPUT
7.142857142857143
TypeError: can only concatenate str (not "int") to str
```

Same digits, different types. Quotes are not decoration — they change what a value **is**. A number stored as text cannot take part in arithmetic until it is converted, which is the business of the next section.

2.6 Type casting — converting on purpose

Type casting means converting a value from one type to another. Python distinguishes two directions of conversion:

- **Implicit conversion** — Python widens automatically when no information would be lost. Mixing an `int` with a `float` in arithmetic yields the wider type, silently:

```
result = 100 + 0.5      # int + float
print(result, type(result))
```

```
OUTPUT
100.5 <class 'float'>
```

- **Explicit conversion (casting)** — *you* order the change with the four converter functions. They are called *constructors*: `int("500")` reads the text, sees a whole number, and builds the matching integer. Because Python refuses most silent conversions (text never becomes a number by accident), explicit casting is a statement of intent — one of the language's safety features.

int() — to whole numbers (five examples):

```
print(int("500"))      # numeric text -> whole number
print(int(98.6))       # CUTS toward zero (never rounds)
print(int(-3.9))       # -3, not -4
print(int(True))       # True counts as 1
# int("4.5")           # ValueError - the dot makes it non-integer text
```

```
OUTPUT
500
98
-3
1
```

float() — to decimals (four examples):

```
print(float("70.5"))   # numeric text with a dot
print(float(70))       # int gains a dot
print(float("500"))    # whole-number text is fine too
print(float(True))
```

```
OUTPUT
70.5
70.0
500.0
1.0
```

str() — to text (four examples):

```
print(str(500))
print(str(37.5))
print(str(True))
print("Dose: " + str(500) + " mg")   # the pre-f-string way to build messages
```

```
OUTPUT
500
37.5
True
Dose: 500 mg
```

bool() — to truth values (two examples, one per outcome):

```
print(bool(0), bool(0.0), bool(""))    # falsy: zero and empty
print(bool(7), bool(0.5), bool("tablet"))   # everything else is True
```

```
OUTPUT
False False False
True True True
```

Why casting matters daily. Everything a user types arrives as text — even numbers:

```
weight = input("Weight in kg: ")   # user types 70
type(weight)
```

```
OUTPUT
<class 'str'>
```

So the everyday idiom casts at the door, in one line:

```
weight = float(input("Weight in kg: "))
```

Three warnings.

1. `int()` **cuts**, never rounds: `int(98.6)` → `98`. (When you truly want rounding, `round(98.6)` → `99`.)
2. The text must *look like* the target type: `int("500")` ✓, but `int("Paracetamol")` and `int("4.5")` both raise `ValueError` — the dot makes `"4.5"` non-integer text; convert via `float("4.5")` first.
3. `float()` always decorates: `float(70)` → `70.0`. Harmless — and it explains the mystery `.0` that appears in your answers.

2.7 Comments — notes to the reader (and to yourself)

A **comment** is a note inside the code that Python completely ignores at run time. Comments exist for humans: they explain *why* something is written, mark section headings, or temporarily switch a line off during testing.

Two forms:

```
# Single-line comment: everything after the hash is ignored
dose_mg = 500          # trailing comments explain one line

"""
Multi-line note (often used at the top of a program):
This program calculates a weight-based paediatric dose.
"""
print(dose_mg)
```

OUTPUT

```
500
```

Style rule for this book: comment the *unit* of every pharmacy number (`dose_mg = 500    # mg per tablet`) — future-you reads the notebook at revision time and needs no decoder ring.

2.8 Worked example — paediatric syrup calculation

```
# Paracetamol syrup for a child: 15 mg/kg, syrup has 24 mg per mL
weight_kg = 22
mg_per_kg = 15
mg_per_ml = 24

total_mg = weight_kg * mg_per_kg
ml_needed = total_mg / mg_per_ml
print("Give", ml_needed, "mL")
```

OUTPUT

```
Give 13.75 mL
```

Change `weight_kg` to 30 and re-run: `18.75 mL`. The formula lives in code; only the values change.

Chapter summary. Variables are labels bound to values by `name = value`; names follow five rules and snake_case style. Values carry one of four basic types — `int` (exact wholes), `float` (binary decimals with the 0.1 quirk), `str` (immutable character sequences), `bool` (truth values, secretly numbers) — inspectable with `type()`. Quotes and decimal points decide the type, and the constructors `int()/float()/str()/bool()` convert between them. Since `input()` always delivers text, casting is not optional — it is the door every number passes through.

Quick checks 2. (a) Why is `2dose` invalid? (b) What does *dynamic typing* mean? (c) `type("98.6")`? (d) `int(9.9)` vs `round(9.9)`? (e) Write one line that asks for a patient's age in years and stores it as an `int`.

Chapter 3 — Operators, Input and Output

3.1 The seven arithmetic operators — theory

An **operator** is a symbol that combines values (its *operands*) into a new value. Python's seven arithmetic operators look like mathematics, with two pharmacy-friendly additions. The essential theory:

- Operands steer the result: **integer op integer stays integer** — with the single exception of `/`, which *always* returns a float, because real division so often needs decimals.
- `//` (floor division) gives the whole-number part of a division; `%` (modulo) gives what is left over. Together they split any count into *wholes and remainders*.
- `**` is exponentiation and works with fractional powers (`0.5 ** 3`), which is exactly what pharmacokinetics needs.
- Operator precedence follows mathematics (`**` binds tighter than `*`, which binds tighter than `+`), and brackets override everything.

Operator	Meaning	Example	Result
`+`	add	`500 + 50`	`550`
`-`	subtract	`70 - 5`	`65`
`*`	multiply	`10 * 30`	`300`
`/`	divide (always float)	`7 / 2`	`3.5`
`//`	floor division (whole part)	`43 // 10`	`4`
`%`	remainder	`43 % 10`	`3`
`**`	power	`2 ** 3`	`8`

Five examples with pharmacy meaning:

```
print(500 + 50)        # total after adding a half-strip
print(140 - 55)        # the Cockcroft-Gault numerator (140 - age)
print(22 * 15)         # mg for a 22 kg child at 15 mg/kg
print(330 / 24)        # mL of a 24 mg/mL syrup
print(0.5 ** 3)        # three half-lives -> one-eighth of the dose remains
```

OUTPUT

```
550
85
330
13.75
0.125
```

The pharmacy pair `//` and `%` deserves its own worked example. A patient needs 43 tablets; strips hold 10:

```
total = 43
strips = total // 10      # 4 full strips
loose = total % 10        # 3 loose tablets
print(strips, "strips +", loose, "loose")
```

OUTPUT

```
4 strips + 3 loose
```

Check: 4 × 10 + 3 = 43. The same pair splits minutes into hours (`95 // 60` → 1, `95 % 60` → 35) and packs into cases (`100 // 24` → 4 full cases, `100 % 24` → 4 short).

3.2 Operator precedence — who goes first?

Python evaluates in the order `**` → `* / // %` → `+ -`, then comparisons, then `not`, `and`, `or`. Equal-priority operators evaluate left to right. When in doubt, use brackets — they never lie and they document intent:

```
print(2 + 3 * 4)        # 14 - multiplication first
print((2 + 3) * 4)      # 20 - brackets first
```

OUTPUT

```
14
20
```

The Mosteller body-surface-area formula in Chapter 5 is written `math.sqrt(height_cm * weight / 3600)` — brackets (and precedence) make the fraction explicit.

3.3 Comparison operators — theory

Six operators compare two values and answer a **Boolean** — they are questions, not arithmetic. Theory worth knowing: comparisons *chain* naturally in Python (`0 < stock < 10` means both must hold, as in mathematics); equality (`==`) compares *values*, not appearance (`3 == 3.0` is `True` — same value, different type); and text comparisons use dictionary order (`"A" < "B"` is `True`), capital letters sorting before lowercase.

Operator	Question	Example	Result
`==`	equal to?	`5 == 5`	`True`
`!=`	not equal to?	`5 != 3`	`True`
`>`	greater than?	`37.5 > 37`	`True`
`<`	less than?	`stock < 50`	depends
`>=`	greater or equal?	`dose >= 500`	`True` for 500
`<=`	less or equal?	`age <= 12`	child dose

Five examples:

```
temp = 37.5
print(temp > 37.5)       # > EXCLUDES the boundary
print(temp >= 37.5)      # >= INCLUDES it
stock = 8
print(stock < 10)
print("A" != "B")
print(0 < stock < 10)    # chained: both must hold
```

OUTPUT

```
False
True
True
True
True
```

Two cautions. **`=` stores, `==` asks** — writing `=` inside a condition is a syntax error. And **boundaries are policy**: a pass mark of exactly 50 passes under `>=` and fails under `>`; a fever cut-off of 37.5 °C is "not yet fever" under `> 37.5`. Choose the operator deliberately.

3.4 Logical operators — and, or, not

Logical operators combine Booleans into one verdict. Theory: `and` needs **every** side True; `or` needs **at least one**; `not` flips. Python also *short-circuits*: in `A and B`, if `A` is already False, `B` is never even evaluated — useful when the second check would be unsafe (Unit II uses this).

Five examples — a dispensing checklist in pieces:

```
in_stock = True
rx_ok = True
expired = False
print(in_stock and rx_ok)          # both must hold
print(in_stock and not expired)    # condition + flip
print(expired or not rx_ok)        # either red flag
print(not expired)                 # plain flip
print(not in_stock or rx_ok)       # mixed: at least one side True
```

OUTPUT

```
True
True
False
True
True
```

Read the whole line `in_stock and rx_ok and not expired` aloud — it *is* the pharmacy's dispensing rule, written in code.

3.5 Assignment shortcuts

Augmented assignment folds “operate on yourself” into one operator — `x += 1` is exactly `x = x + 1`. Five examples:

```
tablets = 90
tablets += 10            # -> 100
stock = 20
stock -= 1               # -> 19
price = 32.50
price *= 1.05            # -> 34.125
dose = 500
dose /= 2                # -> 250.0  (note the float!)
shelf = 24
shelf //= 2              # -> 12
print(tablets, stock, price, dose, shelf)
```

OUTPUT

```
100 19 34.125 250.0 12
```

These forms appear constantly inside loops (Unit II), where a counter grows or stock shrinks once per pass.

3.6 input() — asking the user

`input()` pauses the program, shows a **prompt**, and returns whatever the user typed — **always as a string**, because keyboards produce text. The prompt is your question; the cast belongs in the same line:

```
name = input("Patient name: ")
weight = float(input("Weight in kg: "))
```

The program waits at each line until Enter is pressed. (Unit II turns this into whole question-and-answer programs; for now, one-line reads suffice.)

3.7 print() — more than it looks

`print()` converts each of its arguments to text, joins them, and adds a new line at the end. Two optional dials: `sep` (what goes *between* values — default one space) and `end` (what goes *after* — default a new line).

```
print("Dose", 500, "mg")          # several values, space-separated
print("A", "B", "C", sep="-")     # sep = what goes BETWEEN
print("next: ", end="")           # end="" -> no newline
print("room 3")                  # continues the same line
```

```
OUTPUT
Dose 500 mg
A-B-C
next: room 3
```

3.8 f-strings — printing like a human

Put an `f` before the quotes and the string becomes a *template*: inside curly braces `{ }`, Python evaluates an expression and inserts its result. Everything after a colon inside the braces is a **format specification** — a dressing instruction for display only:

Spec	Effect	Example	Shows
`:.1f`	one decimal	`f"{24.22:.1f}"`	`24.2`
`:.2f`	two decimals	`f"{7.142857:.2f}"`	`7.14`
`:6`	width 6, right-align	`f"\`	{500:6}\
`:<12`	width 12, left-align	`f"[{drug:<12}]"`	`[Paracetamol ]`

Five examples:

```
drug = "Paracetamol"
dose = 500
print(f"{drug} {dose} mg")
print(f"Double: {dose * 2}")            # expressions allowed
print(f"Per kg: {dose / 70:.2f} mg")
print(f"|{dose:6}|")                    # column alignment
print(f"[{drug:<12}]")                  # label alignment
```

```
OUTPUT
Paracetamol 500 mg
Double: 1000
Per kg: 7.14 mg
|   500|
[Paracetamol ]
```

The dressing changes only what is *printed* — the stored value is untouched (`round(x, 2)` is the separate function that rounds the value itself). Forgetting the `f` prints the braces literally: `print("{dose} mg")` → `{dose} mg`.

3.9 Four complete pharmacy programs

Program 1 — BMI calculator.

```
weight = float(input("Weight in kg: "))
height_m = float(input("Height in m: "))
bmi = weight / (height_m ** 2)
print(f"Your BMI is {bmi:.1f}")
```

```
OUTPUT
Weight in kg: 70
Height in m: 1.7
Your BMI is 24.2
```

Program 2 — dose per kilogram.

```
dose = float(input("Dose in mg: "))
weight = float(input("Weight in kg: "))
print(f"Dose per kg: {dose / weight:.2f} mg")
```

```
OUTPUT
Dose in mg: 500
Weight in kg: 70
Dose per kg: 7.14 mg
```

Program 3 — creatinine clearance (Cockcroft–Gault). CrCl (mL/min) = (140 − age) × weight / (72 × SCr); multiply by 0.85 for female patients.

```
age = int(input("Age in years: "))
weight = float(input("Weight in kg: "))
scr = float(input("Serum creatinine (mg/dL): "))
sex = input("Sex (M/F): ").strip().upper()

factor = 1.0
if sex == "F":
    factor = 0.85

crcl = (140 - age) * weight / (72 * scr) * factor
print(f"Creatinine clearance: {crcl:.1f} mL/min")
```

```
OUTPUT
Age in years: 55
Weight in kg: 70
Serum creatinine (mg/dL): 1.0
Sex (M/F): M
Creatinine clearance: 82.6 mL/min
```

(The `if` is a small preview of Unit II.)

Program 4 — child dose, two ways. Clark's rule uses weight; Young's rule uses age:

```
age = int(input("Child age in years: "))
weight = float(input("Child weight in kg: "))
adult_dose = float(input("Adult dose in mg: "))

clark = (weight / 70) * adult_dose
young = (age / (age + 12)) * adult_dose
print(f"Clark (weight): {clark:.1f} mg")
print(f"Young (age)  : {young:.1f} mg")
```

```
OUTPUT
Child age in years: 6
Child weight in kg: 20
Adult dose in mg: 500
Clark (weight): 142.9 mg
Young (age)  : 166.7 mg
```

Two formulas, one shape — *fraction × adult dose*. Notice the brackets in `age / (age + 12)`: without them the answer would be wrong. *In practice, mg/kg dosing is preferred; these rules give quick estimates.*

Chapter summary. Seven arithmetic operators (operands steer the result; `/` always floats; `//` and `%` split counts into wholes and remainders), precedence with brackets as the honest fix, six comparisons that manufacture Booleans (chaining allowed; boundaries are policy), and `and`/`or`/`not` with short-circuit evaluation. `input()` asks (returning text — cast immediately); `print()` reports with `sep`/`end` dials; f-strings evaluate expressions in braces and dress numbers with `:.1f`/`:.2f`/width specs. The four programs of §3.9 are worth retyping until they feel natural — they are the skeleton of every calculation you will ever automate.

Quick checks 3. (a) `43 // 10` and `43 % 10`? (b) Why does `7 / 2` give `3.5` but `7 // 2` gives `3`? (c) `50 >= 50` vs `50 > 50`? (d) What do `sep=` and `end=` control? (e) Produce `Per kg: 7.14` from `dose = 500` in one f-string.

Chapter 4 — String Operations and Manipulation

4.1 The bead-on-a-thread model — theory

A string is not one object; it is an **ordered sequence** of characters — beads on a thread. Each bead has a position number, positions start at **zero**, and every string-processing skill in this chapter (indexing, slicing, searching) is just different ways of addressing those beads. Two consequences: `len()` counts the beads (spaces included); and the sequence cannot be edited in place — strings are **immutable**, because Python shares text everywhere (labels, files, the internet) and protects it by never changing it after creation. Every "change" you will ever make is really the creation of a *new* string.

```
drug = "Paracetamol"
print(len(drug))
print(len("A+"))
print(len("500 mg"))     # the space counts
```

OUTPUT

```
11
2
6
```

4.2 Indexing — every bead has a number

Square brackets address a single character. Python counts **from zero**:

```
P   a   r   a   c   e   t   a   m   o   l
0   1   2   3   4   5   6   7   8   9   10
```

Negative indexes count from the end — a genuine convenience when you do not know the length. *Five examples:*

```
drug = "Paracetamol"
print(drug[0], drug[4])          # P c
print(drug[-1], drug[-2])        # l o
print(len(drug) - 1)             # the last valid index
print(drug[len(drug) - 1])       # the last character, computed
# drug[11]                        # IndexError - only 0..10 exist
```

OUTPUT

```
P c
l o
10
l
```

Reading past the last bead raises `IndexError` — the address exists, the bead does not.

4.3 Slicing — cutting out a piece

`[start : stop]` takes characters from `start` up to **but not including** `stop` — the same exclusion rule as `range()`. Omit `start` to begin at 0; omit `stop` to run to the end. Unlike indexing, an over-generous slice never crashes — it simply stops where the string stops.

Five examples — one batch ID, decoded:

```
batch = "MRP2026-114"
print(batch[0:3])       # 'MRP'   plant code
print(batch[3:7])       # '2026'  year (middle of the string!)
print(batch[8:])        # '114'   sequence
print(batch[:3])        # 'MRP'   missing start = from 0
print(batch[-4:])       # '-114'  from the 4th-last to the end
```

OUTPUT

```
MRP
2026
114
MRP
-114
```

Counting on fingers: `[3:7]` delivers positions 3, 4, 5, 6. One batch string, three facts — slicing *decodes* it, exactly the way barcodes and traceability codes work.

4.4 Immutability in action

```
drug = "Paracetamol"
drug[0] = "p"
```

OUTPUT

```
TypeError: 'str' object does not support item assignment
```

Strings are sealed. The workaround *is* the normal way — build a new string from pieces:

```
drug = "Paracetamol"
drug = "p" + drug[1:]
print(drug)
```

OUTPUT

```
paracetamol
```

This also explains the rule of the next section: **every string method returns a new string** and leaves the original untouched. Write `clean = raw.strip()`, not `raw.strip()` alone — the bare call computes and throws away.

4.5 Case and cleaning methods — upper, lower, title, strip

Theory: case methods map every letter through the Unicode case rules (digits and symbols pass through); `strip()` removes *whitespace* (spaces, tabs, newlines) from both ends — never from the middle; all of them return new strings.

Five examples:

```
name = "  pARACETamol  "
print(name.upper())              # shouty labels
print(name.lower())              # search keys
print(name.strip().title())      # clean AND case-fixed
print("vitamin b12".title())     # Vitamin B12
raw = "  Paracetamol  "
print(len(raw), len(raw.strip()), len(raw.lstrip()), len(raw.rstrip()))
```

OUTPUT

```
  PARACETAMOL
  paracetamol
Paracetamol
Vitamin B12
15 11 13 13
```

`strip()` pairs naturally with `input()`, which delivers stray spaces: `input("Drug: ").strip().title()` cleans a whole name in one line.

4.6 replace — swapping text

Theory: `replace(old, new)` substitutes **every** occurrence, left to right; an optional third argument limits the count; matching is case-sensitive and exact.

Four examples (the fourth is the safety lesson):

```
print("500 mg".replace("mg", "mcg"))       # a 1000-fold swap - handle with care
print("BC-2026-114".replace("-", ""))      # strip separators
print("aaa".replace("a", "b", 2))          # first 2 only
print("MG".replace("mg", "mcg"))           # case-sensitive: no match, no change
```

OUTPUT

```
500 mcg
BC2026114
bba
MG
```

A real mg→mcg swap is precisely how dispensing errors happen — the computer obeys exactly, so the pharmacist must choose the right operation.

4.7 Searching — find, count, in

Theory: `find()` returns the position of the **first** match (searching from the left) or **−1** when absent — never a crash; `count()` totals the matches; `in` is the plain-English membership test returning a Boolean, ready for Unit II's `if`.

Five examples:

```
drug = "Paracetamol"
print(drug.find("cetamol"))        # 4  - first match position
print(drug.find("xyz"))            # -1 - absent, no crash
print(drug.count("a"))             # 3  - positions 1, 3 and 7
print("cetamol" in drug)           # substring test
print("statin" in "Atorvastatin")  # drug-class suffix found
```

OUTPUT

```
4
-1
3
True
True
```

Drug-class suffixes (`-caine`, `-statin`, `-pril`) checked with `in` — cheminformatics in miniature.

4.8 Taking apart and joining — split, join

Theory: `split()` cuts a string into a **list** of pieces — at whitespace by default, at a separator you name otherwise; `join()` is its mirror, gluing a list of strings with one separator *between* them (the separator belongs to the string in front of `.join`). This pair is the mechanism behind CSV files (Unit III) and every "text → data" conversion.

Five examples:

```
print("500 mg".split())                    # at spaces
print("BC2026-114".split("-"))             # at a named separator
print("a,b,c".split(","))                  # CSV thinking
print(" + ".join(["Aspirin", "Paracetamol"]))   # glue with ' + '
print("\n".join(["Morning", "Night"]))     # glue with newlines
```

OUTPUT

```
['500', 'mg']
['BC2026', '114']
['a', 'b', 'c']
Aspirin + Paracetamol
Morning
Night
```

4.9 Joining and repeating — +, *, f-strings

The `+` operator glues strings (numbers must be dressed with `str()` first — or better, use an f-string); `*` repeats a string a whole number of times:

```
label = f"Dose: {500} mg"    # preferred - readable, computes inside { }
line = "-" * 30              # a 30-dash report divider
print(line)
```

OUTPUT

```
------------------------------
```

4.10 Worked programs

Program 1 — drug-name standardiser.

```
raw = input("Drug name: ")
clean = raw.strip().title()
print(f"Record : {clean}")
print(f"Label  : {clean.upper()}")
```

OUTPUT

```
Drug name:   aSPIRin
Record : Aspirin
Label  : ASPIRIN
```

Program 2 — dose-string parser.

```
dose_text = input("Dose (e.g. 500 mg): ")
parts = dose_text.split()
amount = float(parts[0])
unit = parts[1]
print(f"{amount / 70:.2f} {unit} per kg")
```

OUTPUT

```
Dose (e.g. 500 mg): 500 mg
7.14 mg per kg
```

Program 3 — batch-ID decoder.

```
batch = input("Batch ID: ")
print(f"Plant: {batch[:3]}")
print(f"Year : {batch[3:7]}")
print(f"Seq  : {batch[8:]}")
```

OUTPUT

```
Batch ID: MRP2026-114
Plant: MRP
Year : 2026
Seq  : 114
```

Chapter summary. Strings are immutable sequences: `len()` counts, `[i]` addresses (from zero; `[-1]` is last; `IndexError` past the end), `[a:b]` slices (stop excluded, over-runs forgiven) and decodes batch IDs. Methods return new strings: `upper/lower/title` re-case, `strip/lstrip/rstrip` trim edges, `replace` swaps (case-sensitive, all matches), `find/count/in` search without crashing, `split/join` take apart and reassemble — the bridge from text to data. `"500 mg"` becomes a number and a unit in two lines.

Quick checks 4. (a) `"Cetirizine"[:4]`? (b) `"Amoxicillin".count("i")`? (c) `"  Metformin ".strip()`? (d) Why does `drug[0] = "p"` fail, and what is the workaround? (e) `"BC2026-114".split("-")`?

Chapter 5 — Libraries: Standard, Third-Party and pip

5.1 What is a library? — theory

A **library** (often called a *module* when it is a single file) is a box of ready-made code that someone else wrote, tested and shared. `import` brings the box into your program and gives you access to everything inside through its name and a dot: `math.sqrt` means "the `sqrt` function that lives in the `math` module". This dot-addressing is called a *namespace* — it prevents name collisions, which is why your variable `pi` never disturbs `math.pi`.

Two kinds of library:

- **Standard library** — ships *with* Python; nothing to install, works offline. Over 200 modules: `math`, `datetime`, `random`, `statistics`, …
- **Third-party libraries** — built by the worldwide community and downloaded from **PyPI**, the Python Package Index hosting more than 400,000 packages. NumPy, Pandas, Matplotlib (Units III–V) and RDKit (drug discovery) all live here.

The dividing line matters for installation only — after import, both feel identical.

5.2 Three ways to import

```
import math                    # whole module: math.sqrt(16) -> 4.0
from math import sqrt          # just one tool: sqrt(16) -> 4.0
import numpy as np             # a nickname - the data-science convention
```

Style guidance: `import module` is clearest for beginners; `from module import name` saves typing for one or two heavily used functions; aliases (`as np`) shine when a module's name is used constantly — by Unit IV you will write `import pandas as pd` from muscle memory.

5.3 The math module

Theory: `math` holds the classical mathematical functions and constants, all operating on floats with full precision. Its two rounding cousins deserve special attention: `ceil` always rounds **up** (toward +∞) and `floor` always rounds **down** — neither *truncates toward zero* the way `int()` does on negatives.

Five examples:

```
import math
print(math.sqrt(16))            # 4.0 - always a float
print(math.sqrt(2))             # full precision
print(math.ceil(7.14), math.floor(7.8))
print(math.ceil(-7.14))         # -7, toward +infinity (int(-7.14) is -7 too
here; floor(-7.14) is -8)
print(round(math.pi, 4))
```

```
OUTPUT
4.0
1.4142135623730951
8 7
-7
3.1416
```

Pharmacy reading of `ceil`: when 7.14 packs of medicine are needed, you order **8** — never short the patient.

Worked program — BSA by the Mosteller formula (BSA in m² = √(height_cm × weight / 3600); BSA drives chemotherapy and paediatric dosing):

```
import math
height_cm = float(input("Height in cm: "))
weight = float(input("Weight in kg: "))
bsa = math.sqrt(height_cm * weight / 3600)
print(f"BSA = {bsa:.2f} m2")
```

```
OUTPUT
Height in cm: 170
Weight in kg: 70
BSA = 1.82 m2
```

(The formula needs height in **centimetres** — with metres the answer shrinks by a factor of ten and becomes clinically wrong. Units are policy.)

5.4 The datetime module — dates that calculate

Theory: a `date` object stores year, month and day as one value that knows the calendar. That is why dates *compare* with `>`, `<` like numbers and *subtract* into a duration whose `.days` you can read. `date.today()` is your program's connection to the actual current day.

Five examples:

```
from datetime import date
mfg = date(2026, 1, 15)
expiry = date(2027, 3, 31)
print(expiry.year, expiry.month, expiry.day)
print((expiry - mfg).days)          # shelf life in days
print(expiry > mfg)                 # dates compare like numbers
print(date.today())                 # the run date
print(mfg.weekday())                # 0 = Monday ... 3 = Thursday
```

```
OUTPUT
2027 3 31
440
True
2026-11-11
3
```

A near-expiry alert in two lines:

```
from datetime import date
expiry = date(2027, 3, 31)
today = date.today()

print(expiry > today)              # True - safe to dispense
print((expiry - today).days, "days left")
```

```
OUTPUT
True
140 days left
```

(The day count depends on the day you run it.) Real pharmacies run precisely this check on every batch.

5.5 Two more for the road

random — unpredictable numbers for sampling and simulation (it returns properly in Unit II and beyond):

```
import random
print(random.randint(1, 6))        # a die roll: one of 1..6
print(random.random())             # a float between 0 and 1
drugs = ["Aspirin", "Dolo", "Brufen"]
print(random.choice(drugs))        # one of the three, unpredictably
random.shuffle(drugs)              # mixes the list in place
print(drugs)
```

(Typical output: `3` · `0.721349...` · `Dolo` · `['Brufen', 'Aspirin', 'Dolo']` — different on every run.)

statistics — the descriptive trio, quietly previewing Unit IV where Pandas computes these for entire columns:

```
import statistics
print(statistics.mean([2, 4, 6]))
print(statistics.median([1, 3, 100]))          # 3 - the middle value, immune to
outliers
print(statistics.mode(["tab", "cap", "tab"]))  # the most frequent
print(statistics.stdev([2, 4, 6]))             # sample standard deviation
```

```
OUTPUT
4
3
tab
2.0
```

5.6 pip — install, list, uninstall (syllabus point)

`pip` arrived with Python (§1.6). From any terminal or Anaconda Prompt:

```
pip install pandas          # download + install from PyPI
pip install numpy pandas matplotlib   # several at once
pip list                    # every library you have, with versions
pip show pandas             # details of one package
pip install --upgrade pandas
pip install pandas==2.2.0   # pin an exact version
pip uninstall pandas        # asks y/n, then removes
```

Notes. Installing needs the internet and happens **once per computer**; after that `import` works forever, offline too. Never delete library files by hand — always `pip uninstall`. Anaconda users may also use `conda install` / `conda list` / `conda remove` (Anaconda's own store); in this course `pip` is the rule because it works in both setups.

5.7 Worked program — expiry report, complete

```
from datetime import date

drug = input("Drug: ").strip().title()
mfg = date(2026, 1, 15)
expiry = date(2027, 3, 31)
shelf_days = (expiry - mfg).days

print(f"{drug}: shelf life {shelf_days} days")
if expiry > date.today():
    print("Status: in date")
```

```
OUTPUT
Drug: paracetamol
Paracetamol: shelf life 440 days
Status: in date
```

Chapter summary. Libraries are borrowed expertise reached through namespaces: standard-library modules arrive with Python, third-party packages arrive by `pip` from PyPI. Three import styles serve three habits. `math` brings `sqrt/ceil/floor/pi` (Mosteller BSA in three lines; `ceil` is the pharmacy-grade rounding); `datetime` makes dates comparable and subtractable (expiry checking, shelf life, weekday); `random` and `statistics` wait in the wings. The pip lifecycle — install, list, show, upgrade, pin, uninstall — is the syllabus's final Unit I skill.

Quick checks 5. (a) `math.ceil(7.2)` and `math.floor(7.2)`? (b) Two differences between standard and third-party libraries. (c) The command to remove a library? (d) Write the import line that brings only `sqrt` into your program. (e) How would you print the number of days between two `date` objects?

Unit I — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Installing Python and an IDE; IDE vs text editor	Ch 1 (§1.6–1.9)
Variables, data types, casting, operators, I/O	Ch 2–3
String operations and manipulation	Ch 4
Standard and third-party libraries; install/uninstall	Ch 5

Twenty-question exercise bank

Predict the output (1–8).

1. `print(43 // 10, 43 % 10)`
2. `s = "Paracetamol"; print(s[0], s[-1])`
3. `print(type("500"))`
4. `total = 5; total += 5; print(total)`
5. `print(f"{2 ** 3}")`
6. `print("A", "B", sep="+")`
7. `print(int(9.99))`
8. `print("Dolo".replace("Dolo", "Paracetamol"))`

Short answers (9–14).

9. State the five rules for naming variables, with one invalid example each.
10. Why does `input()` always require casting for arithmetic? Show the one-line idiom.
11. Explain `>=` vs `>` with a pass-mark example.
12. What is the difference between `int(9.9)`, `round(9.9)` and `math.ceil(9.9)`?
13. Why is `height_cm * weight / 3600` written with centimetres in the Mosteller program?
14. Give three advantages of an IDE over a text editor.

Write programs (15–20).

15. Ask two numbers; print sum, difference, product and quotient (2 decimals).
16. Ask a temperature in °C; print whether it is above 37.5 (preview of Unit II).
17. Batch decoder: read a batch ID like `HYD2027-042` and print plant, year, sequence.
18. Ask a drug name with messy spacing and case; print a clean title-case record and an UPPER-CASE label.
19. Days-supply: tablets in hand and per-day dose; print full days and leftovers (`//` and `%`).

20. Expiry checker: create `date(2028, 8, 31)` and print how many days remain from today, with an “in date / expired” line.

Answers to selected exercises

1. `4 3` — 4 strips, 3 loose. 2. `P l`. 3. `<class 'str'>`. 4. `10`. 5. `8`. 6. `A+B`. 7. `9` (cut, not rounded). 8. `Paracetamol`. 10. Because keyboard input arrives as text; `float(input("…"))`. 11. `>=` includes the boundary (50 passes); `>` excludes it. 12. `int` truncates to 9; `round` gives 10; `ceil` gives 10. 19. With 31 tablets and 2 per day: `31 // 2` → 15 days, `31 % 2` → 1 left.

Important points for examination

1. Guido van Rossum published the first public release in 1991. The language is *interpreted* (it runs line by line), *high-level* (it reads close to English) and *dynamically typed* (each value carries its own type). The name borrows from the Monty Python comedy troupe rather than the reptile.
2. While installing on Windows, tick **“Add python.exe to PATH”** on the first screen; afterwards `python --version` in a terminal confirms success.
3. An IDE bundles editor, debugger and one-click running — colour highlighting, name completion and instant error location that Notepad cannot offer. Jupyter suits stepwise data work; PyCharm suits big projects; VS Code is the light all-rounder; IDLE ships inside every Python install.
4. The Unit I quartet of types is int, float, str and bool — quotes make text, a decimal point makes a float. The containers (list, tuple, dict, set) wait until Unit III.
5. Name rules: letters, digits and underscore only; never digit-first; capitalisation matters; keywords are off-limits. snake_case is the house style.
6. Whatever the user types, `input()` hands back text — wrap it in `int()` or `float()` before any arithmetic. The converters are `int()`, `float()`, `str()` and `bool()`.
7. `int()` chops (98.6 → 98); `round()` rounds; `math.ceil()` rounds upward — when 7.14 packs are needed, order 8.
8. Operator families: arithmetic `+ - * / // % **` (true division always yields a float), comparisons `== != > < >= <=` (each yields a bool), logic `and or not`. Precedence runs `()` → `**` → `* / // %` → `+ -`, and brackets remain the final authority.
9. `>=` admits the boundary value; `>` rejects it — on pass marks and fever thresholds, that single character is a decision.
10. String positions begin at 0 (or at −1 counting from the rear); a slice `[start:end]` never includes `end`; and strings never change in place — methods hand back new strings.
11. The standard library needs only `import`; everything else arrives by `pip install` and departs by `pip uninstall`.

Sources for this unit: course decks Classes 1–5 and the author's classroom programs; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 1* notes, Yamuna Devi, Jeyabharathi & Vajpayee *Python Programming* (GCS) ch. 1–4, MCA R24 lecture notes Unit I, MRCET R17 notes Unit I, and the curated learn.biodockify.com catalog (trekhleb/learn-python, jackfrued/Python-100-Days, joaoventura/full-speed-python, gto76/python-cheatsheet, exercism/python). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source (zero 7-word overlaps). Full credits: see the book's Sources and Acknowledgements page.

UNIT II

Control Structures & Functions

Chapters 6–8 · 6 teaching hours

Chapter 6 — Conditional Statements

6.1 Why programs must decide

Every program written so far runs the same lines in the same order, every time. Real pharmacy work is not like that: a dose above the maximum must raise an alert and a dose within limits must not; a batch either clears quality control or it does not. A **conditional statement** tests a condition and lets the program choose which block of code to carry out — a Boolean expression that evaluates to `True` or `False`.

The decision vocabulary from Unit I already supplies the conditions: comparisons (`dose > 500`, `age <= 12`) and logic (`in_stock and not expired`). This chapter adds the *grammar* that acts on them.

6.2 The if statement — theory and anatomy

`if` executes its guarded block **only** on a True condition; otherwise the block is skipped and the program continues below it. Three syntax rules are absolute:

1. The condition line ends with a **colon**.
2. The guarded lines form an indented block underneath (four spaces; Jupyter adds them automatically after the colon).
3. **Indentation is not decoration in Python** — it *is* the block structure. Python uses indentation instead of curly braces; wrong indentation raises an `IndentationError`, and lines back at the margin belong to the surrounding program, not the `if`.

```
dose = 600
if dose > 500:
    print("High dose - check with the prescriber")
if dose > 0:
    print("Dose recorded")
print("Check complete")
```

OUTPUT

```
High dose - check with the prescriber
Dose recorded
Check complete
```

Read it aloud: *if* the dose exceeds 500 mg, warn; *if* it is positive, record it; *always* print "Check complete" (no indent — no guard). Each `if` is tested independently.

Five examples — independent guards:

```
temp_c = 36.9
if temp_c > 37.5:
    print("Fever detected")
print("Reading done")            # runs - it is outside the if
```

OUTPUT

```
Reading done
```

```
stock = 10
if stock < 20:
    print("LOW STOCK - reorder")
```

OUTPUT

```
LOW STOCK - reorder
```

```
expired = False
if expired:
    print("Quarantine this batch")
# nothing prints: False skips the block
```

```
if "statin" in "Atorvastatin":
    print("Lipid-lowering family")
```

OUTPUT

```
Lipid-lowering family
```

```
rx_ok, in_stock, expired = True, True, False
if rx_ok and in_stock and not expired:
    print("Dispense")
```

OUTPUT

```
Dispense
```

The last example is the pharmacy's dispensing checklist switched on: one line of decision logic replacing three manual checks.

6.3 if-else — two roads

`if-else` guarantees exactly one of two blocks runs: the first when the condition is True, the `else` block in every other case. The `else` line takes **no condition** — it is the catch-all. Picture a fork in the road: the condition is the signpost, and both roads rejoin afterwards.

```
temperature = 39.5
if temperature > 38:
    print("Fever detected")
else:
    print("Temperature normal")
```

OUTPUT

```
Fever detected
```

Five examples, with the boundary lesson embedded:

```
marks = 50
if marks >= 50:
    print("PASS")
else:
    print("FAIL - reattempt needed")
```

OUTPUT

```
PASS
```

```
marks = 49.9
if marks >= 50:
    print("PASS")
else:
    print("FAIL - reattempt needed")
```

OUTPUT

```
FAIL - reattempt needed
```

Q1 vs Q2 — one-tenth of a mark apart, opposite outcomes: `>=` **includes** the boundary value itself, `>` excludes it. On pass marks and fever thresholds, the choice of operator is policy, not style.

```
dose, max_dose = 650, 500.0
if dose > max_dose:
    print("Alert: exceeds the safe maximum")
else:
    print("Within the safe range")
```

OUTPUT

```
Alert: exceeds the safe maximum
```

```
bp = 118
if bp < 90:
    print("Low BP")
else:
    print("Acceptable BP")
```

OUTPUT

```
Acceptable BP
```

```
sex = "F"
if sex == "F":
    print("CrCl correction factor 0.85 applies")
else:
    print("No correction factor")
```

```
OUTPUT
CrCl correction factor 0.85 applies
```

6.4 if-elif-else — many roads, first True wins

When several mutually exclusive cases exist, chain them with `elif` (short for *else if*). Python evaluates the conditions **top to bottom** and runs the block of the **first** True condition; every later condition is skipped without being checked. A trailing `else` catches "none of the above".

```
marks = 72
if marks >= 70:
    grade = "Distinction"
elif marks >= 60:
    grade = "First Class"
elif marks >= 50:
    grade = "Pass"
else:
    grade = "Fail"
print(f"Grade: {grade}")
```

```
OUTPUT
Grade: Distinction
```

Trace ritual (an examination skill). For `marks = 65`, write T/F beside each condition: `65 >= 70` F (skip), `65 >= 60` **T** (run, then stop) — the `>= 50` test below is never even evaluated. For `marks = 55`: F, F, T → Pass. For `marks = 40`: F, F, F → the `else` net catches it → Fail.

Order matters — the silent bug. Put the loosest test first and it swallows everything:

```
marks = 88
if marks >= 50:
    grade = "Pass"           # 88 lands here and stops!
elif marks >= 60:
    grade = "First Class"    # dead code
elif marks >= 70:
    grade = "Distinction"    # unreachable
print(grade)
```

```
OUTPUT
Pass
```

No error message appears — the program simply answers wrongly. Rule for the notebook: **order conditions from the strictest to the loosest** (highest bar first, catch-all last).

Five examples:

```
bp = 145
if bp < 90:
    print("Low BP")
elif bp < 120:
    print("Normal BP")
elif bp < 140:
    print("Elevated BP")
else:
    print("High BP - consult doctor")
```

OUTPUT

```
High BP - consult doctor
```

```
age = 8
if age < 12:
    band = "Paediatric - weight-based dosing"
elif age < 65:
    band = "Adult - standard dosing"
else:
    band = "Geriatric - start low, go slow"
print(band)
```

OUTPUT

```
Paediatric - weight-based dosing
```

```
bmi = 24.2
if bmi < 18.5:
    category = "Low weight"
elif bmi < 25:
    category = "Normal"
elif bmi < 30:
    category = "Overweight"
else:
    category = "Obese"
print(category)
```

OUTPUT

```
Normal
```

```
severity = "moderate"
if severity == "mild":
    action = "Document and observe"
elif severity == "moderate":
    action = "Monitor closely, consider dechallenge"
elif severity == "severe":
    action = "Stop the drug, notify the physician"
else:
    action = "Unknown grade - review the report"
print(action)
```

```
OUTPUT
Monitor closely, consider dechallenge
```

```
scr = 2.8
if scr < 1.2: level = "Normal creatinine"
elif scr < 2.0: level = "Mildly raised"
else: level = "Markedly raised - renal review"
print(level)
```

```
OUTPUT
Markedly raised - renal review
```

Note the ADR example's `else` net: an unrecognized severity (a typo like `"Medium"`) lands safely in "Unknown grade" instead of being silently ignored — in pharmacovigilance, unhandled categories must never vanish.

6.5 Nested conditions — a question inside a question

A **nested condition** means an `if` sits inside the block of a further `if`. Use it when a second question only makes sense after the first is answered: the pharmacist verifies the prescription first and only then checks stock. Indentation deepens by four spaces per level — three levels (twelve spaces) is the practical limit before the logic should be reorganised.

```
rx_ok = True
in_stock = True
expired = False

if rx_ok:
    if in_stock:
        if not expired:
            print("DISPENSE")
        else:
            print("QUARANTINE - expired stock")
    else:
        print("OUT OF STOCK - reorder")
else:
    print("REFUSE - prescription required")
```

```
OUTPUT
DISPENSE
```

The funnel: each gate closes before deeper questions are asked. Flip `expired` to `True` and the quarantine branch fires without touching stock or prescription logic.

Nest or combine? If every branch needs a *different* message, nest. If one action covers a combination, combine with `and`: `if rx_ok and in_stock and not expired: print("DISPENSE")` — flatter, but with no room for separate replies.

The validate-first pattern. The most valuable nesting idiom in this course checks that input is usable *before* acting on it:

```
marks = 105
if marks < 0 or marks > 100:
    print("Invalid marks - enter 0 to 100")
else:
    if marks >= 50:
        print("PASS")
    else:
        print("FAIL")
```

OUTPUT

```
Invalid marks - enter 0 to 100
```

Invalid data gets one message and nothing else runs — the grade logic lives inside `else`, guaranteed to see only valid marks.

Age-and-weight dosing tree (symmetric nesting, four leaves):

```
age, weight = 8, 15
if age < 12:
    if weight < 20:
        dose = 100      # small child
    else:
        dose = 200      # older child
else:
    if age >= 65:
        dose = 400      # cautious senior dose
    else:
        dose = 500      # standard adult
print(f"Dose: {dose} mg")
```

OUTPUT

```
Dose: 100 mg
```

Hospital dosing protocols are written as exactly this kind of tree; a student who can trace it can read a guideline.

Chapter summary. `if` guards a block behind a True condition (colon + indent, no exceptions). `if-else` runs exactly one of two roads; the boundary rule (`>=` includes, `>` excludes) turns one character into policy. `elif` chains test top-down, first True wins, strictest-first ordering prevents silent wrong answers, and `else` is the safety net. Nesting puts a question inside a question — validate first, act second.

Quick checks 6. (a) What two things end an `if` line? (b) Output of the BP chain with `bp = 135`? (c) Why does `marks = 88` grade as "Pass" in the buggy chain? (d) When would you nest instead of using `and`? (e) Rewrite the dispensing gate so each failure has its own message.

Chapter 7 — Loops

7.1 Why loops exist

Typing `print("Check: Asha")` forty times for forty patients is not programming. A **loop** runs one block of code again and again — once per patient, per drug, per reading — while the program itself stays the same size. Python offers two: `for` (one pass per item of a known collection) and `while` (repeat as long as a condition holds).

7.2 The for loop — anatomy and the loop variable

```
patients = ["Asha", "Ravi", "Meena"]
for patient in patients:
    print("Checking:", patient)
print("Ward round complete")
```

OUTPUT

```
Checking: Asha
Checking: Ravi
Checking: Meena
Ward round complete
```

Anatomy: `for` and `in` are keywords; `patient` is the **loop variable** — a fresh box that Python fills with the next item before each pass; the collection follows `in`; the indented body runs once per item; lines back at the margin run once, after the loop finishes.

Trace table for the run above:

pass	patient holds	body prints
1	"Asha"	Checking: Asha
2	"Ravi"	Checking: Ravi
3	"Meena"	Checking: Meena
after	loop exhausted	Ward round complete

Any sequence can be walked. A string is a sequence of characters (Unit I's bead-on-a-thread model):

```
for letter in "ASPIRIN":
    print(letter)
```

```
OUTPUT
A
S
P
I
R
I
N
```

7.3 range() — counting without a list

`range()` manufactures a sequence of numbers. It counts **from zero** and **excludes the stop value** — the same two conventions as string slicing.

Form	Produces
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8 (step 2)

Five examples:

```
for i in range(5):
    print("Ward round", i)
```

```
OUTPUT
Ward round 0
Ward round 1
Ward round 2
Ward round 3
Ward round 4
```

```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
```

```
OUTPUT
10
```

```
for t in range(36, 40):
    print(t, "C")
```

```
OUTPUT
36 C
37 C
38 C
39 C
```

```
for n in range(10, 0, -2):
    print(n)
```

OUTPUT

```
10
8
6
4
2
```

```
for strip in range(1, 4):
    print("Blister strip", strip, "checked")
```

OUTPUT

```
Blister strip 1 checked
Blister strip 2 checked
Blister strip 3 checked
```

7.4 The accumulator and counter patterns

Two patterns power every calculation over a collection. **Accumulate**: start a total at zero *before* the loop, add each item *inside*, use the result *after*. **Count**: the same skeleton, but add 1 only when a condition holds.

```
doses = [500, 250, 500, 1000, 250]
total = 0
for dose in doses:
    total = total + dose
print(total)
```

OUTPUT

```
2500
```

```
temps = [36.5, 37.8, 38.2, 36.9, 37.1]
total = 0
for t in temps:
    total = total + t
print(f"Ward average: {total / len(temps):.1f} C")
```

OUTPUT

```
Ward average: 37.3 C
```

```
statuses = [False, True, False, True, True]
expired = 0
for status in statuses:
    if status:
        expired = expired + 1
print(expired, "expired batch(es)")
```

OUTPUT

```
3 expired batch(es)
```

```
temps = [36.5, 37.8, 38.2, 36.9, 38.0]
fevers = 0
for t in temps:
    if t > 37.5:
        print("Fever:", t)
        fevers = fevers + 1
print(f"{fevers} fever case(s)")
```

OUTPUT

```
Fever: 37.8
Fever: 38.2
Fever: 38.0
3 fever case(s)
```

```
items = ["Paracetamol", "Aspirin", "Cetirizine"]
qtys = [45, 8, 60]
shortages = 0
for i in range(3):
    if qtys[i] < 10:
        print("REORDER:", items[i], "-", qtys[i])
        shortages = shortages + 1
print(shortages, "item(s) below minimum")
```

OUTPUT

```
REORDER: Aspirin - 8
1 item(s) below minimum
```

The last example introduces the index walk: `range(3)` produces 0, 1, 2 and `items[i]`/`qtys[i]` pick matching pairs — two lists walked in step (Unit III replaces this with dictionaries and tables).

7.5 The while loop — repeat while a condition holds

`while` re-checks its condition **before every pass** and repeats as long as it stays True. Two theoretical duties follow: the body must contain an **update** that moves the situation toward False; and because the check comes first, a condition that is False initially means the body runs **zero** times.

```
stock = 3
while stock > 0:
    print("Dispensing 1 strip; remaining:", stock - 1)
    stock = stock - 1
print("Out of stock!")
```

```
OUTPUT
Dispensing 1 strip; remaining: 2
Dispensing 1 strip; remaining: 1
Dispensing 1 strip; remaining: 0
Out of stock!
```

Forget the update line and the condition never changes — the **infinite loop**. It is not dangerous (press `Ctrl+C`, or *Interrupt Kernel* in Jupyter) but it is always a bug: every `while` body must answer the question *"which line moves toward the exit?"*

for or while? If the repeats can be counted in advance ("each patient", "five times"), use `for`. If only a condition decides ("while stock remains", "until the user types done"), use `while`.

Five examples:

```
tablets, daily = 31, 2
days = 0
while tablets >= daily:
    tablets = tablets - daily
    days = days + 1
print(f"{days} full days, {tablets} tablet(s) left")
```

```
OUTPUT
15 full days, 1 tablet(s) left
```

```
count = 5
while count > 0:
    print(count)
    count = count - 1
```

```
OUTPUT
5
4
3
2
1
```

```
x = 1
while x < 100:
    x = x * 3
print(x)
```

```
OUTPUT
243
```

```
stock = 0
while stock > 0:
    print("never runs")
print("checked first: body skipped")
```

OUTPUT

```
checked first: body skipped
```

```
dose = 1000
while dose > 120:
    dose = dose // 2        # halving until a safe region
print(dose, "mg")
```

OUTPUT

```
62 mg
```

7.6 break and continue — the syllabus pair

`break` exits the whole loop immediately — remaining items are never touched. `continue` abandons only the current round and jumps to the next item. Mnemonic: **break breaks out; continue carries on.**

break — stop at the first problem. Scanning batch labels, the first expired batch is a quarantine event; scanning further is pointless:

```
batches = ["B1", "B2", "EXPIRED", "B4"]
for b in batches:
    if b == "EXPIRED":
        print("STOP - expired batch found")
        break
    print("Checking batch:", b)
```

OUTPUT

```
Checking batch: B1
Checking batch: B2
STOP - expired batch found
```

B4 is never examined — break means stop.

continue — skip the bad, keep the good. Sensor readings of −1 mark errors; ignore them, keep the rest:

```
temps = [36.5, -1, 37.9, -1, 38.1]
for t in temps:
    if t < 0:
        continue
    print("Read:", t)
```

OUTPUT

```
Read: 36.5
Read: 37.9
Read: 38.1
```

More contrasts:

```
for i in range(4):
    if i == 2:
        break
    print(i)
```

OUTPUT

```
0
1
```

```
for i in range(4):
    if i == 2:
        continue
    print(i)
```

OUTPUT

```
0
1
3
```

```
for n in range(10, 15):
    if n % 2 == 0:      # even batch number
        continue        # audit odd-numbered batches only
    print("Auditing batch", n)
```

OUTPUT

```
Auditing batch 11
Auditing batch 13
```

The sentinel loop. `while True` looks infinite; `break` is its designed exit door. This eight-line shape is the standard data-entry tool of the unit:

```
total = 0
while True:
    entry = input("Dose in mg (or 'done'): ")
    if entry == "done":
        break
    total = total + float(entry)
print(f"Total: {total} mg")
```

OUTPUT

```
Dose in mg (or 'done'): 500
Dose in mg (or 'done'): 250
Dose in mg (or 'done'): done
Total: 750.0 mg
```

`"done"` is the **sentinel** — the magic word that means stop (it is never added). Read the body top-to-bottom: ask; if done, leave; otherwise add and ask again.

Chapter summary. `for` walks a known collection once per item, the loop variable refreshing each pass; `range()` counts for you (from 0, stop excluded, optional step). Accumulator/counter patterns

(`init before, add inside, use after`) turn collections into statistics. `while` repeats on a condition — checked first, updated inside, `Ctrl+C` as the fire exit when the update is forgotten. `break` abandons the loop, `continue` skips one round, and the sentinel loop (`while True` + `break`) is the unit's data-entry workhorse.

7.7 enumerate() — looping with the index

Sometimes you need both the position and the value — for example, printing "Reading 1: 36.5, Reading 2: 38.2…". `enumerate()` supplies both without manual counting:

```
temps = [36.5, 38.2, 37.0, 38.9]
for i, t in enumerate(temps):
    print(f"Reading {i + 1}: {t} C")
```

OUTPUT

```
Reading 1: 36.5 C
Reading 2: 38.2 C
Reading 3: 37.0 C
Reading 4: 38.9 C
```

Without `enumerate`, you would need `for i in range(len(temps)):` followed by `temps[i]` — longer and less readable.

More enumerate examples:

```
drugs = ["Metformin", "Amoxicillin", "Ibuprofen"]
for position, drug in enumerate(drugs, start=1):
    print(f"{position}. {drug}")
```

OUTPUT

```
1. Metformin
2. Amoxicillin
3. Ibuprofen
```

```
# Flag the first fever reading
for i, t in enumerate(temps):
    if t > 37.5:
        print(f"Fever at reading {i + 1}: {t} C")
        break
```

OUTPUT

```
Fever at reading 2: 38.2 C
```

```
# Build a numbered batch report
batches = ["B2026-001", "B2026-002", "B2026-003"]
statuses = ["Pass", "Fail", "Pass"]
for i, (batch, status) in enumerate(zip(batches, statuses)):
    print(f"Row {i}: {batch} -> {status}")
```

OUTPUT

```
Row 0: B2026-001 -> Pass
Row 1: B2026-002 -> Fail
Row 2: B2026-003 -> Pass
```

7.8 zip() — looping two lists together

`zip()` pairs corresponding items from two (or more) lists, stopping at the shorter one:

```
drugs = ["Aspirin", "Dolo", "Brufen"]
doses = [100, 500, 200]
for drug, dose in zip(drugs, doses):
    print(f"{drug}: {dose} mg")
```

OUTPUT

```
Aspirin: 100 mg
Dolo: 500 mg
Brufen: 200 mg
```

The parallel-lists pattern from Unit III (`items[i]`, `qtys[i]`) becomes cleaner with `zip` — no index needed.

More zip examples:

```
# Pair pre- and post-treatment values
pre = [45, 52, 38, 61]
post = [38, 44, 35, 50]
for p_before, p_after in zip(pre, post):
    change = p_before - p_after
    print(f"Change: {change}")
```

OUTPUT

```
Change: 7
Change: 8
Change: 3
Change: 11
```

```
# Three lists at once
names = ["Ravi", "Sana"]
ages = [45, 38]
doses = [500, 250]
for name, age, dose in zip(names, ages, doses):
    print(f"{name} ({age} y): {dose} mg")
```

OUTPUT

```
Ravi (45 y): 500 mg
Sana (38 y): 250 mg
```

```
# Convert zip output to a list of tuples (for later use)
pairs = list(zip(drugs, doses))
print(pairs)
```

OUTPUT

```
[('Aspirin', 100), ('Dolo', 500), ('Brufen', 200)]
```

```
# Unzip (inverse operation)
drugs_back, doses_back = zip(*pairs)
print(drugs_back)
```

OUTPUT

```
('Aspirin', 'Dolo', 'Brufen')
```

7.9 Nested loops — a loop inside a loop

A **nested loop** repeats its body for every combination of an outer and inner iteration — the natural shape for checking every patient against every drug, or every batch against every time point:

```
wards = ["A", "B", "C"]
drugs = ["Metformin", "Amoxicillin"]
for ward in wards:
    for drug in drugs:
        print(f"Ward {ward}: checking stock of {drug}")
```

OUTPUT

```
Ward A: checking stock of Metformin
Ward A: checking stock of Amoxicillin
Ward B: checking stock of Metformin
Ward B: checking stock of Amoxicillin
Ward C: checking stock of Metformin
Ward C: checking stock of Amoxicillin
```

The outer loop runs 3 times; for each pass, the inner loop runs 2 times — 6 combinations in total. Use nested loops sparingly (they multiply quickly), but they are the right tool for all-pairs comparisons.

More nested loop examples:

```
# Dose timing table
times = ["08:00", "14:00", "20:00"]
days = ["Mon", "Tue"]
for day in days:
    for time in times:
        print(f"{day} {time}: dispense dose")
```

OUTPUT

```
Mon 08:00: dispense dose
Mon 14:00: dispense dose
Mon 20:00: dispense dose
Tue 08:00: dispense dose
Tue 14:00: dispense dose
Tue 20:00: dispense dose
```

```
# Find all drug-drug interaction pairs
drugs = ["Warfarin", "Aspirin", "Ibuprofen"]
for i in range(len(drugs)):
    for j in range(i + 1, len(drugs)):
        print(f"Check interaction: {drugs[i]} + {drugs[j]}")
```

OUTPUT

```
Check interaction: Warfarin + Aspirin
Check interaction: Warfarin + Ibuprofen
Check interaction: Aspirin + Ibuprofen
```

Quick checks 7. (a) How many times does `range(3)` repeat its body, and what numbers? (b) In the days-supply loop, which line is the update? (c) Difference between break and continue in one sentence. (d) Predict: `total` after `for x in [2, 4, 6]: total += x`. (e) Why must the sentinel string never be converted with `float()`? (f) What does `enumerate(temps, start=1)` give you? (g) What does `zip()` do when one list is shorter? (h) How many total iterations does a 3×4 nested loop produce?

Chapter 8 — Functions

8.1 Why functions exist

Copy-pasting the BMI formula into five notebooks means five places to fix when the formula changes. A **function** is a named, reusable block of code that performs one task: define it once, call it by name forever. The benefits are the syllabus's own words — *reusability* (a single definition serves unlimited calls), *modularity* (large programs become small, single-purpose pieces), and *easier debugging* (each piece can be tested separately). Every calculator written so far in this book becomes a one-line tool by the end of this chapter.

8.2 Anatomy of a function

```
def greet_patient():
    """Print a welcome message for the pharmacy counter."""
    print("Welcome to the pharmacy!")

greet_patient()
```

```
OUTPUT
Welcome to the pharmacy!
```

Four parts: `def` introduces the definition; the **function name** follows (snake_case, same naming rules as variables); **parameters** — the inputs the function accepts — sit in brackets (here, none); the indented body contains the work, optionally opened by a **docstring**, a triple-quoted note describing the function (Unit I's comment rules apply). Defining a function does nothing by itself; **calling** it — name plus brackets — runs it.

8.3 Parameters and arguments

A **parameter** is the placeholder variable in the definition; an **argument** is the concrete value handed over when the call happens. Inside the function, parameters behave like ordinary variables — pre-filled boxes:

```
def calculate_dose(weight, dose_per_kg):    # parameters
    total = weight * dose_per_kg
    return total

result = calculate_dose(60, 5)              # arguments
print("Total dose:", result, "mg")
```

```
OUTPUT
Total dose: 300 mg
```

8.4 return — sending the answer back

`return` delivers a value from inside the function back to its caller, where it can be stored, reused, or fed into another calculation. Without `return`, a function returns the special value `None` — and its work evaporates when it ends.

	`return`	`print()`
Purpose	send a value back to the caller	display text on screen
Result reusable?	yes — store it in a variable	no — the value is lost after display

```
def mg_per_kg(dose, weight):
    return dose / weight

r = mg_per_kg(500, 70)
print(f"{r:.2f} mg per kg")
print(f"Doubled: {mg_per_kg(1000, 70):.2f} mg per kg")
```

OUTPUT

```
7.14 mg per kg
Doubled: 14.29 mg per kg
```

Five examples of returning values:

```
def area_label(w, h):
    return f"{w * h} sq cm"

print(area_label(5, 4))
```

OUTPUT

```
20 sq cm
```

```
def is_fever(temp):
    return temp > 37.5

print(is_fever(38.2), is_fever(36.9))
```

OUTPUT

```
True False
```

```
def no_return():
    x = 42

print(no_return())
```

OUTPUT

```
None
```

```
def strip_count(total, per_strip=10):        # default parameter
    return total // per_strip

print(strip_count(43))
print(strip_count(43, per_strip=15))         # keyword argument
```

OUTPUT

```
4
2
```

```
def total_dose(*doses):                      # variable-length: any number of
args
    return sum(doses)

print(total_dose(500, 250, 750))
print(total_dose(100))
```

OUTPUT

```
1500
100
```

The last three examples introduce the **argument types** the syllabus names: *positional* (matched by order — all earlier calls), *default* (`per_strip=10` supplies a value when the caller omits it), *keyword* (`per_strip=15` names its target, order no longer matters), and *variable-length* (`*doses` gathers any number of arguments into one collection).

8.5 Local scope — a function's variables are private

Variables created inside a function exist only inside it. This **local scope** is a feature: functions cannot accidentally overwrite the caller's variables.

```
def report():
    dose = 500               # local: exists only inside report()

report()
print(dose + 1)
```

OUTPUT

```
NameError: name 'dose' is not defined
```

(A *beyond the exam* note: a function can call itself — **recursion**; it powers elegant mathematics but is unnecessary for pharmacy programming, and `while` handles those cases more plainly.)

8.6 Modular program 1 — dosage calculation (syllabus)

The syllabus asks for a modular dosage calculator. Three functions, each doing one job: a rate decision (if-elif-else from Chapter 6), a multiplication (Unit I arithmetic), and a report. A loop over a patient list then runs the whole ward through them:

```
def get_dose_rate(age):
    """Weight-based rate in mg/kg by age group."""
    if age < 12:
        return 8            # children
    elif age < 65:
        return 12           # adults
    else:
        return 6            # elderly - reduced rate

def calculate_dose(weight, rate):
    return weight * rate

def dosage_report(name, age, weight):
    rate = get_dose_rate(age)
    dose = calculate_dose(weight, rate)
    print(f"{name} | Age {age} | {weight} kg -> {dose} mg")

patients = [("Aarav", 8, 22), ("Meera", 35, 50), ("Ravi", 68, 55)]
for name, age, weight in patients:
    dosage_report(name, age, weight)
```

OUTPUT

```
Aarav | Age 8 | 22 kg -> 176 mg
Meera | Age 35 | 50 kg -> 600 mg
Ravi | Age 68 | 55 kg -> 330 mg
```

Note how the loop unpacks each tuple directly into `name, age, weight`, and how `dosage_report` *calls* the other two functions — assembly, not copy-paste. Change the adult rate to 10 and all three reports update: one definition, many uses.

8.7 Modular program 2 — BMI calculation (syllabus)

The same architecture for the BMI calculator: one function computes, one classifies (the Chapter 6 chain), one reports:

```
def calculate_bmi(weight, height):
    """BMI = weight (kg) / height (m) squared."""
    return weight / (height ** 2)

def classify_bmi(bmi):
    if bmi < 18.5:
        return "Low weight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def bmi_report(name, weight, height):
    bmi = calculate_bmi(weight, height)
    category = classify_bmi(bmi)
    print(f"{name}: BMI {bmi:.1f} -> {category}")

bmi_report("Anita", 55, 1.62)
bmi_report("Vikram", 88, 1.70)
```

OUTPUT

```
Anita: BMI 21.0 -> Normal
Vikram: BMI 30.4 -> Obese
```

Why modular? the two small functions slot straight into other tools — a diet-advice program, a ward screening sheet, a Unit IV dataframe — with the logic written once and never retyped. That is the entire point of functions.

8.8 The dispensary toolkit — a third assembler

To close the unit, the Cockcroft–Gault calculator from Unit I becomes a function and joins the toolkit:

```
def crcl(age, weight, scr, sex="M"):
    """Creatinine clearance (Cockcroft-Gault), mL/min."""
    factor = 0.85 if sex == "F" else 1.0
    return (140 - age) * weight / (72 * scr) * factor

print(f"{crcl(55, 70, 1.0):.1f} mL/min")
print(f"{crcl(55, 70, 1.0, sex='F'):.1f} mL/min")
```

OUTPUT

```
82.6 mL/min
70.2 mL/min
```

One definition, keyword-flexible, unit-tested against the hand calculation — this is what "modular program" means on the examination paper.

Chapter summary. `def name(parameters):` defines a reusable block; calling it runs it. Arguments fill parameters — positionally, by keyword, with defaults, or gathered by `*args`. `return` delivers the answer for storing and reuse (`print` merely displays; no `return` means `None`). Function variables are local. The two syllabus programs — dosage by age group and BMI with classification — demonstrate the modular style: small single-job functions, assembled by a caller.

8.9 Lambda functions — one-line anonymous functions

A **lambda function** is a function without a name, defined in a single expression. It exists for the moments when a full `def` block is overkill — typically as an argument to another function:

```
# Full def version
def double(x):
    return x * 2

# Lambda equivalent
double = lambda x: x * 2

print(double(250))
```

OUTPUT

```
500
```

The syntax: `lambda parameters: expression`. The expression's value is the return value — no `return` keyword needed. Lambdas are limited to one expression and are best kept for short, single-use transformations.

Where lambdas shine — with `sorted()`, `map()` and `filter()`:

```
doses = [("Aspirin", 100), ("Dolo", 500), ("Brufen", 200)]

# Sort by the second element (the dose) — lambda extracts the sort key
sorted_by_dose = sorted(doses, key=lambda pair: pair[1])
print(sorted_by_dose)
```

OUTPUT

```
[('Aspirin', 100), ('Brufen', 200), ('Dolo', 500)]
```

```
# map(): apply a function to every element
doses_list = [250, 500, 750]
doubled = list(map(lambda d: d * 2, doses_list))
print(doubled)
```

OUTPUT

```
[500, 1000, 1500]
```

```
# filter(): keep only elements matching a condition
high = list(filter(lambda d: d >= 500, doses_list))
print(high)
```

OUTPUT

```
[500, 750]
```

```
# Combined: filter + map
safe_doubled = list(map(lambda d: d * 2, filter(lambda d: d < 500, doses_list)))
print(safe_doubled)
```

OUTPUT

```
[500]
```

Lambda vs def — when to use each:

	`def`	`lambda`
Lines	multi-line body	one expression only
Name	has a name (reusable)	anonymous (single use)
Best for	named tools, docstrings	sort keys, `map`/`filter` arguments

Five lambda examples:

```
# Extract the drug name from a tuple
get_name = lambda pair: pair[0]
print(get_name(("Aspirin", 100)))
```

OUTPUT

```
Aspirin
```

```
# Categorise a temperature
fever = lambda t: "Fever" if t > 37.5 else "Normal"
print(fever(38.2), fever(36.9))
```

OUTPUT

```
Fever Normal
```

```
# Sort patients by age
patients = [("Ravi", 45), ("Sana", 38), ("Meera", 52)]
oldest_first = sorted(patients, key=lambda p: p[1], reverse=True)
print(oldest_first[0])
```

OUTPUT

```
('Meera', 52)
```

```
# Round to 1 decimal
clean = lambda x: round(x, 1)
print(clean(45.234))
```

OUTPUT

```
45.2
```

```
# Compute mg/kg in one line
per_kg = lambda dose, weight: round(dose / weight, 2)
print(per_kg(500, 70))
```

OUTPUT

```
7.14
```

8.10 Error handling — try, except, else, finally

Real programs meet errors: files that do not exist, users who type letters where numbers are expected, divisions by zero. **Exception handling** lets the program anticipate errors and respond gracefully instead of crashing.

The try/except pattern:

```
try:
    dose = float(input("Dose in mg: "))
    per_kg = dose / 70
    print(f"{per_kg:.2f} mg per kg")
except ValueError:
    print("Please enter a number, not text.")
except ZeroDivisionError:
    print("Weight cannot be zero.")
```

OUTPUT

```
Dose in mg: abc
Please enter a number, not text.
```

The `try` block runs; if an error occurs, Python jumps to the matching `except` block instead of crashing. Multiple `except` clauses handle different error types.

Common error types:

Error	Triggered by	Pharmacy example
`ValueError`	right type, wrong value	`int("Paracetamol")`
`TypeError`	wrong type for the operation	`"500" + 100`
`ZeroDivisionError`	dividing by zero	`dose / 0`
`NameError`	using a name that doesn't exist	misspelled variable
`IndexError`	index beyond the collection	`temps[99]` on 4 items
`KeyError`	dictionary key not found	`d["missing_key"]`
`FileNotFoundError`	file does not exist	missing `patients.csv`

else and finally:

```
try:
    file = open("patients.csv", "r")
    data = file.read()
    file.close()
except FileNotFoundError:
    print("Error: patients.csv not found")
else:
    print("File loaded:", len(data), "characters")
finally:
    print("Attempt complete.")
```

OUTPUT

```
Error: patients.csv not found
Attempt complete.
```

`else` runs only when no error occurred; `finally` runs **always** — perfect for cleanup code (closing files, database connections).

Raising your own errors with `raise`:

```
def calculate_dose(weight, dose_per_kg):
    if weight <= 0:
        raise ValueError(f"Weight must be positive, got {weight}")
    return weight * dose_per_kg

try:
    result = calculate_dose(-70, 5)
except ValueError as e:
    print(f"Rejected: {e}")
```

OUTPUT

```
Rejected: Weight must be positive, got -70
```

`raise` deliberately stops the program with a clear message when an input is clinically nonsensical — a negative weight, a dose above a safety ceiling.

Five try/except examples:

```
# Guard a calculation
try:
    result = 500 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

OUTPUT

```
Cannot divide by zero
```

```
# Guard a file read
try:
    with open("missing.txt") as f:
        content = f.read()
except FileNotFoundError:
    print("File not found — using default data")
```

OUTPUT

```
File not found — using default data
```

```
# Guard user input
try:
    age = int(input("Age: "))
except ValueError:
    age = 0
    print("Invalid age — set to 0")
```

```
# Catch any error
try:
    risky_operation()
except Exception as e:
    print(f"Unexpected: {type(e).__name__}: {e}")
```

```
# The safe CSV loader
def safe_load(filename):
    try:
        with open(filename) as f:
            return f.read()
    except FileNotFoundError:
        print(f"{filename} not found")
        return None
```

8.11 Variable arguments — *args and **kwargs

Sometimes a function should accept any number of inputs: total any number of doses, average any number of readings. Python's `*args` (variable positional) and `**kwargs` (variable keyword) parameters make this possible:

```
def total_dose(*doses):
    """Sum any number of dose values."""
    return sum(doses)

print(total_dose(500))
print(total_dose(500, 250, 750))
print(total_dose(100, 200, 300, 400, 500))
```

```
OUTPUT
500
1500
1500
```

Inside the function, `*doses` arrives as a **tuple** — the star in the *definition* collects; the star in the *call* unpacks.

```
def patient_report(name, **details):
    """Print a patient record from any number of key-value pairs."""
    print(f"Patient: {name}")
    for key, value in details.items():
        print(f"  {key}: {value}")

patient_report("Ravi", age=45, drug="Metformin", dose=500)
```

```
OUTPUT
Patient: Ravi
  age: 45
  drug: Metformin
  dose: 500
```

Inside the function, `**details` arrives as a **dictionary** — the double star collects named arguments.

More examples:

```
# Unpacking a list into separate arguments
doses = [500, 250, 750]
print(total_dose(*doses))     # star unpacks: 500, 250, 750
```

```
OUTPUT
1500
```

```
# Mixing fixed and variable arguments
def dose_check(patient, *doses, unit="mg"):
    total = sum(doses)
    return f"{patient}: {total} {unit}"

print(dose_check("Ravi", 500, 250))
```

```
OUTPUT
Ravi: 750 mg
```

```
# A flexible averaging function
def average(*values):
    if not values:
        return 0
    return sum(values) / len(values)

print(average(36.5, 37.0, 38.2))
```

OUTPUT

```
37.233333333333334
```

```
# Building a drug dictionary on the fly
def add_drug(catalog, **info):
    name = info.pop("name")
    catalog[name] = info
    return catalog

catalog = {}
add_drug(catalog, name="Metformin", dose=500, category="Biguanide")
print(catalog)
```

OUTPUT

```
{'Metformin': {'dose': 500, 'category': 'Biguanide'}}
```

Quick checks 8. (a) Parameter vs argument, in one sentence each. (b) What does a function without `return` give back? (c) In `strip_count(43, per_strip=15)`, which part is a keyword argument? (d) Why is `is_fever` returning a Boolean more useful than printing "fever"? (e) Name the three functions of the BMI program and their single jobs. (f) Write a lambda that triples a value. (g) What error does `int("abc")` raise? (h) What does `*args` collect inside a function? (i) What does `finally` guarantee?

Unit II — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Conditional statements (if, if-else, if-elif-else), nested conditions	Ch 6
Loops (for loop, while loop)	Ch 7 (§7.2–7.5)
break and continue statements	Ch 7 (§7.6)
Defining and calling functions; arguments and return values	Ch 8 (§8.1–8.5)
Modular programs: dosage calculation and BMI calculation	Ch 8 (§8.6–8.8)

Twenty-question exercise bank

Predict the output (1–8).

1. `x = 7` then `if x > 5: print("A")` then `if x > 10: print("B")`
2. `m = 60`; the grade chain (70/60/50) — which grade?
3. `s = 0` then `for i in range(4): s += i` then `print(s)`
4. `for c in "PK": print(c)`
5. `stock = 2`; `while stock > 0: print("give"); stock -= 1`
6. `for i in range(6): if i == 3: break` then `print(i)` (outside the loop)
7. `for i in range(4): if i % 2 == 0: continue` then `print(i)` (outside)
8. `print(nothing())` where `def nothing(): x = 1` — output?

Short answers (9–14).

9. State the three syntax rules of an `if` block.
10. Explain first-True-wins with the buggy grade chain as evidence.
11. Why must every `while` body contain an update line?
12. break vs continue — one sentence and one pharmacy example each.
13. What are the four argument types? Give a one-line example of each.
14. Why does the sentinel loop use `while True` instead of a condition?

Write programs (15–20).

15. Temperature classifier: <36 hypothermia, 36–37.5 normal, >37.5 fever (chain).
16. Stock gate: `if qty >= need: dispense` else print shortfall.
17. Count fevers in `[36.8, 38.1, 37.9, 36.5, 38.4]` with a for + if.

18. Days supply for 45 tablets at 3 per day (while loop).
19. Sentinel dose-total loop that also counts entries.
20. `def bsa(height_cm, weight)` returning Mosteller BSA; call it for 170 cm, 70 kg (expect 1.82).

Answers to selected exercises

1. `A` only. 2. First Class. 3. `6` (0+1+2+3). 4. `P` then `K`. 5. `give give`. 6. `3`. 7. `3`. 8. `None`. 10. 88 caught by `>= 50` first, deeper branches unreachable. 11. Otherwise the condition never changes — infinite loop. 14. Because the exit condition is *inside* the body (the sentinel check) rather than in the `while` line. 17. 3 fevers. 18. 15 days, 0 left. 20. `import math` inside or `** 0.5`; `math.sqrt(170*70/3600)` → 1.82.

Important points for examination

1. A conditional runs a block only when its condition holds; Python marks blocks by **indentation**, not braces — a wrong indent is an `IndentationError`, and four spaces per level is the convention.
2. `else` takes no condition and is the catch-all; `elif` is tested only when every earlier test was False.
3. The chain is greedy from the top: place the **strictest condition first** or loose tests swallow the rest — with no error message.
4. Boundaries are policy: `>=` admits the limit value, `>` rejects it.
5. Nesting is one decision inside another; use it when inner questions only make sense after outer ones, and keep three levels as the ceiling.
6. `for` suits a known collection or count; `while` suits an open-ended condition, checked before each pass, updated inside each pass.
7. An forgotten update is the infinite loop; stop it with Ctrl+C, fix it with an update line.
8. `break` leaves the whole loop; `continue` abandons one round only; the sentinel loop is `while True` plus `break` on a magic word.
9. `range()` starts at 0 and excludes its stop; `range(1, 6)` yields 1–5.
10. A function is defined with `def`, runs when called, receives arguments in its parameters, and hands back a value with `return` — or `None` if no return executes.
11. `print()` displays; `return` delivers for reuse. Argument kinds: positional, keyword, default, variable-length (`*args`).
12. Modular style: one job per function (`get_dose_rate`, `calculate_dose`, `classify_bmi`), assembled by a caller — the dosage and BMI programs of this chapter are the models.

Sources for this unit: course decks Classes 6–7 and the classroom programs (`pass_fail_checker.py`, ward monitor); topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 2* notes, MCA R24 lecture notes Unit II (functions), Yamuna Devi, Jeyabharathi & Vajpayee *Python Programming* (GCS), MRCET R17 notes, and the curated learn.biodockify.com catalog (trekhleb/learn-python, exercism/python, gto76/python-cheatsheet, W3Schools Python If...Else / For Loops). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source (zero 7-word overlaps). Full credits: see the book's Sources and Acknowledgements page.

UNIT III

Data Structures & File Handling

Chapters 9–12 · 6 teaching hours

Chapter 9 — Lists and Tuples

9.1 Why collections exist

Units I–II gave every value its own variable — fine for one patient's dose, hopeless for a ward of forty. A **collection** stores many values inside one named variable, and the three you will meet — list, tuple, dictionary — differ in exactly two ways: can you *change* them after creation (*mutability*), and how do you *address* an item (by position or by key)? Choose the right one and the program almost writes itself.

9.2 Creating, indexing and slicing lists

Think of a **list** as a numbered, editable collection inside square brackets. Items keep their insertion order, may repeat, and may mix types.

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin", "Cetirizine"]
prices = [12.5, 8.0, 5.5, 15.0]
mixed = ["Tab", 500, True]        # any type may live in a list
empty = []                         # starts empty, grows later
print(len(drugs))
```

OUTPUT

```
4
```

Indexing and slicing work exactly as they do on strings (Unit I, §4.2–4.3): positions count from zero, negative positions count from the end, slices exclude the stop, and over-generous slices stop gracefully.

```
print(drugs[0], "|", drugs[-1])
print(drugs[1:4])
print(drugs[:3])
print(drugs[2:])
print(drugs[::-1])
```

OUTPUT

```
Paracetamol | Cetirizine
['Ibuprofen', 'Aspirin', 'Cetirizine']
['Paracetamol', 'Ibuprofen', 'Aspirin']
['Aspirin', 'Cetirizine']
['Cetirizine', 'Aspirin', 'Ibuprofen', 'Paracetamol']
```

Five slicing patterns to memorise: `[a:b]` a range, `[:b]` from the start, `[a:]` to the end, `[::-1]` reversed, `[::2]` every second item.

```
stock = [50, 30, 80, 15, 60]
print(stock[::2])          # every 2nd
print(stock[:-2])          # all except the last 2
print(stock[1:4:2])        # indices 1 and 3
```

OUTPUT

```
[50, 80, 60]
[50, 30, 80]
[30, 15]
```

Reading past the end raises `IndexError` — the last valid position is `len(list) - 1`; a slice past the end simply returns what exists.

9.3 List methods — adding, removing, sorting

Method	What it does	Example
`.append(item)`	adds to the end	`drugs.append("Metformin")`
`.insert(i, item)`	inserts at position i	`drugs.insert(1, "Amoxicillin")`
`.remove(item)`	removes the first match	`drugs.remove("Aspirin")`
`.pop(i)`	removes and returns position i (last by default)	`drugs.pop(0)`
`.sort()`	sorts ascending, in place	`prices.sort()`
`.reverse()`	reverses, in place	`drugs.reverse()`
`.count(item)`	counts occurrences	`stock.count(50)`
`.index(item)`	position of the first match	`drugs.index("Aspirin")`
`.extend(list2)`	appends all of list2	`drugs.extend(more)`
`.clear()`	empties the list	`stock.clear()`

Five examples:

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin"]
drugs.append("Cetirizine")
print(drugs)
```

OUTPUT

```
['Paracetamol', 'Ibuprofen', 'Aspirin', 'Cetirizine']
```

```
stock = [50, 30, 80, 15]
stock.sort()
print(stock)
print(sum(stock), max(stock), min(stock))
```

```
OUTPUT
[15, 30, 50, 80]
175 80 15
```

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin", "Ibuprofen"]
print(drugs.count("Ibuprofen"))
print(drugs.index("Aspirin"))
```

```
OUTPUT
2
2
```

```
list1 = ["Aspirin", "Dolo"]
list2 = ["Brufen", "Meftal"]
list1.extend(list2)
print(list1)
```

```
OUTPUT
['Aspirin', 'Dolo', 'Brufen', 'Meftal']
```

```
stock = [50, 30, 80]
print("Aspirin" in drugs)
removed = stock.pop()
print(removed, stock)
```

```
OUTPUT
True
80 [50, 30]
```

The copying pitfall. `new = old` does **not** copy a list — both names refer to one shared object, so editing through either name alters the single list. Use `new = old.copy()` (or `list(old)`) for a genuine independent copy.

9.4 List comprehensions — one-line list building

A **list comprehension** constructs a fresh list in a single expression, standing in for a three-line loop:

```
doses = [250, 500, 750, 1000]
doubled = [d * 2 for d in doses]
print(doubled)
```

```
OUTPUT
[500, 1000, 1500, 2000]
```

Add a condition to filter while building:

```
high = [d for d in doses if d >= 500]
print(high)
```

```
OUTPUT
[500, 750, 1000]
```

Three more examples:

```
names = ["paracetamol", "aspirin"]
cleaned = [n.title() for n in names]
print(cleaned)
```

```
OUTPUT
['Paracetamol', 'Aspirin']
```

```
prices = [12.5, 8.0, 5.5]
with_tax = [round(p * 1.1, 2) for p in prices]
print(with_tax)
```

```
OUTPUT
[13.75, 8.8, 6.05]
```

```
temps = [36.5, 38.2, 37.0, 38.9]
fever_only = [t for t in temps if t > 37.5]
print(fever_only)
```

```
OUTPUT
[38.2, 38.9]
```

Read the last example aloud: *the temperatures that are above 37.5* — the comprehension is the sentence.

9.5 Nested lists — a table in a variable

A list can hold other lists — a natural way to represent a table, one inner list per row:

```
patients = [
    ["Sana", 32, 500],
    ["Ravi", 45, 250],
    ["Meera", 28, 750],
]
print(patients[0])          # the first row
print(patients[1][2])       # Ravi's dose
print(len(patients))        # number of patients
```

```
OUTPUT
['Sana', 32, 500]
250
3
```

Two indexes address one cell: `patients[1][2]` means row 1, column 2. This pattern scales straight into Unit IV's DataFrames — a DataFrame *is* a named, indexed table.

9.6 Tuples — the sealed record

A **tuple** is an ordered collection that, once created, **cannot be changed** — no appending, no replacing, no removing. It is written in round brackets (or just commas), and its immutability is a feature: fixed data — a blood-pressure reading, a date of birth, a set of coordinates — *should* resist accidental editing.

```
vitals = (120, 80)          # systolic, diastolic
systolic, diastolic = vitals    # unpacking
print(f"{systolic}/{diastolic} mmHg")
```

OUTPUT

```
120/80 mmHg
```

Feature	List	Tuple
Brackets	`[]`	`()`
Mutable?	yes — add, remove, change	no — sealed after creation
Speed	slower	faster
Best for	changing collections (stock)	fixed records (BP, DOB, coordinates)

Five examples:

```
vitals = (120, 80)
print(vitals[0], vitals[-1])
```

OUTPUT

```
120 80
```

```
single = (500,)           # note the comma - (500) is just an int!
print(type(single))
```

OUTPUT

```
<class 'tuple'>
```

```
bp_visits = [(120, 80), (128, 84), (135, 88)]
for sys_, dia in bp_visits:
    print(f"{sys_}/{dia} mmHg")
```

OUTPUT

```
120/80 mmHg
128/84 mmHg
135/88 mmHg
```

```
vitals = (120, 80, 120, 80)
print(vitals.count(120), vitals.index(80))
```

```
OUTPUT
2 1
```

```
# vitals[0] = 130     # TypeError - tuples are sealed
```

A list of tuples — `bp_visits` above — is the standard shape for repeated fixed readings: each visit is a sealed record, the collection of visits grows.

Chapter summary. Lists are numbered, changeable, mixed-type collections in `[]`; index from 0, slice with `[a:b]`, mutate with `append/insert/remove/pop/sort`. Copy with `.copy()`, never `=`. Comprehensions `[expr for item in list if cond]` build lists in one readable line. Nested lists form tables (`patients[1][2]`). Tuples are the immutable cousin in `()` — unpack them into named variables, use them for fixed records; a single item needs a trailing comma.

Quick checks 9. (a) `drugs = ["A", "B", "C"]; drugs[1:3]`? (b) Difference between `append` and `extend`? (c) What does `[d for d in [100, 200, 300] if d > 150]` produce? (d) Why does `(500)` not create a tuple? (e) Give one pharmacy situation for a list and one for a tuple.

Chapter 10 — Dictionaries and String Tools

10.1 Dictionaries — data with names on it

A **dictionary** holds information as **key : value** pairs within curly braces. Each key is unique and acts as the item's name — you look things up *by key*, not by position, exactly like finding a drug in an index by its name rather than by the page it happens to sit on.

```
drug_info = {"name": "Paracetamol", "dose_mg": 500, "category": "Analgesic"}
print(drug_info["name"])
print(drug_info["dose_mg"])
```

OUTPUT

```
Paracetamol
500
```

10.2 Adding, updating and removing

Dictionaries are mutable: assign to a key to create or update; pop to remove.

```
drug_info["stock"] = 120          # add a new key
drug_info["dose_mg"] = 650        # update an existing one
print(drug_info)
```

OUTPUT

```
{'name': 'Paracetamol', 'dose_mg': 650, 'category': 'Analgesic', 'stock': 120}
```

Five examples:

```
stock = {"Paracetamol": 120, "Ibuprofen": 45, "Aspirin": 12}
print(stock["Aspirin"])
```

OUTPUT

```
12
```

```
stock["Cetirizine"] = 80          # new entry
print(len(stock))
```

OUTPUT

```
4
```

```
stock.pop("Aspirin")              # remove
print("Aspirin" in stock)
```

OUTPUT

```
False
```

```
stock.update({"Ibuprofen": 60, "Paracetamol": 150})  # bulk update
print(stock["Ibuprofen"], stock["Paracetamol"])
```

OUTPUT

```
60 150
```

```
print(stock.get("Dolo", 0))       # safe lookup: 0 if key is absent
print(stock.get("Paracetamol"))
```

OUTPUT

```
0
150
```

`.get(key, default)` is the safe accessor — it returns the default (here 0) instead of raising `KeyError` when the key does not exist.

10.3 Looping through a dictionary

`.keys()`, `.values()` and `.items()` expose the three views; `.items()` paired with tuple unpacking is the everyday idiom:

```
stock = {"Paracetamol": 120, "Ibuprofen": 45, "Aspirin": 12}
for name, qty in stock.items():
    print(name, ":", qty, "units")
```

OUTPUT

```
Paracetamol : 120 units
Ibuprofen : 45 units
Aspirin : 12 units
```

10.4 Nested dictionaries — a small patient database

```
patients = {
    "P1": {"name": "Ravi", "age": 32, "dose": 500},
    "P2": {"name": "Sana", "age": 45, "dose": 250},
}
print(patients["P1"]["name"])
print(patients["P2"]["dose"])
```

OUTPUT

```
Ravi
250
```

One dictionary whose values are themselves dictionaries: each patient ID maps to a full record. This is the exact shape of a JSON record from a hospital API.

10.5 Dictionary comprehension

Same one-line idea as list comprehension, but building key:value pairs:

```
prices = {"Paracetamol": 12.5, "Ibuprofen": 8.0, "Aspirin": 5.5}
with_tax = {name: round(p * 1.1, 2) for name, p in prices.items()}
print(with_tax)
```

OUTPUT

```
{'Paracetamol': 13.75, 'Ibuprofen': 8.8, 'Aspirin': 6.05}
```

10.6 String manipulation techniques (syllabus point)

Unit I covered the core string methods. The table below collects the techniques this syllabus bullet expects you to command, each with a pharmacy-flavoured example:

Technique	Example	Result
f-string formatting	`f"Dose: {500} mg"`	`Dose: 500 mg`
`join()`	`", ".join(["Tab", "Cap"])`	`'Tab, Cap'`
`startswith()` / `endswith()`	`"Tab 123".startswith("Tab")`	`True`
`isdigit()`	`"500".isdigit()`	`True`
`title()`	`"paracetamol".title()`	`'Paracetamol'`
`splitlines()`	`"a\nb".splitlines()`	`['a', 'b']`
`.format()`	`"{} mg".format(500)`	`'500 mg'`
`zfill()`	`"7".zfill(3)`	`'007'`
`count()`	`"Paracetamol".count("a")`	`3`
`lstrip()` / `rstrip()`	`"  drug  ".lstrip()`	`'drug  '`

Worked example — processing a clinical note:

```
note = "Patient reports mild headache and nausea after dose"
words = note.split()
print("Word count:", len(words))
print("Contains nausea:", "nausea" in note)
cleaned = note.replace("mild", "moderate")
print(cleaned)
```

OUTPUT

```
Word count: 8
Contains nausea: True
Patient reports moderate headache and nausea after dose
```

Chapter summary. Dictionaries map unique keys to values in `{}`; access/update by key; `.get` for safe lookups; `.items()` for looping; nested dicts store records; comprehensions transform in one line. String tools — `join`, `startswith`, `isdigit`, `title`, `splitlines`, `zfill`, `format`, `count` — clean and shape pharmacy text; `split` turns a sentence into a word list; `replace` edits safely by returning a new string.

10.7 Sets — the collection of unique values

A **set** is an unordered collection in which every element appears only once. Written in curly braces like a dictionary, but with values only (no colons), sets automatically discard duplicates:

```
drugs_prescribed = {"Metformin", "Aspirin", "Metformin", "Ibuprofen", "Aspirin"}
print(drugs_prescribed)
print(len(drugs_prescribed))
```

OUTPUT

```
{'Metformin', 'Aspirin', 'Ibuprofen'}
3
```

Metformin and Aspirin were listed twice, but the set keeps each name once. This de-duplication is the set's superpower.

Creating a set:

```
empty = set()                          # NOT {} — that creates an empty dict
from_list = set([1, 2, 2, 3, 3])      # from a list (duplicates removed)
print(from_list)
```

OUTPUT

```
{1, 2, 3}
```

Set operations — the algebra of overlap:

Operation	Operator	Meaning
Union	`\`	`
Intersection	&	items in both sets
Difference	-	items in the first but not the second
Symmetric difference	^	items in exactly one set

Five set examples with pharmacy context:

```
ward_a_drugs = {"Metformin", "Aspirin", "Ibuprofen"}
ward_b_drugs = {"Aspirin", "Amoxicillin", "Metformin"}

# Union: all drugs used across both wards
all_drugs = ward_a_drugs | ward_b_drugs
print(all_drugs)
```

OUTPUT

```
{'Metformin', 'Aspirin', 'Ibuprofen', 'Amoxicillin'}
```

```
# Intersection: drugs common to both wards
common = ward_a_drugs & ward_b_drugs
print(common)
```

```
OUTPUT
{'Metformin', 'Aspirin'}
```

```
# Difference: drugs in A but not in B
only_a = ward_a_drugs - ward_b_drugs
print(only_a)
```

```
OUTPUT
{'Ibuprofen'}
```

```
# Symmetric difference: drugs in exactly one ward
exclusive = ward_a_drugs ^ ward_b_drugs
print(exclusive)
```

```
OUTPUT
{'Ibuprofen', 'Amoxicillin'}
```

```
# Membership testing is very fast on sets
print("Aspirin" in ward_a_drugs)
```

```
OUTPUT
True
```

When to use a set:

- De-duplicating a list: `set(my_list)`
- Fast membership testing: `"drug" in my_set` (faster than on a list)
- Finding overlaps between groups (common patients, shared drugs)
- Mathematical set operations (union, intersection, difference)

Set methods:

Method	Effect
`.add(item)`	adds one item
`.remove(item)`	removes (raises KeyError if absent)
`.discard(item)`	removes (no error if absent)
`.pop()`	removes and returns an arbitrary item
`.clear()`	empties the set

Quick checks 10. (a) `stock = {"A": 10, "B": 20}; stock["C"] = 30; len(stock)`? (b) Why is `.get()` safer than `[]`? (c) What does `", ".join(["a", "b"])` give? (d) `"123abc".isdigit()`? (e) Build a dict comprehension that doubles every value in `{"a": 1, "b": 2}`. (f) What does `set([1, 2, 2, 3])` produce? (g) What is `{1, 2} & {2, 3}`? (h) When would you use a set instead of a list?

Chapter 11 — NumPy Arrays

11.1 What NumPy is and why it exists

NumPy (Numerical Python) is the third-party library that makes Python a serious scientific tool. Its core object is the **ndarray** — an n-dimensional array of numbers stored in contiguous memory and processed by compiled code. The practical consequence: operations that need a Python loop on a plain list happen in one vectorized expression on an array, tens to hundreds of times faster on large datasets. Every tool in Units IV–V (Pandas, Matplotlib) is built on NumPy arrays underneath.

Import with the universal alias:

```
import numpy as np
```

11.2 Creating arrays

Function	Creates	Example	Result
`np.array(list)`	from a list	`np.array([1, 2, 3])`	`array([1, 2, 3])`
`np.zeros(n)`	n zeros	`np.zeros(3)`	`array([0., 0., 0.])`
`np.ones(n)`	n ones	`np.ones(3)`	`array([1., 1., 1.])`
`np.arange(a, b)`	like range()	`np.arange(0, 10, 2)`	`array([0, 2, 4, 6, 8])`
`np.linspace(a, b, n)`	n evenly spaced	`np.linspace(0, 1, 5)`	`array([0., 0.25, 0.5, 0.75, 1.])`

Five examples:

```
doses = np.array([500, 750, 1000])
print(doses)
print(doses.shape)
print(doses.dtype)
```

OUTPUT

```
[ 500  750 1000]
(3,)
int64
```

`.shape` gives the dimensions as a tuple (here, 3 elements in one dimension); `.dtype` reports the element type — `int64` means 64-bit integers.

```
print(np.zeros(5))
```

OUTPUT

```
[0. 0. 0. 0. 0.]
```

```
print(np.arange(0, 10, 3))
```

OUTPUT

```
[0 3 6 9]
```

```
weights = np.array([60, 70, 55])
doses = weights * 5              # element-wise multiply
print(doses)
```

OUTPUT

```
[300 350 275]
```

```
doses_2d = np.array([[250, 500], [300, 600], [200, 400]])
print(doses_2d.shape)
print(doses_2d[1, 0])            # row 1, column 0
```

OUTPUT

```
(3, 2)
300
```

The 2-D example is a 3-row × 2-column matrix — the same `patients` table idea from §9.5, now with fast maths.

11.3 Element-wise arithmetic — the vectorized advantage

On a plain list, adding 5 to every dose requires a loop. On a NumPy array, `arr + 5` does it in one expression — and every arithmetic operator works this way (element-wise):

```
doses = np.array([500, 750, 1000])
print(doses + 100)
print(doses * 2)
print(doses / 2)
```

OUTPUT

```
[ 600  850 1100]
[1000 1500 2000]
[250. 375. 500.]
```

Note division yields floats — the same `/`-always-floats rule from Unit I, now applied to whole arrays at once.

11.4 Aggregation functions

Function	Purpose
`.mean()`	average of all elements
`.sum()`	total
`.max()` / `.min()`	largest / smallest
`.std()`	standard deviation

```
doses = np.array([500, 750, 1000, 1250])
print(doses.mean())
print(doses.sum())
print(doses.max(), doses.min())
```

OUTPUT

```
875.0
3500
1250 500
```

11.5 Slicing and boolean filtering

Slicing works exactly like list slicing:

```
doses = np.array([500, 750, 1000, 1250])
print(doses[1:4])
```

OUTPUT

```
[ 750 1000 1250]
```

Boolean filtering is the NumPy superpower: a comparison on an array yields an array of True/False; using that as an index selects only the True elements:

```
print(doses > 600)
print(doses[doses > 600])
```

OUTPUT

```
[False  True  True  True]
[1000 1250]
```

More examples:

```
temps = np.array([36.5, 38.2, 37.0, 38.9, 36.8])
print(temps[temps > 37.5])
```

OUTPUT

```
[38.2 38.9]
```

```
bp = np.array([118, 125, 160, 110, 145])
print(np.sort(bp))
print(np.where(bp > 130))
```

OUTPUT

```
[110 118 125 145 160]
(array([2, 4]),)
```

`np.where(condition)` returns the *positions* where the condition holds — here, indices 2 and 4 carry readings above 130. Together with boolean filtering (`bp[bp > 130]` gives the *values*), these two cover most screening questions a pharmacist will ask of an array.

```
weights = np.array([60, 70, 55, 80])
safe = weights[(weights >= 50) & (weights <= 75)]
print(safe)
```

OUTPUT

```
[60 70 55]
```

(Note: NumPy uses `&` and `|` for element-wise and/or, with brackets around each comparison.)

Chapter summary. `import numpy as np`; create arrays with `np.array` / `zeros` / `ones` / `arange` / `linspace`; check `.shape` and `.dtype`. Arithmetic is element-wise (`arr + 5`, `arr * 2`); aggregation with `.mean()`, `.sum()`, `.max()`, `.min()`. Slicing copies list behaviour; boolean filtering `arr[arr > limit]` selects matching elements and `np.where` finds their positions — the everyday tools for screening health data at speed.

11.6 Two-dimensional arrays — tables of numbers

A **2-D array** is a matrix — rows and columns of numbers stored as one NumPy object. It is the natural shape for multi-patient, multi-time-point data:

```
import numpy as np

# Each row = one patient; each column = one time point
data = np.array([
    [45.2, 38.6, 30.1, 22.5],    # Patient 1
    [50.1, 42.8, 35.5, 28.0],    # Patient 2
    [38.6, 32.0, 25.3, 19.8],    # Patient 3
])
print(data.shape)
```

OUTPUT

```
(3, 4)
```

Three rows (patients) and four columns (time points). Indexing uses `[row, column]`:

```
print(data[0, 0])     # Patient 1, time 0
print(data[1, 2])     # Patient 2, time 2
print(data[2, -1])    # Patient 3, last reading
```

OUTPUT

```
45.2
35.5
19.8
```

Slicing 2-D arrays:

```
print(data[0, :])     # all columns of row 0 (Patient 1's full curve)
```

OUTPUT

```
[45.2 38.6 30.1 22.5]
```

```
print(data[:, 0])     # all rows of column 0 (everyone's baseline)
```

OUTPUT

```
[45.2 50.1 38.6]
```

```
print(data[0:2, 1:3]) # rows 0-1, columns 1-2
```

OUTPUT

```
[[38.6 30.1]
 [42.8 35.5]]
```

The `axis` parameter — row-wise or column-wise:

Aggregation functions accept an `axis` argument that controls the direction of computation:

axis=	Direction	Effect
`axis=0`	down the rows	one result per column
`axis=1`	across the columns	one result per row
(omitted)	the whole array	one single number

```
print(data.mean(axis=0))    # mean per column = per time point
```

OUTPUT

```
[44.63333333 37.8        30.3        23.43333333]
```

```
print(data.mean(axis=1))    # mean per row = per patient
```

OUTPUT

```
[34.1  39.1  28.925]
```

```
print(data.max(axis=1))     # Cmax per patient (row maximum)
```

OUTPUT

```
[45.2 50.1 38.6]
```

```
print(data.sum(axis=0))     # total per time point
```

OUTPUT

```
[133.9 113.4  90.9  70.3]
```

11.7 reshape() — changing the shape without changing the data

`reshape()` rearranges an array's elements into a new shape (the total count must match):

```
flat = np.arange(0, 12)
matrix = flat.reshape(3, 4)     # 12 elements -> 3 rows x 4 columns
print(matrix)
```

OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
print(matrix.reshape(4, 3))    # same data, 4 rows x 3 columns
```

OUTPUT

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
print(flat.reshape(2, 6))      # 2 rows x 6 columns
```

OUTPUT

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

Use `-1` to let NumPy infer one dimension:

```
print(flat.reshape(3, -1))     # "3 rows, however many columns fit"
```

OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
print(flat.reshape(-1, 1))     # one column (12 rows) — for column vectors
```

OUTPUT

```
[[ 0]
 [ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]
 [11]]
```

11.8 Broadcasting — operating on mismatched shapes

Broadcasting is NumPy's rules for combining arrays of different shapes. The simplest and most useful case: a scalar (single number) operates element-wise on an entire array (you have already seen `arr * 2`); the next level is a 1-D array operating on a 2-D array:

```
weights = np.array([60, 70, 55])          # 1-D: 3 patients
rate = 5                                    # scalar: mg per kg

doses = weights * rate                      # broadcasts: scalar to every
element
print(doses)
```

OUTPUT

```
[300 350 275]
```

```
# 1-D array broadcasting against a 2-D array
time_points = np.array([0, 2, 4, 6])        # 1-D: 4 time points
baseline = np.array([[45.2], [50.1], [38.6]])  # 2-D: 3x1 (one per patient)

# Broadcasting: 3x1 - 1x4 -> 3x4 (each patient's decline from baseline)
decline = baseline - time_points
print(decline)
```

OUTPUT

```
[[45.2 43.2 41.2 39.2]
 [50.1 48.1 46.1 44.1]
 [38.6 36.6 34.6 32.6]]
```

NumPy stretched the `(3, 1)` array across 4 columns and the `(1, 4)` array down 3 rows, producing a `(3, 4)` result. The rule: dimensions of size 1 are stretched to match; dimensions must otherwise be equal.

```
# More practical: a conversion factor applied to every element
temps_f = np.array([[98.6, 99.2], [97.8, 98.4]])
temps_c = (temps_f - 32) * 5 / 9           # broadcasting a scalar expression
print(np.round(temps_c, 1))
```

OUTPUT

```
[[37.  37.3]
 [36.6 36.9]]
```

Quick checks 11. (a) `np.arange(0, 6, 2)`? (b) What does `doses.shape` return for a 1-D array of 4 elements? (c) `np.array([10, 20, 30]).mean()`? (d) What does `temps[temps > 37]` do? (e) Why is `arr * 2` faster than a loop? (f) What is `data.shape` for a 3-row × 4-column array? (g) What does `data.mean(axis=1)` compute? (h) What does `np.arange(12).reshape(3, 4)` produce? (i) What is broadcasting?

Chapter 12 — CSV Files and Healthcare Datasets

12.1 What CSV is

CSV (Comma-Separated Values) is the plain-text interchange format for tables: every line holds one row, and commas split it into columns. Hospital systems, lab instruments and spreadsheet software all export CSV — it is the lingua franca of data exchange.

A small patient file, `patients.csv`, might look like this in plain text:

```
Name,Age,Dose_mg
Ravi,32,500
Sana,45,250
Meera,28,750
```

12.2 Reading a CSV file — the csv module

Python's built-in `csv` module parses the commas for you. The pattern: `open` the file, wrap it in `csv.reader`, loop the rows (each row arrives as a **list of strings**):

```
import csv
with open("patients.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

OUTPUT

```
['Name', 'Age', 'Dose_mg']
['Ravi', '32', '500']
['Sana', '45', '250']
['Meera', '28', '750']
```

Two pitfalls the syllabus expects you to know:

1. **Everything arrives as text** — `'500'` is a string. Cast with `int()` or `float()` before any arithmetic.
2. **On Windows, always pass `newline=""` when writing** — forgetting it inserts blank lines between rows.

12.3 Writing a CSV file

```
import csv
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "Dose_mg"])
    writer.writerow(["Ravi", 32, 500])
    writer.writerow(["Sana", 45, 250])
```

The file `output.csv` now contains the same three lines shown in §12.1.

12.4 DictReader — rows as dictionaries

`csv.DictReader` reads the header line as column names, then delivers every following row as a **dictionary** keyed by those names — usually more convenient than positional lists:

```
import csv
with open("patients.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["Name"], row["Dose_mg"])
```

OUTPUT

```
Ravi 500
Sana 250
Meera 750
```

12.5 Handling a missing file safely

```
try:
    with open("patients.csv", "r") as file:
        data = file.read()
    print("File loaded:", len(data), "characters")
except FileNotFoundError:
    print("Error: patients.csv not found")
```

OUTPUT

```
Error: patients.csv not found
```

(`try/except` catches the error and prints a friendly message instead of crashing — a preview of defensive programming.)

12.6 Structured healthcare datasets (syllabus point)

A **structured dataset** arranges information into **rows** (each row one patient, sample or observation) alongside **columns** (each column one attribute) — the same arrangement as a spreadsheet or database table:

Column	Example	Data type
Patient_ID	P1024	string
Age	45	integer
Drug_Name	Metformin	string
Dose_mg	500	integer
BP_Systolic	128	integer
Adverse_Reaction	False	boolean

The dataset's **shape** is (rows, columns) — `(500, 6)` means 500 patient records with 6 fields each. Structure matters because it enables filtering, sorting and grouping ("find all patients above 60 on Metformin") — operations that are nearly impossible on unorganised text.

Dirty data preview (fully treated in Unit IV): missing values (`NaN`), wrong types (`'500'` stored as text), duplicate records — with Pandas fixes `dropna()`, `astype(int)`, `drop_duplicates()`.

12.7 Importing a pharmaceutical dataset — the complete workflow

Combining everything in this unit, here is a small but complete analysis on a pharmacy stock dataset stored as CSV:

```
import csv

# Build a list of dictionaries from the file
inventory = []
with open("drug_stock.csv", "r") as file:
    for row in csv.DictReader(file):
        inventory.append({
            "name": row["Drug_Name"],
            "qty": int(row["Stock_Qty"]),
            "price": float(row["Price"]),
        })

# Low-stock report
low_stock = [item["name"] for item in inventory if item["qty"] < 20]
print("Low stock:", low_stock)

# Total inventory value
total_value = sum(item["qty"] * item["price"] for item in inventory)
print(f"Total value: Rs. {total_value:.2f}")
```

OUTPUT

```
Low stock: ['Aspirin', 'Cetirizine']
Total value: Rs. 344.40
```

Every line of that program is a Unit III skill: `open` + `csv.DictReader` reads the file; the comprehension filters; `sum` with a generator expression totals; the f-string formats. Change the file, re-run, and the report updates — the shape of every data pipeline in Unit IV and beyond.

Chapter summary. CSV is plain-text tabular data — the exchange format of healthcare systems. `csv.reader` yields rows as string lists (cast numbers before maths); `csv.writer` with `newline=""` writes; `DictReader/DictWriter` key rows by header names; `try/except FileNotFoundError` guards missing files. A structured dataset has rows (records) and columns (fields) with a (rows, columns) shape; missing values, wrong types and duplicates are the three standard dirt problems. The §12.7 workflow — read → cast → filter → total — is the template for every small pharmacy dataset task.

Quick checks 12. (a) Why must CSV numbers be cast before arithmetic? (b) What does `newline=""` prevent? (c) `DictReader` vs `reader` — one advantage? (d) A dataset with 200 rows and 8 columns has what shape? (e) Name three data-quality problems and their Pandas fixes.

Unit III — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Lists, tuples, and dictionaries	Ch 9–10
Indexing and slicing lists; basic operations on lists and dictionaries	Ch 9 (§9.2–9.3), Ch 10 (§10.2–10.3)
String manipulation techniques	Ch 10 (§10.6)
Introduction to NumPy arrays; basic operations	Ch 11
Reading and writing CSV files	Ch 12 (§12.1–12.5)
Understanding structured healthcare datasets; importing pharmaceutical datasets	Ch 12 (§12.6–12.7)

Twenty-question exercise bank

Predict the output (1–8).

1. `drugs = ["A", "B", "C"]; drugs.append("D"); print(len(drugs))`
2. `print([10, 20, 30][::-1])`
3. `vitals = (120, 80); print(vitals[-1])`
4. `d = {"x": 1, "y": 2}; d["z"] = 3; print(len(d))`
5. `import numpy as np; print(np.array([1, 2, 3]) * 3)`
6. `print(np.arange(0, 10, 5))`
7. `temps = np.array([36, 38, 37, 39]); print(temps[temps > 37])`
8. `print("  strip me  ".strip())`

Short answers (9–14).

9. List vs tuple — two differences and one use-case each.
10. Why does `new = old` not copy a list? Show the correct way.
11. What is a list comprehension? Give one with a condition.
12. Boolean filtering in one sentence, with a temperature example.
13. Why is a dictionary better than parallel lists for a patient record?
14. Name three data-quality problems in healthcare CSV files.

Write programs (15–20).

15. Build a list of 5 drug names; print the first, last, and sorted list.
16. Count fevers in a list of temperatures using a comprehension.

17. Create a dictionary for one drug (name, dose, stock); add a price key; loop `.items()`.
18. `np.linspace(0, 10, 5)` — what array? Then compute its mean.
19. Read `patients.csv` with DictReader and print each patient's name and dose.
20. Given a list of tuples `[(120, 80), (130, 85)]`, unpack and print each as `sys/dia mmHg`.

Answers to selected exercises

1. `4`. 2. `[30, 20, 10]`. 3. `80`. 4. `3`. 5. `[3 6 9]`. 6. `[0 5]`. 7. `[38 39]`. 8. `strip me`. 10. The two names share one list object; use `old.copy()`. 11. `[expr for x in list if cond]`; e.g. `[t for t in temps if t > 37.5]`. 12. `arr[arr > limit]` selects only elements above the limit. 13. Keys are self-describing (`p["dose_mg"]` vs `row[2]`); no risk of column-order drift. 18. `[0. 2.5 5. 7.5 10.]`, mean `5.0`.

Important points for examination

1. A **list** (`[]`) is ordered and mutable; a **tuple** (`()`) is ordered and immutable; a **dict** (`{}`) maps unique keys to values and is mutable.
2. List indexing starts at 0; slices `[a:b]` exclude b; `[::-1]` reverses; `IndexError` on reading past `len-1`.
3. Core list methods: `append` (end), `insert` (position), `remove` (value), `pop` (position, returns item), `sort`, `reverse`, `count`, `index`, `extend`, `clear`.
4. `new = old` aliases a list; `.copy()` creates an independent one — a classic examination trap.
5. A list comprehension `[expr for item in collection if condition]` builds a list in one readable line; the same idea works for dictionaries.
6. A tuple of one item needs a trailing comma: `(500,)`.
7. Dict access by key `d["k"]`; safe access with `d.get("k", default)`; loop with `for k, v in d.items()`.
8. Useful string tools: `join`, `startswith/endswith`, `isdigit`, `title`, `splitlines`, `zfill`, `format`, `count`, `lstrip/rstrip` — beyond the Unit I core of `split`, `strip`, `replace`, `find`, `in`.
9. NumPy arrays support **element-wise** arithmetic (`arr * 2`, `arr + 5`) — vectorization replaces loops and is dramatically faster.
10. Create arrays with `np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace`; inspect with `.shape` and `.dtype`; aggregate with `.mean()`, `.sum()`, `.max()`, `.min()`.
11. Boolean filtering `arr[arr > limit]` selects matching elements; `np.where(condition)` gives their positions.
12. CSV = comma-separated plain text. `csv.reader` yields string-list rows (cast numbers!); `csv.writer` needs `newline=""` on Windows; `DictReader` keys rows by header.

13. A structured dataset is rows × columns; its shape is `(rows, columns)`; typical dirt: missing values, wrong types, duplicates.
14. The standard workflow: read with DictReader → cast to int/float → filter with a comprehension → aggregate with sum or a NumPy method — exactly the §12.7 inventory report.

Sources for this unit: course decks and the book plan's source matrix; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 3* notes, MCA R24 lecture notes Unit II (data structures section), Yamuna Devi, Jeyabharathi & Vajpayee *Python Programming* (GCS), MRCET R17 notes, numpy.org tutorials, and the curated learn.biodockify.com catalog (numpy, pandas, trekhleb/learn-python, exercism/python, W3Schools). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source (zero 7-word overlaps). Full credits: see the book's Sources and Acknowledgements page.

UNIT IV

Data Handling with Pandas

Chapters 13–15 · 6 teaching hours

Chapter 13 — Pandas Fundamentals

13.1 What Pandas is and why it exists

Pandas ranks as the leading data-analysis package in the Python ecosystem — it converts raw numeric columns into figures suitable for journals and reports. Sitting above NumPy (Unit III), it supplies fast, flexible containers purpose-built for the spreadsheet-shaped data pharmacists work with: PK study sheets, ADR reports, hospital records, clinical trial exports.

Install with `pip install pandas`, import with the universal alias:

```
import pandas as pd
```

What Pandas adds over the raw Python of Unit III:

- **Direct file access** — one function reads an entire CSV or Excel file into a ready-to-analyse table.
- **Missing-data handling** — clinical datasets arrive with holes; Pandas represents each as `NaN` and provides clean removal or replacement.
- **Fast filtering** — "all patients with Cmax above 50" is one line, not a loop.
- **Column operations** — rename, retype, transform or compute new columns in a single expression.
- **Grouping and statistics** — average dose per drug category, count of ADRs per severity band, in a single expression.
- **Integration** — Pandas works seamlessly with NumPy (Unit III) and Matplotlib (Unit V).

13.2 The Series — one labelled column

A **Series** behaves as a single indexed column: every value carries a readable label, so you address items by name rather than by position.

```
import pandas as pd

doses = pd.Series([500, 850, 1000], index=["Aspirin", "Dolo", "Meftal"])
print(doses)
```

```
OUTPUT
Aspirin     500
Dolo        850
Meftal     1000
dtype: int64
```

Three things to notice: the index labels appear on the left; the values on the right; and the `dtype` line reports the data type of the column.

Series attributes:

Attribute	Returns
`.values`	the raw values as a NumPy array
`.index`	the index labels
`.dtype`	the data type
`.shape`	dimensions as a tuple

Five Series examples:

```
s = pd.Series([10, 20, 30, 40])
print(s.values)
```

```
OUTPUT
[10 20 30 40]
```

```
print(s.index)
```

```
OUTPUT
RangeIndex(start=0, stop=4, step=1)
```

```
print(s.mean(), s.sum(), s.max(), s.min())
```

```
OUTPUT
25.0 100 40 10
```

```
named = pd.Series([500, 250], index=["Paracetamol", "Aspirin"])
print(named["Aspirin"])
```

```
OUTPUT
250
```

```
print(named[named > 300])       # boolean filtering works on Series too
```

```
OUTPUT
Paracetamol    500
dtype: int64
```

13.3 The DataFrame — a full table

A **DataFrame** is Pandas' flagship structure: a labelled grid in which both rows and columns carry names, built from multiple Series aligned over a common index. Every real dataset in this unit and Unit V lives inside one.

Term	Meaning
Column	one attribute or field (e.g. Age)
Row / record	one observation (one patient, one ADR)
Index	row labels (default numbering: 0, 1, 2, …)

Build one from a dictionary — keys become column names, values become the columns:

```
data = {
    "Patient_ID": ["P101", "P102", "P103", "P104"],
    "Age": [34, 58, 45, 28],
    "Cmax": [45.2, 38.6, 50.1, 42.8],
    "Tmax": [2.1, 1.8, 2.4, 1.9],
    "Drug": ["Metformin", "Metformin", "Amlodipine", "Metformin"],
}
df = pd.DataFrame(data)
print(df)
```

OUTPUT

```
  Patient_ID  Age  Cmax  Tmax        Drug
0       P101   34  45.2   2.1   Metformin
1       P102   58  38.6   1.8   Metformin
2       P103   45  50.1   2.4  Amlodipine
3       P104   28  42.8   1.9   Metformin
```

Selecting from a DataFrame — five examples:

```
print(df["Age"])                      # one column -> a Series
```

OUTPUT

```
0    34
1    58
2    45
3    28
Name: Age, dtype: int64
```

```
print(df[["Age", "Cmax"]])            # multiple columns -> a DataFrame
```

OUTPUT

```
   Age  Cmax
0   34  45.2
1   58  38.6
2   45  50.1
3   28  42.8
```

```
print(df.columns)                    # list of column names
```

OUTPUT

```
Index(['Patient_ID', 'Age', 'Cmax', 'Tmax', 'Drug'], dtype='object')
```

```
print(df.shape)                      # (rows, columns)
```

OUTPUT

```
(4, 5)
```

```
print(round(df["Cmax"].mean(), 3))    # statistics on one column
```

OUTPUT

```
44.175
```

13.4 Reading CSV and Excel files (syllabus point)

The whole point of Pandas is loading real data. Two functions cover the syllabus file formats:

```
pk_data = pd.read_csv("pk_study.csv")
adr_data = pd.read_excel("adr_reports.xlsx", sheet_name="Sheet1")
```

Function	File type	Key parameters
`pd.read_csv()`	`.csv`	`filepath`, `sep=","`, `header=0`
`pd.read_excel()`	`.xlsx` / `.xls`	`filepath`, `sheet_name`
`df.to_csv()`	saves as CSV	`filepath`, `index=False`
`df.to_excel()`	saves as Excel	`filepath`, `index=False`

Note: `pd.read_excel()` for `.xlsx` files requires the `openpyxl` library (`pip install openpyxl`). A `ModuleNotFoundError` on first use usually means this is missing.

Pass `index=False` when saving to avoid writing the row numbers as an extra column.

13.5 Inspecting datasets (syllabus point)

Before analysing any dataset, inspect its structure, size and quality:

Function	What it shows
`df.head(n)`	the first n rows (default 5)
`df.tail(n)`	the last n rows (default 5)

Function	What it shows
`df.info()`	column names, data types, non-null counts, memory
`df.describe()`	count, mean, std, min, 25%, 50%, 75%, max

Five inspection examples on a PK study dataset:

```
pk_data = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P103", "P104", "P105", "P106",
                   "P107", "P108", "P109", "P110"],
    "Age": [34, 58, 45, 28, 62, 51, 39, 47, 55, 33],
    "Cmax": [45.2, 38.6, 50.1, 42.8, 47.5, 39.2, 44.0, 41.3, 46.8, 43.5],
    "Drug": ["Metformin", "Metformin", "Amlodipine", "Metformin",
             "Amlodipine", "Metformin", "Ibuprofen", "Metformin",
             "Amlodipine", "Ibuprofen"],
})
print(pk_data.head(3))
```

OUTPUT

```
  Patient_ID  Age  Cmax        Drug
0       P101   34  45.2   Metformin
1       P102   58  38.6   Metformin
2       P103   45  50.1  Amlodipine
```

```
print(pk_data.tail(2))
```

OUTPUT

```
  Patient_ID  Age  Cmax        Drug
8       P109   55  46.8  Amlodipine
9       P110   33  43.5   Ibuprofen
```

```
print(pk_data.shape)
```

OUTPUT

```
(10, 4)
```

```
print(pk_data["Cmax"].describe())
```

OUTPUT

```
count    10.000000
mean     43.900000
std       3.610595
min      38.600000
25%      42.225000
50%      43.750000
75%      45.900000
max      50.100000
Name: Cmax, dtype: float64
```

```
print(pk_data["Drug"].value_counts())
```

OUTPUT

```
Drug
Metformin     4
Amlodipine    3
Ibuprofen     3
Name: count, dtype: int64
```

`value_counts()` — introduced here — counts how many times each unique value appears. It is the fastest frequency table in Pandas and appears in every ADR analysis.

13.6 Sorting and ranking

`sort_values()` reorders rows by one or more columns; `sort_index()` restores the original row order:

```
sorted_by_cmax = pk_data.sort_values("Cmax", ascending=False)
print(sorted_by_cmax[["Patient_ID", "Cmax"]].head(3))
```

OUTPUT

```
  Patient_ID  Cmax
2       P103  50.1
4       P105  47.5
8       P109  46.8
```

```
sorted_by_age = pk_data.sort_values(["Drug", "Age"])
print(sorted_by_age[["Drug", "Age"]].head(4))
```

OUTPUT

```
          Drug  Age
2  Amlodipine   45
4  Amlodipine   55
8  Amlodipine   62
9   Ibuprofen   33
```

```
print(pk_data["Cmax"].rank(ascending=False).head(3))
```

OUTPUT

```
0    6.0
1   10.0
2    1.0
Name: Cmax, dtype: float64
```

`.rank()` gives each row's position if the column were sorted — useful for identifying the top-N performers without reordering.

13.7 Adding, removing and renaming columns

```
# Add a new computed column
pk_data["Dose_per_kg"] = pk_data["Cmax"] / 70    # rough mg/kg estimate
print(pk_data[["Patient_ID", "Cmax", "Dose_per_kg"]].head(2))
```

OUTPUT

```
  Patient_ID  Cmax  Dose_per_kg
0       P101  45.2     0.645714
1       P102  38.6     0.551429
```

```
# Add a column from a list
pk_data["Ward"] = ["A", "B", "A", "C", "B", "A", "C", "B", "A", "C"]
print(pk_data["Ward"].value_counts())
```

OUTPUT

```
Ward
A    4
B    3
C    3
Name: count, dtype: int64
```

```
# Drop a column
pk_dropped = pk_data.drop(columns=["Dose_per_kg", "Ward"])
print(pk_dropped.columns)
```

OUTPUT

```
Index(['Patient_ID', 'Age', 'Cmax', 'Drug'], dtype='object')
```

```
# Rename columns
pk_renamed = pk_data.rename(columns={"Cmax": "Peak_Conc", "Drug": "Medication"})
print(pk_renamed.columns)
```

OUTPUT

```
Index(['Patient_ID', 'Age', 'Peak_Conc', 'Medication', 'Dose_per_kg', 'Ward'],
dtype='object')
```

```
# Column from a condition (a Boolean flag)
pk_data["High_Cmax"] = pk_data["Cmax"] > 45
print(pk_data["High_Cmax"].sum(), "patients above 45")
```

OUTPUT

```
4 patients above 45
```

Chapter summary. Pandas (import as `pd`) delivers two structures: the **Series** (a labelled one-dimensional column with built-in statistics, boolean filtering, and `.values`/`.index`/`.dtype` attributes) and the **DataFrame** (a labelled grid built from multiple Series). `read_csv()` and `read_excel()` load files; `to_csv()`/`to_excel()` with `index=False` save. Inspect with `head`/`tail`/`info`/`describe`/`shape`/`value_counts`. Sort with `sort_values()`, rank with `.rank()`.

Add columns by assignment, drop with `.drop(columns=[...])`, rename with `.rename(columns={...})`.

Quick checks 13. (a) Series vs DataFrame in one sentence each. (b) `df.shape` for 120 rows, 6 columns? (c) Which function gives count, mean, std, min, quartiles, max in one call? (d) What does `value_counts()` return? (e) How do you add a column that is True when Age > 50?

Chapter 14 — Advanced DataFrame Operations

14.1 Why more operations matter

Real pharmacy analysis rarely involves a single clean table. You merge patient demographics with lab results, join ADR reports with drug catalogues, clean text columns, convert dates, and transform values column-wide. This chapter covers the operations between "load the file" and "make the chart."

14.2 String methods on columns

Every string method from Unit I works on an entire DataFrame column via the `.str` accessor — no loop needed:

```
adr = pd.DataFrame({
    "Drug_Name": ["  metformin  ", "AMOXICILLIN", "ibuprofen"],
    "Severity": ["Mild", "Moderate", "Serious"],
})
adr["Drug_Name"] = adr["Drug_Name"].str.strip().str.title()
print(adr["Drug_Name"])
```

OUTPUT

```
0      Metformin
1    Amoxicillin
2      Ibuprofen
Name: Drug_Name, dtype: object
```

Five `.str` examples:

```
# Check if a column value starts with a prefix
adr["Is_Antibiotic"] = adr["Drug_Name"].str.startswith("Amox")
print(adr["Is_Antibiotic"])
```

OUTPUT

```
0    False
1     True
2    False
Name: Is_Antibiotic, dtype: bool
```

```
# Replace text in a column
adr["Severity"] = adr["Severity"].str.replace("Moderate", "Mild")
print(adr["Severity"])
```

OUTPUT

```
0      Mild
1      Mild
2    Serious
Name: Severity, dtype: object
```

```
# Extract the first letter
adr["First_Letter"] = adr["Drug_Name"].str[0]
print(adr["First_Letter"])
```

OUTPUT

```
0    M
1    A
2    I
Name: First_Letter, dtype: object
```

```
# Check if all values are uppercase
print(adr["Severity"].str.isupper())
```

OUTPUT

```
0    False
1    False
2    False
Name: Severity, dtype: bool
```

```
# Split a dose string like "500 mg" into parts
doses = pd.DataFrame({"Dose": ["500 mg", "250 mg", "1000 mg"]})
doses[["Amount", "Unit"]] = doses["Dose"].str.split(expand=True)
print(doses)
```

OUTPUT

```
      Dose Amount Unit
0   500 mg    500   mg
1   250 mg    250   mg
2  1000 mg   1000   mg
```

The `.str.split(expand=True)` pattern is the bridge from text to numbers — it turns `"500 mg"` into separate `Amount` and `Unit` columns, ready for `astype(float)`.

14.3 Applying functions to columns — apply() and map()

`.apply()` runs any function on every value in a column; `.map()` translates values using a dictionary:

```
pk = pd.DataFrame({"Cmax": [45.2, 38.6, 50.1], "Age": [34, 58, 45]})

# apply a function to each value
pk["Cmax_Rounded"] = pk["Cmax"].apply(round)
print(pk["Cmax_Rounded"])
```

OUTPUT

```
0    45
1    39
2    50
Name: Cmax_Rounded, dtype: int64
```

```
# apply a lambda (anonymous one-line function)
pk["Age_Band"] = pk["Age"].apply(lambda a: "Senior" if a >= 60 else "Adult")
print(pk["Age_Band"])
```

OUTPUT

```
0    Adult
1    Senior
2    Adult
Name: Age_Band, dtype: object
```

```
# map: translate values using a lookup dictionary
severity_map = {"Mild": 1, "Moderate": 2, "Serious": 3}
adr2 = pd.DataFrame({"Severity": ["Mild", "Serious", "Moderate"]})
adr2["Severity_Score"] = adr2["Severity"].map(severity_map)
print(adr2)
```

OUTPUT

```
  Severity  Severity_Score
0     Mild               1
1  Serious               3
2 Moderate               2
```

```
# apply with a custom function
def dose_category(cmax):
    if cmax < 40:
        return "Low"
    elif cmax < 48:
        return "Normal"
    else:
        return "High"

pk["Category"] = pk["Cmax"].apply(dose_category)
print(pk[["Cmax", "Category"]])
```

OUTPUT

```
   Cmax Category
0  45.2   Normal
1  38.6      Low
2  50.1     High
```

```
# apply to entire rows with axis=1
pk["Risk"] = pk.apply(lambda row: "High" if row["Age"] > 50 and row["Cmax"] > 45
else "Low", axis=1)
print(pk["Risk"])
```

```
OUTPUT
0     Low
1    High
2     Low
Name: Risk, dtype: object
```

The `axis=1` form gives the function the entire row (accessed as `row["col"]`), enabling conditions that span multiple columns — the Pandas equivalent of the nested conditions from Unit II.

14.4 Merging and joining DataFrames

Real analysis combines tables. `pd.merge()` joins two DataFrames on a shared column, exactly like a SQL JOIN:

```
patients = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P103"],
    "Name": ["Ravi", "Sana", "Meera"],
    "Age": [45, 38, 52],
})
lab_results = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P104"],
    "Cmax": [42.5, 38.9, 51.2],
    "Tmax": [2.1, 1.8, 2.5],
})
merged = pd.merge(patients, lab_results, on="Patient_ID", how="inner")
print(merged)
```

```
OUTPUT
  Patient_ID   Name  Age  Cmax  Tmax
0       P101   Ravi   45  42.5   2.1
1       P102   Sana   38  38.9   1.8
```

Join types:

how=	Behaviour	Rows in result
`"inner"`	only IDs present in **both** tables	intersection
`"left"`	all rows from the left table; NaN where the right has no match	left table's rows
`"right"`	all rows from the right table	right table's rows
`"outer"`	all rows from either table	union

```
# Left join: keep all patients, even those without lab results
left = pd.merge(patients, lab_results, on="Patient_ID", how="left")
print(left)
```

```
OUTPUT
  Patient_ID   Name  Age  Cmax  Tmax
0       P101   Ravi   45  42.5   2.1
1       P102   Sana   38  38.9   1.8
2       P103  Meera   52   NaN   NaN
```

Meera has no lab result, so her Cmax and Tmax appear as NaN — the missing-data problem of §15.1, now arrived via a merge.

```
# Outer join: all patients AND all lab results
outer = pd.merge(patients, lab_results, on="Patient_ID", how="outer")
print(outer)
```

```
OUTPUT
  Patient_ID   Name   Age  Cmax  Tmax
0       P101   Ravi   45.0  42.5   2.1
1       P102   Sana   38.0  38.9   1.8
2       P103  Meera   52.0   NaN   NaN
3       P104    NaN   NaN  51.2   2.5
```

```
# Concatenating DataFrames (stacking rows)
q1 = pd.DataFrame({"Month": ["Jan"], "ADR": [15]})
q2 = pd.DataFrame({"Month": ["Feb"], "ADR": [18]})
combined = pd.concat([q1, q2], ignore_index=True)
print(combined)
```

```
OUTPUT
  Month  ADR
0   Jan   15
1   Feb   18
```

```
# Adding a column from another DataFrame (same index)
pk["Ward"] = pd.Series(["A", "B", "C"])
print(pk[["Cmax", "Ward"]])
```

```
OUTPUT
   Cmax Ward
0  45.2    A
1  38.6    B
2  50.1    C
```

14.5 Datetime handling in Pandas

ADR reports and PK studies always carry dates. Pandas parses date strings into `datetime` objects, enabling sorting, filtering by date range, and extracting components:

```
adr = pd.DataFrame({
    "Date": ["2026-01-15", "2026-02-20", "2026-03-10"],
    "Drug": ["Metformin", "Ibuprofen", "Amoxicillin"],
})
adr["Date"] = pd.to_datetime(adr["Date"])
print(adr.dtypes)
```

OUTPUT

```
Date            datetime64[ns]
Drug                    object
dtype: object
```

Five datetime examples:

```
# Extract year, month, day
adr["Year"] = adr["Date"].dt.year
adr["Month"] = adr["Date"].dt.month
print(adr[["Date", "Year", "Month"]])
```

OUTPUT

```
        Date  Year  Month
0 2026-01-15  2026      1
1 2026-02-20  2026      2
2 2026-03-10  2026      3
```

```
# Filter by date range
jan_feb = adr[(adr["Date"] >= "2026-01-01") & (adr["Date"] <= "2026-02-28")]
print(len(jan_feb))
```

OUTPUT

```
2
```

```
# Sort chronologically
sorted_adr = adr.sort_values("Date")
print(sorted_adr["Drug"].tolist())
```

OUTPUT

```
['Metformin', 'Ibuprofen', 'Amoxicillin']
```

```
# Day of the week (0=Monday)
adr["Weekday"] = adr["Date"].dt.dayofweek
print(adr["Weekday"].tolist())
```

OUTPUT

```
[3, 4, 1]
```

```
# Days since a reference date
adr["Days_Since_Start"] = (adr["Date"] - adr["Date"].min()).dt.days
print(adr["Days_Since_Start"].tolist())
```

```
OUTPUT
[0, 36, 54]
```

Chapter summary. The `.str` accessor applies string methods to entire columns (`.str.strip()`, `.str.title()`, `.str.split(expand=True)`); `.apply()` runs any function (including `lambda`) on each value or row; `.map()` translates via a lookup dictionary. `pd.merge(on, how)` joins tables — inner (intersection), left (all left rows), outer (union); `pd.concat` stacks rows; `pd.to_datetime()` + `.dt` accessor enables date filtering, sorting and component extraction. These operations bridge the gap between raw files and the cleaning/filtering/grouping of Chapter 15.

Quick checks 14. (a) Write one line that strips and title-cases a `Drug_Name` column. (b) `apply()` vs `map()` — when would you use each? (c) What join type keeps all rows from both tables? (d) How do you extract the month from a datetime column? (e) What does `pd.concat([df1, df2])` do?

Chapter 15 — Cleaning, Filtering and Grouping

15.1 Missing values — the NaN problem (syllabus point)

Real PK/ADR datasets arrive with holes. Pandas marks each missing entry as **NaN** (Not a Number), and the first cleaning job is to find and handle them.

Function	Purpose
`df.isnull()`	True/False for every cell — is it missing?
`df.isnull().sum()`	count of missing values per column
`df.dropna()`	removes rows (or columns) containing NaN
`df.fillna(value)`	replaces NaN with a value (0, mean, etc.)
`df.drop_duplicates()`	removes duplicate rows
`df.astype(type)`	converts a column to a different type

Five cleaning examples:

```
pk = pd.DataFrame({
    "Age": [34, None, 45, 28, None, 52],
    "Cmax": [45.2, 38.6, None, 42.8, 47.5, 39.2],
    "Drug": ["A", "B", "A", "C", "B", "A"],
})
print(pk.isnull().sum())
```

OUTPUT

```
Age     2
Cmax    1
dtype: int64
```

```
# Fill missing ages with the column's mean
pk["Age"] = pk["Age"].fillna(pk["Age"].mean())
print(pk["Age"])
```

OUTPUT

```
0    34.0
1    39.8
2    45.0
3    28.0
4    39.8
5    52.0
Name: Age, dtype: float64
```

```
# Drop rows where Cmax is missing (the reading is unusable)
pk_clean = pk.dropna(subset=["Cmax"])
print(pk_clean.shape)
```

OUTPUT

```
(5, 3)
```

```
# Remove duplicates
pk = pk.drop_duplicates()
print(pk.shape)
```

OUTPUT

```
(5, 3)
```

```
# Convert a text column to numeric
df = pd.DataFrame({"Dose_mg": ["500", "250", "750"]})
df["Dose_mg"] = df["Dose_mg"].astype(float)
print(df["Dose_mg"].mean())
```

OUTPUT

```
500.0
```

The cleaning decision — fill with the mean, drop the row, or something else — is clinical, not computational. Filling an age with the average is convenient; filling a creatinine with the average could be dangerous. The pharmacist decides; Pandas executes.

15.2 Filtering and selecting data (syllabus point)

Filtering answers screening questions: *which patients are above 60? which readings exceed a threshold?* The syntax is one line: square brackets containing a condition on a column.

Selecting columns:

```
pk_data = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P103", "P104", "P105"],
    "Age": [34, 58, 45, 28, 62],
    "Cmax": [45.2, 38.6, 50.1, 42.8, 47.5],
    "Drug": ["Metformin", "Metformin", "Amlodipine", "Metformin", "Amlodipine"],
})
print(pk_data["Cmax"])
```

OUTPUT

```
0    45.2
1    38.6
2    50.1
3    42.8
4    47.5
Name: Cmax, dtype: float64
```

Filtering rows — seven patterns:

```
# 1. Single condition
high_cmax = pk_data[pk_data["Cmax"] > 45]
print(len(high_cmax))
```

OUTPUT

```
3
```

```
# 2. Multiple conditions: & (and)
elderly_high = pk_data[(pk_data["Age"] >= 45) & (pk_data["Cmax"] > 45)]
print(elderly_high[["Patient_ID", "Age", "Cmax"]])
```

OUTPUT

```
  Patient_ID  Age  Cmax
2       P103   45  50.1
4       P105   62  47.5
```

```
# 3. Multiple conditions: | (or)
priority = pk_data[(pk_data["Age"] >= 60) | (pk_data["Cmax"] > 48)]
print(len(priority))
```

OUTPUT

```
2
```

```
# 4. Filter by a list of values (isin)
target_drugs = ["Metformin", "Amlodipine"]
on_target = pk_data[pk_data["Drug"].isin(target_drugs)]
print(len(on_target))
```

OUTPUT

```
5
```

```
# 5. Filter by a range (between)
mid_range = pk_data[pk_data["Age"].between(30, 60)]
print(mid_range["Patient_ID"].tolist())
```

OUTPUT

```
['P101', 'P102', 'P103']
```

```
# 6. Negation (~)
not_metformin = pk_data[~(pk_data["Drug"] == "Metformin")]
print(not_metformin["Drug"].tolist())
```

OUTPUT

```
['Amlodipine', 'Amlodipine']
```

```
# 7. String contains
contains_cillin = pk_data[pk_data["Drug"].str.contains("dipine")]
print(contains_cillin["Drug"].tolist())
```

```
OUTPUT
['Amlodipine', 'Amlodipine']
```

loc[] and iloc[]:

Method	Selects by	Example
`df.loc[label, col]`	label or condition	`df.loc[0, "Age"]`
`df.iloc[i, j]`	integer position	`df.iloc[2, 2]`

```
print(pk_data.loc[0, "Age"])
print(pk_data.iloc[2, 2])
```

```
OUTPUT
34
50.1
```

15.3 Grouping and aggregation (syllabus point)

`groupby()` partitions rows according to the unique values found in a chosen column; whatever aggregation follows condenses each partition into a single summary line.

Group → count:

```
adr = pd.DataFrame({
    "Drug_Name": ["Metformin", "Amoxicillin", "Ibuprofen",
                  "Metformin", "Ibuprofen", "Metformin",
                  "Amlodipine", "Amoxicillin"],
    "Severity": ["Mild", "Moderate", "Serious", "Mild",
                 "Mild", "Moderate", "Serious", "Mild"],
    "Age": [58, 34, 62, 45, 28, 51, 67, 41],
})
counts = adr.groupby("Drug_Name")["Severity"].count()
print(counts)
```

```
OUTPUT
Drug_Name
Amoxicillin     2
Amlodipine      1
Ibuprofen       2
Metformin       3
Name: Severity, dtype: int64
```

Group → mean:

```
pk = pd.DataFrame({
    "Age_Group": ["Adult", "Adult", "Elderly", "Elderly", "Adult", "Elderly"],
    "Cmax": [45.2, 38.6, 50.1, 47.5, 42.0, 48.8],
})
avg = pk.groupby("Age_Group")["Cmax"].mean()
print(avg)
```

OUTPUT

```
Age_Group
Adult      41.933333
Elderly    48.800000
Name: Cmax, dtype: float64
```

More aggregation examples:

```
# Multiple functions at once
summary = pk.groupby("Age_Group")["Cmax"].agg(["mean", "max", "min"])
print(summary)
```

OUTPUT

```
              mean   max   min
Age_Group
Adult     41.933333  45.2  38.6
Elderly   48.800000  50.1  47.5
```

```
# Group by multiple columns
adr_summary = adr.groupby(["Drug_Name", "Severity"])["Age"].mean()
print(adr_summary)
```

OUTPUT

```
Drug_Name    Severity
Amoxicillin  Mild         41.0
             Moderate     34.0
Amlodipine   Serious      67.0
Ibuprofen    Mild         28.0
             Serious      62.0
Metformin    Mild         51.5
             Moderate     51.0
Name: Age, dtype: float64
```

```
# Sort a groupby result, descending
top = counts.sort_values(ascending=False)
print(top)
```

OUTPUT

```
Drug_Name
Metformin       3
Amoxicillin     2
Ibuprofen       2
Amlodipine      1
Name: Severity, dtype: int64
```

```
# Average age per severity band
avg_age = adr.groupby("Severity")["Age"].agg(["mean", "count"])
print(avg_age)
```

OUTPUT

```
              mean  count
Severity
Mild     43.000000      4
Moderate 42.500000      2
Serious  64.500000      2
```

15.4 The ADR case study — the complete workflow

The syllabus names PK study datasets and ADR reports as the target data. Here is a complete ADR analysis exercising every skill in this unit:

```
import pandas as pd

adr = pd.DataFrame({
    "Drug_Name": ["Metformin", "Amoxicillin", "Ibuprofen", "Metformin",
                  "Ibuprofen", "Metformin", "Amlodipine", "Amoxicillin",
                  "Metformin", "Atorvastatin"],
    "Severity": ["Mild", "Moderate", "Serious", "Mild",
                 "Mild", "Moderate", "Serious", "Mild",
                 "Serious", "Moderate"],
    "Age": [58, 34, 62, 45, 28, 51, None, 41, 67, 55],
    "Date": ["2026-01-15", "2026-01-20", "2026-02-01", "2026-02-10",
             "2026-02-15", "2026-03-01", "2026-03-05", "2026-03-10",
             "2026-03-15", "2026-03-20"],
})

# 1. Inspect
print(adr.shape)
print(adr.isnull().sum())

# 2. Clean — fill the missing age with the mean
adr["Age"] = adr["Age"].fillna(adr["Age"].mean())
adr["Date"] = pd.to_datetime(adr["Date"])

# 3. Filter — serious cases
serious = adr[adr["Severity"] == "Serious"]
print(f"Serious ADR cases: {len(serious)}")

# 4. Group — count per drug, sorted
summary = adr.groupby("Drug_Name")["Severity"].count()
print(summary.sort_values(ascending=False))

# 5. Cross-tabulate severity by drug
cross = adr.groupby(["Drug_Name", "Severity"]).size().unstack(fill_value=0)
print(cross)
```

```
OUTPUT
(10, 4)
Drug_Name    0
Severity     0
Age          1
Date         0
dtype: int64
Serious ADR cases: 3
Drug_Name
Metformin       4
Amoxicillin     2
Ibuprofen       2
Amlodipine      1
Atorvastatin    1
Name: Severity, dtype: int64
Severity    Mild  Moderate  Serious
Drug_Name
Amlodipine     0         0        1
Amoxicillin    1         1        0
Atorvastatin   0         1        0
Ibuprofen      1         0        1
Metformin      1         1        2
```

Every line is a unit skill: `shape` and `isnull().sum()` inspect; `fillna` and `to_datetime` clean; a condition filters; `groupby().count()` + `sort_values()` aggregates and ranks; `groupby([...]).size().unstack()` cross-tabulates. Change the dataset, re-run, and the report updates — this is the shape of Unit V's plots and of every real pharmacovigilance query.

Chapter summary. `isnull().sum()` counts gaps per column; `dropna(subset=[...])` removes incomplete rows; `fillna(mean)` patches numeric gaps. Filter with `df[df["col"] > x]`; join conditions via `&`/`|` each inside brackets; `isin()` matches a list; `between()` matches a range; `~` negates; `.str.contains()` searches text. Address by label with `loc[]`, position with `iloc[]`. `groupby("col")` partitions rows; `.count()`, `.mean()`, `.sum()`, `.agg([f1, f2])` condense each partition; `sort_values()` ranks; `groupby([c1, c2]).size().unstack()` cross-tabulates. The five-step workflow — Read → Inspect → Clean → Filter → Group — drives every analysis.

Quick checks 15. (a) What does `df.isnull().sum()` tell you? (b) Write one line: average Cmax per Drug group. (c) Filter patients where Drug is in `["A", "B"]`. (d) What does `groupby(["Drug", "Severity"]).size().unstack()` produce? (e) Why is filling a missing creatinine with the column mean a clinical decision?

Unit IV — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Introduction to the Pandas library	Ch 13 (§13.1)
Pandas Series and DataFrame structures	Ch 13 (§13.2–13.3)
Reading CSV and Excel files — PK datasets and ADR reports	Ch 13 (§13.4)
Inspecting datasets: head(), tail(), info(), describe()	Ch 13 (§13.5)
Data cleaning and handling missing values	Ch 15 (§15.1)
Filtering and selecting data based on conditions	Ch 15 (§15.2)
Grouping data and performing aggregation functions	Ch 15 (§15.3–15.4)

Twenty-question exercise bank

Predict the output (1–8).

1. `df.shape` for 120 rows, 6 columns?
2. `print(pd.Series([10, 20, 30]).mean())`
3. `df["A"].value_counts()` on `["x", "y", "x", "x", "y"]` — what is `df["A"].value_counts()["x"]`?
4. `df.head(2)` on a 5-row DataFrame — how many rows appear?
5. `len(df[df["Age"] > 50])` on ages [34, 58, 45, 62] — result?
6. `df["col"].str.upper()` on `["abc", "def"]` — result?
7. `df.groupby("Drug")["Dose"].mean()` — what does it produce?
8. `pd.merge(df1, df2, on="ID", how="inner")` — what rows appear?

Short answers (9–14).

9. What is the difference between a Series and a DataFrame?
10. Name the four inspection functions and what each shows.
11. `dropna()` vs `fillna()` — when would you use each?
12. What does `&` do in a Pandas filter? Why are brackets needed?
13. What does `apply()` do? Write one example with `lambda`.
14. What is the five-step Pandas workflow?

Write programs (15–20).

15. Create a 4-row DataFrame with Patient_ID, Age, Drug columns; print `head()` and `describe()`.

16. Load a CSV, add a column `Age_Group` using `apply(lambda ...)`.
17. Fill missing Cmax values with the column median; drop duplicate rows.
18. Filter patients above 50 on Metformin using `isin()`.
19. Group an ADR dataset by Severity and count; sort descending.
20. Merge a patients table with a lab-results table on Patient_ID (left join).

Answers to selected exercises

1. `(120, 6)`. 2. `20.0`. 3. `3`. 4. 2. 5. 2. 6. `['ABC', 'DEF']`. 7. A Series mapping each drug to its average dose. 8. Only IDs present in both tables. 10. head/tail show rows; info gives types and non-null counts; describe gives statistics; value_counts gives frequency. 11. dropna removes incomplete rows; fillna replaces NaN. 12. Element-wise AND; brackets make each comparison evaluate first. 13. Runs a function on every value: `df["x"].apply(lambda v: v * 2)`. 14. Read → Inspect → Clean → Filter → Group.

Important points for examination

1. Pandas is an external package layered on NumPy, designed for tabular data; the universal import is `import pandas as pd`.
2. A **Series** holds one indexed column; a **DataFrame** is a grid of labelled rows and columns.
3. `pd.read_csv("file.csv")` imports a CSV; `pd.read_excel("file.xlsx")` imports Excel (requires `openpyxl`); `to_csv(..., index=False)` saves.
4. The four inspection functions: `head(n)` / `tail(n)` / `info()` / `describe()`; plus `value_counts()` for frequency tables and `.shape` for dimensions.
5. Sort with `sort_values(col, ascending=False)`; rank with `.rank()`.
6. Add columns by assignment (`df["new"] = ...`); drop with `.drop(columns=[...])`; rename with `.rename(columns={...})`.
7. `.str` accessor applies string methods to columns: `.str.strip()`, `.str.title()`, `.str.split(expand=True)`, `.str.contains()`.
8. `.apply(function)` or `.apply(lambda x: ...)` transforms each value; `.map(dict)` translates via lookup; `axis=1` gives the whole row.
9. `pd.merge(left, right, on="key", how="inner"/"left"/"outer")` joins tables; `pd.concat([a, b])` stacks rows.
10. `pd.to_datetime(col)` converts to dates; `.dt.year` / `.dt.month` / `.dt.dayofweek` extract components; date ranges filter with `>=` and `<=`.
11. `isnull().sum()` counts gaps; `dropna(subset=[...])` removes incomplete rows; `fillna(mean)` patches; `drop_duplicates()` and `astype(type)` complete the cleaning toolkit.

12. Filter: `df[df["col"] > x]`; combine with `&` / `|` in brackets; `isin(list)`, `between(a, b)`, `~` negation, `.str.contains()`.
13. `groupby("col")` partitions rows by the column's unique values; `.count()`, `.mean()`, `.sum()`, `.agg(["mean", "max"])` condense each partition; `groupby([c1, c2]).size().unstack()` cross-tabulates.
14. The Pandas workflow: **Read → Inspect → Clean → Filter → Group** — every analysis follows this shape.

Sources for this unit: course materials; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 4* notes, pandas official documentation (pandas.pydata.org), MCA R24 lecture notes, trekhleb/learn-python, jackfrued/Python-100-Days, exercism/python, and the curated learn.biodockify.com catalog. All prose and examples are original; code outputs verified by execution; originality audited against every consulted source.

UNIT V

Data Visualization with Matplotlib

Chapters 16–18 · 6 teaching hours

Chapter 16 — Matplotlib Fundamentals

16.1 What Matplotlib is and why pharmacists plot

Matplotlib is Python'leading plotting library — converting raw numeric columns into figures suitable for journals and reports. In pharmaceutical research, a graph often reveals what raw numbers hide: the peak and fall of a concentration-time curve, the spread of dissolution results across batches, the dose at which a response plateaus. Visuals are not decoration; they are evidence, and regulatory submissions, research papers and hospital reports all demand them.

Install with `pip install matplotlib`, then import the pyplot interface with its universal alias:

```
import matplotlib.pyplot as plt
```

16.2 The plotting workflow

Every Matplotlib figure follows the same five steps:

1. **Pick the right chart** and pass the data (`plt.plot`, `plt.hist`, `plt.scatter`, `plt.boxplot`, `plt.bar`)
2. **Label the x-axis** (`plt.xlabel`)
3. **Label the y-axis** (`plt.ylabel`)
4. **Add a title** (`plt.title`)
5. **Display** (`plt.show`)

In a Jupyter notebook, `plt.show()` is optional — the figure renders below the cell automatically. In a script, it opens the display window.

16.3 Line plots — the workhorse

A line plot connects data points in order — the natural choice for time-series data such as concentration-time curves:

```
import matplotlib.pyplot as plt

time_h = [0, 2, 4, 6, 8, 10, 12]
conc = [0, 18, 25, 22, 16, 10, 6]

plt.plot(time_h, conc, color="teal", marker="o", linewidth=2)
plt.xlabel("Time (hours)")
plt.ylabel("Plasma Concentration (mcg/mL)")
plt.title("Drug Concentration vs Time")
plt.show()
```

This produces a smooth line with circular markers at each data point, teal colour, and fully labelled axes — the minimum standard for any scientific figure.

Line-customisation parameters:

Parameter	Purpose	Example
`color`	line colour	`"teal"`, `"red"`, `"#0F6E6E"`
`marker`	point style at each datum	`"o"` circle, `"s"` square, `"^"` triangle
`linewidth` / `lw`	thickness of the line	`2`
`linestyle` / `ls`	line pattern	`"-"` solid, `"--"` dashed, `":"` dotted
`alpha`	transparency (0–1)	`0.7`
`label`	name for the legend	`"Drug A"`

Five line-plot examples:

```
# 1. Simple line
plt.plot([1, 2, 3, 4], [10, 20, 25, 30])
```

```
# 2. With markers and colour
plt.plot(time_h, conc, color="teal", marker="o")
```

```
# 3. Dashed line for a reference curve
plt.plot(time_h, conc, linestyle="--", color="gray", label="Reference")
```

```
# 4. Two drugs overlaid
plt.plot(time_h, conc, color="teal", label="Drug A")
plt.plot(time_h, [c * 0.6 for c in conc], color="orange", label="Drug B")
plt.legend()
```

```
# 5. With a light grid
plt.plot(time_h, conc)
plt.grid(alpha=0.3)
```

The `label` parameter stores a name; `plt.legend()` displays a box matching each coloured line to its label — essential whenever more than one dataset appears on the same plot.

16.4 Histograms — the distribution view

A **histogram** shows how a single numeric variable is distributed by grouping values into **bins** and counting how many fall in each — the standard view for patient ages, doses or laboratory values.

```
ages = [23, 45, 34, 67, 45, 52, 29, 61, 38, 44, 55, 48]

plt.hist(ages, bins=6, color="orange", edgecolor="white")
plt.xlabel("Patient Age (years)")
plt.ylabel("Number of Patients")
plt.title("Distribution of Patient Ages")
plt.show()
```

Choosing bins. The `bins` setting determines into how many buckets values are grouped. Sparse buckets blur the shape; excessive buckets add clutter. Begin near `bins=10` and tune until the distribution reads clearly — adjust until the shape is clear without being jagged.

More histogram examples:

```
# Narrower bins (more detail)
plt.hist(ages, bins=12, color="teal", edgecolor="white")
```

```
# Wider bins (smoother)
plt.hist(ages, bins=4, color="orange", edgecolor="white")
```

```
# Dose distribution
doses = [250, 500, 500, 750, 500, 250, 1000, 500, 750, 500]
plt.hist(doses, bins=5, color="teal", edgecolor="white")
plt.xlabel("Dose (mg)")
plt.ylabel("Count")
```

```
# Cmax values across patients
cmax_values = [45.2, 38.6, 50.1, 42.8, 47.5, 39.2, 44.0, 41.3]
plt.hist(cmax_values, bins=8, color="orange", edgecolor="gray")
plt.xlabel("Cmax (mcg/mL)")
```

16.5 Scatter plots — the relationship view

A **scatter plot** draws individual data points for two variables on the same figure, exposing any association between the two variables — for example, dose versus therapeutic response.

```
dose = [50, 100, 150, 200, 250, 300, 350, 400]
response = [8, 18, 25, 33, 38, 44, 50, 55]

plt.scatter(dose, response, color="teal", alpha=0.75, s=60)
plt.xlabel("Dose (mg)")
plt.ylabel("Response (% reduction in symptom score)")
plt.title("Dose vs Therapeutic Response")
plt.show()
```

Scatter customisation:

Parameter	Purpose
`s`	marker (dot) size
`alpha`	transparency (0 = invisible, 1 = solid) — helps when points overlap
`c` / `color`	point colour

More scatter examples:

```
# Larger, semi-transparent points
plt.scatter(dose, response, s=100, alpha=0.5, color="teal")
```

```
# Age vs Cmax (looking for a relationship)
age = [34, 58, 45, 28, 62, 51, 40, 55]
cmax = [45.2, 38.6, 50.1, 42.8, 47.5, 39.2, 44.0, 41.3]
plt.scatter(age, cmax, color="orange", s=70)
plt.xlabel("Age (years)")
plt.ylabel("Cmax (mcg/mL)")
```

```
# Two groups in different colours
plt.scatter(dose[:4], response[:4], color="teal", label="Group A")
plt.scatter(dose[4:], response[4:], color="orange", label="Group B")
plt.legend()
```

16.6 Box plots — the five-number summary

A **box plot** (box-and-whisker) condenses a dataset into median, quartile and outlier markers — the natural choice when contrasting Cmax or dissolution results between several batches on one figure.

```
batch_a = [50, 55, 48, 52, 58, 47, 53]   # fast-release formulation
batch_b = [39, 41, 40, 38, 42, 40, 41]   # sustained-release
batch_c = [42, 47, 43, 45, 44, 46, 44]   # reference standard

plt.boxplot([batch_a, batch_b, batch_c],
            tick_labels=["Batch A", "Batch B", "Batch C"])
plt.ylabel("Cmax (mcg/mL)")
plt.title("Cmax Spread Across Three Batches")
plt.show()
```

Anatomy of a box plot:

Part	Meaning
Box (top edge)	75th percentile (Q3)
Line inside the box	median (50th percentile)
Box (bottom edge)	25th percentile (Q1)
Whiskers	range of typical (non-outlier) values
Dots beyond whiskers	outliers — unusually high or low values

A wide box signals significant manufacturing inconsistency (a quality flag); a tight box indicates consistent production.

More box-plot examples:

```
# Single batch
plt.boxplot(batch_a)
plt.ylabel("Cmax (mcg/mL)")
```

```
# Two batches, different colours
bp = plt.boxplot([batch_a, batch_b], patch_artist=True,
                 tick_labels=["Fast-release", "Sustained"])
bp["boxes"][0].set_facecolor("lightblue")
bp["boxes"][1].set_facecolor("lightyellow")
```

```
# Horizontal box plots (rotate for long labels)
plt.boxplot([batch_a, batch_b, batch_c], vert=False,
            tick_labels=["Formulation A", "Formulation B", "Reference"])
```

16.7 Labelling axes, titles and legends (syllabus point)

Every scientific plot must be fully labelled — axes without names are rejected by journals and examiners alike. When two or more datasets share the same axes, a legend is mandatory.

Function	Purpose
`plt.xlabel("...")`	names the x-axis

Function	Purpose
`plt.ylabel("...")`	names the y-axis
`plt.title("...")`	adds a title above the plot
`label="..."` (in plot call)	sets the name shown in the legend
`plt.legend()`	displays the legend box; `loc` sets position (e.g. `"upper right"`)
`plt.grid(alpha=0.3)`	adds light gridlines for easier reading

The fully labelled standard — every plot in this unit and beyond should look like this:

```
plt.plot(time_h, conc, color="teal", marker="o", label="Drug A")
plt.plot(time_h, [c * 0.6 for c in conc], color="orange",
         linestyle="--", label="Drug B")
plt.xlabel("Time (hours)")
plt.ylabel("Plasma Concentration (mcg/mL)")
plt.title("Comparing Two Drugs")
plt.legend(loc="upper right")
plt.grid(alpha=0.3)
plt.show()
```

Chapter summary. `import matplotlib.pyplot as plt`; the five-step workflow (plot → xlabel → ylabel → title → show). Line plots for time-series (`color`, `marker`, `linewidth`, `linestyle`, `label`); histograms for distributions (`bins`, `edgecolor`); scatter plots for relationships (`s`, `alpha`, `c`); box plots for five-number summaries across groups (box/median/whiskers/outliers). Label everything — axes, title, legend — every time.

16.8 Figure sizing and saving — producing publication-quality files

Every scientific figure eventually needs to live outside the notebook — in a report, a research paper, or a regulatory submission. Two tools control the output: `figsize` sets the physical dimensions; `savefig()` writes the file.

Sizing with figsize. The `figsize=(width, height)` parameter is set *before* plotting, in inches:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))          # 8 inches wide, 5 inches tall
plt.plot([0, 1, 2, 3], [0, 5, 10, 15])
plt.xlabel("Time (hours)")
plt.ylabel("Concentration")
plt.title("Standard figure (8 x 5 inches)")
plt.show()
```

A width of 6–10 inches suits most journals; taller figures (`figsize=(6, 8)`) suit concentration-time curves with long y-axes. The default Matplotlib size is `(6.4, 4.8)` — slightly smaller than a typical journal figure.

Five figsize examples:

```
plt.figure(figsize=(10, 6))         # wide — for dissolution profiles with many
time points
```

```
plt.figure(figsize=(6, 6))          # square — for scatter plots (dose vs
response)
```

```
plt.figure(figsize=(12, 4))         # very wide — for bar charts with many drugs
```

```
plt.figure(figsize=(5, 7))          # tall — for concentration-time curves
```

```
plt.figure(figsize=(8, 6), dpi=100) # 800 x 600 pixels at 100 dpi
```

Saving with savefig(). Call `plt.savefig(filename)` *before* `plt.show()` — once `show()` executes, the figure is consumed:

```
plt.figure(figsize=(8, 5))
plt.plot([0, 1, 2, 3], [0, 5, 10, 15])
plt.xlabel("Time (hours)")
plt.ylabel("Concentration")
plt.title("Saved as PNG at 300 dpi")
plt.savefig("pk_curve.png", dpi=300, bbox_inches="tight")
plt.show()
```

The `dpi=300` setting produces a print-resolution image (300 dots per inch); `dpi=72` suits web display. `bbox_inches="tight"` crops excess white space around the figure edges.

File format guide:

Format	Extension	Best for	Quality
PNG	`.png`	reports, presentations	lossless, crisp lines
PDF	`.pdf`	journal submission	vector (infinitely scalable)
SVG	`.svg`	web, editing in Inkscape	vector
JPG	`.jpg`	photographs (avoid for plots)	lossy

Five savefig examples:

```
plt.savefig("figure.png")                          # default dpi (100)
```

```
plt.savefig("figure.png", dpi=300)                 # publication quality
```

```
plt.savefig("figure.pdf")                          # vector format for journals
```

```
plt.savefig("figure.png", dpi=300, bbox_inches="tight")  # cropped borders
```

```
plt.savefig("figure.png", transparent=True)        # transparent background
```

16.9 Subplots — multiple charts on one figure

A **subplot** is a mini-plot that occupies one cell of a grid on a single figure. Use subplots when you want to compare views side by side — for example, the same patients' concentration-time curve and their ADR severity distribution on one page.

The `plt.subplot(rows, columns, index)` function selects which cell to draw into next:

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 12, 50)

# Create a 1 x 2 grid (1 row, 2 columns)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)                          # left panel
plt.plot(t, 45 * np.exp(-0.3 * t), "teal")
plt.xlabel("Time (hours)")
plt.ylabel("Concentration (mcg/mL)")
plt.title("IV Administration")

plt.subplot(1, 2, 2)                          # right panel
plt.plot(t, 60 * (np.exp(-0.3*t) - np.exp(-1.2*t)), "orange")
plt.xlabel("Time (hours)")
plt.ylabel("Concentration (mcg/mL)")
plt.title("Oral Administration")

plt.suptitle("Concentration-Time Curves: IV vs Oral")   # figure-level title
plt.tight_layout()                                        # prevent label
overlap
plt.show()
```

Read the `subplot` call: `subplot(1, 2, 1)` means "1 row, 2 columns, draw into cell 1 (left)"; `subplot(1, 2, 2)` draws into cell 2 (right). `suptitle` adds a title above the whole figure; `tight_layout()` adjusts spacing so labels do not collide.

Five subplot examples:

```
# 2 x 2 grid — four plots
plt.figure(figsize=(10, 8))
for i in range(1, 5):
    plt.subplot(2, 2, i)
    plt.plot(np.random.rand(20))
    plt.title(f"Plot {i}")
plt.tight_layout()
plt.show()
```

```
# 2 x 1 grid — two stacked plots
plt.figure(figsize=(8, 8))
plt.subplot(2, 1, 1)
plt.plot(t, 45 * np.exp(-0.3 * t), "teal")
plt.title("Concentration")
plt.subplot(2, 1, 2)
plt.plot(t, np.gradient(45 * np.exp(-0.3 * t)), "orange")
plt.title("Rate of change")
plt.tight_layout()
plt.show()
```

```
# 1 x 3 grid — three side-by-side (PK, dissolution, ADR)
plt.figure(figsize=(15, 4))
plt.subplot(1, 3, 1)
plt.hist(np.random.normal(45, 5, 100), bins=15, color="teal")
plt.title("Cmax distribution")
plt.subplot(1, 3, 2)
plt.boxplot([np.random.normal(50, 5, 30), np.random.normal(45, 3, 30)],
            tick_labels=["A", "B"])
plt.title("Batch comparison")
plt.subplot(1, 3, 3)
plt.bar(["Drug X", "Drug Y", "Drug Z"], [15, 22, 8], color="orange")
plt.title("ADR counts")
plt.tight_layout()
plt.show()
```

```
# 3 x 1 grid — three stacked time-series
plt.figure(figsize=(8, 10))
for i, label in enumerate(["Conc", "Rate", "AUC"]):
    plt.subplot(3, 1, i + 1)
    plt.plot(t, np.exp(-0.3 * t) * (45 - i * 10), label=label)
    plt.legend()
plt.tight_layout()
plt.show()
```

```
# Mixed: one large + two small (using gridspec)
plt.figure(figsize=(10, 8))
plt.subplot2grid((3, 3), (0, 0), colspan=3)      # full-width top
plt.plot(t, 45 * np.exp(-0.3 * t), "teal")
plt.title("Main concentration-time curve")
plt.subplot2grid((3, 3), (1, 0), colspan=2)      # left 2/3 middle
plt.hist(np.random.normal(45, 5, 50), bins=10, color="orange")
plt.title("Cmax histogram")
plt.subplot2grid((3, 3), (1, 2))                  # right 1/3 middle
plt.bar(["A", "B"], [10, 15], color="teal")
plt.title("ADR")
plt.subplot2grid((3, 3), (2, 0), colspan=3)      # full-width bottom
plt.scatter(t, np.random.rand(50) * 50, alpha=0.5, color="teal")
plt.title("Scatter: time vs random response")
plt.tight_layout()
plt.show()
```

The `subplot2grid` approach offers full layout control — `colspan=3` makes a subplot span three columns. Combined with `tight_layout()`, it produces professional multi-panel figures suitable for any report.

Quick checks 16. (a) What are the five steps of the plotting workflow? (b) Which plot type for a distribution of patient ages? (c) What does `alpha=0.5` do on a scatter plot? (d) In a box plot, what does the line inside the box represent? (e) What function displays the legend? (f) What does `figsize=(8, 5)` mean? (g) Why call `savefig()` before `show()`? (h) What does `plt.subplot(2, 2, 3)` select? (i) What does `tight_layout()` do?

Chapter 17 — Pharmaceutical Plots (Syllabus Point)

17.1 Concentration-time curves — oral vs IV administration

The **concentration-time curve** charts how plasma drug levels change over time and anchors all pharmacokinetic analysis. The two administration routes produce characteristically different shapes:

- **IV administration** — delivery is straight into the vein, so the trace **opens at maximum concentration** and falls steadily thereafter (elimination alone).
- **Oral administration** — the medicine must first dissolve and cross the gut wall, so the trace **climbs through an absorption phase**, summits at Cmax (reached at Tmax), then tapers under elimination.

The following example uses simple one-compartment pharmacokinetic equations to generate realistic curves:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 12, 100)       # 0 to 12 hours, 100 points
ka, ke = 1.2, 0.3                  # absorption & elimination rate constants

iv_conc = 45 * np.exp(-ke * t)                                    # IV:
elimination only
oral_conc = 60 * (np.exp(-ke * t) - np.exp(-ka * t)) * ka / (ka - ke)  # Oral:
absorption + elimination

plt.plot(t, iv_conc, color="teal", linewidth=2, label="IV Administration")
plt.plot(t, oral_conc, color="orange", linewidth=2, label="Oral Administration")
plt.xlabel("Time (hours)")
plt.ylabel("Plasma Concentration (mcg/mL)")
plt.title("Concentration-Time Curve: Oral vs IV")
plt.legend(loc="upper right")
plt.grid(alpha=0.3)
plt.show()
```

Reading these curves:

- The IV trace (teal) opens at its maximum and falls continuously — no absorption phase.
- The oral trace (orange) climbs from zero to a summit (Cmax at Tmax), then tapers under elimination.
- The **area beneath each trace (AUC)** measures cumulative drug exposure — a key bioequivalence parameter.

17.2 ADR reporting rates — the bar chart

A **bar chart** compares a count or rate across categories — the standard view for ADR reporting by drug:

```
import matplotlib.pyplot as plt

drugs = ["Metformin", "Amoxicillin", "Ibuprofen", "Atorvastatin", "Amlodipine"]
adr_counts = [22, 17, 11, 9, 6]

plt.bar(drugs, adr_counts, color=["teal", "orange", "teal", "orange", "teal"])
plt.xlabel("Drug Name")
plt.ylabel("Number of ADR Reports")
plt.title("ADR Reporting Rates Across Drugs")
plt.xticks(rotation=30)
plt.show()
```

`plt.xticks(rotation=30)` tilts the drug names so they do not overlap — a common fix for categorical labels.

Interpretation caution: Metformin tops this chart, though a count alone does not show *why* — heavier prescribing may account for it rather than a genuinely riskier profile. Weigh the denominator (how many receive each drug) before ranking risk.

17.3 Dissolution profiles — comparing formulations

A **dissolution profile** tracks the fraction of drug released over time — the standard comparison for release behaviour differences between tablet formulations, indispensable in quality assurance and bioequivalence work.

```
import matplotlib.pyplot as plt

time_min = [0, 5, 10, 15, 30, 45, 60, 90]
form_a = [0, 22, 38, 52, 74, 86, 93, 98]   # fast-release
form_b = [0, 10, 18, 26, 45, 58, 68, 80]   # sustained-release

plt.plot(time_min, form_a, color="teal", marker="o", label="Formulation A (fast-
release)")
plt.plot(time_min, form_b, color="orange", marker="s", label="Formulation B
(sustained-release)")
plt.xlabel("Time (minutes)")
plt.ylabel("% Drug Dissolved")
plt.title("Dissolution Profile Comparison")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```

Reading the dissolution plot: Formulation A reaches 85% dissolved within 30 minutes (fast-release behaviour); Formulation B takes about 60 minutes (sustained-release). The two curves diverge most between 15 and 45 minutes — the window where formulation choice matters clinically. Similar shapes and AUCs may suggest bioequivalence; clearly different profiles mean the formulations are not interchangeable.

17.4 The mini PK study — a complete workflow

The capstone of the book: load a concentration CSV → clean with Pandas → compute parameters with NumPy → plot with Matplotlib. Every unit's skills in one program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Read the PK dataset
pk = pd.DataFrame({
    "Time_h": [0, 0.5, 1, 2, 4, 6, 8, 10, 12],
    "Conc": [0.0, 8.2, 15.6, 22.1, 18.3, 12.5, 8.1, 5.2, 3.1],
})

# 2. Compute pharmacokinetic parameters
cmax = pk["Conc"].max()
tmax = pk.loc[pk["Conc"].idxmax(), "Time_h"]
auc = np.trapezoid(pk["Conc"], pk["Time_h"])
# Note: on NumPy versions prior to 2.0, use np.trapz() instead

# 3. Plot the curve
plt.plot(pk["Time_h"], pk["Conc"], color="teal", marker="o", linewidth=2)
plt.axhline(cmax, color="orange", linestyle="--", alpha=0.7, label=f"Cmax =
{cmax:.1f}")
plt.axvline(tmax, color="gray", linestyle=":", alpha=0.7, label=f"Tmax =
{tmax:.1f} h")
plt.fill_between(pk["Time_h"], pk["Conc"], alpha=0.15, color="teal", label=f"AUC
= {auc:.1f}")
plt.xlabel("Time (hours)")
plt.ylabel("Plasma Concentration (mcg/mL)")
plt.title("Mini PK Study: Concentration-Time Curve")
plt.legend(loc="upper right")
plt.grid(alpha=0.3)
plt.show()

print(f"Cmax = {cmax:.1f} mcg/mL at Tmax = {tmax:.1f} h")
print(f"AUC (0-12h) = {auc:.1f} mcg*h/mL")
```

OUTPUT

```
Cmax = 22.1 mcg/mL at Tmax = 2.0 h
AUC (0-12h) = 140.2 mcg*h/mL
```

Three units in one script: Pandas reads and structures; NumPy computes the AUC by the trapezoidal rule; Matplotlib draws the curve with Cmax (dashed line), Tmax (dotted line) and the AUC (shaded area) annotated. This is the shape of every real PK analysis — and it fits on one page.

Chapter summary. Concentration-time curves compare IV (immediate peak, pure elimination) with oral (absorption phase → Cmax at Tmax → elimination); the AUC reflects total exposure. ADR rates are compared with bar charts — but always contextualise counts against prescription volume. Dissolution profiles compare formulation release rates; separate curves signal non-interchangeability. The mini PK study (§16.4) integrates the entire book: Pandas for data, NumPy for computation, Matplotlib for the figure.

17.5 Annotations — labelling points on the plot

Annotations point the reader's eye to specific features — the Cmax, a threshold, or an outlier. `plt.annotate()` draws an arrow from a text label to a coordinate on the plot:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 12, 100)
oral = 60 * (np.exp(-0.3*t) - np.exp(-1.2*t))
cmax_val = oral.max()
tmax_val = t[oral.argmax()]

plt.plot(t, oral, "teal", linewidth=2)
plt.annotate(
    f"Cmax = {cmax_val:.1f}",
    xy=(tmax_val, cmax_val),           # arrow tip
    xytext=(tmax_val + 2, cmax_val + 3), # text position
    arrowprops={"arrowstyle": "->", "color": "gray"},
    fontsize=11, color="darkred",
)
plt.xlabel("Time (hours)")
plt.ylabel("Concentration (mcg/mL)")
plt.title("Cmax Annotated on the Curve")
plt.show()
```

Five annotation examples:

```
# Point out Tmax with a vertical dashed line
plt.axvline(tmax_val, color="gray", linestyle="--", alpha=0.7, label=f"Tmax =
{tmax_val:.1f} h")
plt.legend()
```

```
# Shade the AUC region
plt.fill_between(t, oral, alpha=0.15, color="teal", label="AUC (total
exposure)")
plt.legend()
```

```
# Simple text label (no arrow)
plt.text(6, 30, "Elimination phase", fontsize=10, style="italic", color="gray")
```

```
# Circle an outlier on a scatter plot
plt.scatter([2, 4, 6, 8, 15], [10, 20, 15, 25, 80], s=60)
plt.annotate("Outlier?", xy=(15, 80), xytext=(11, 60),
             arrowprops={"arrowstyle": "->", "color": "red"},
             color="red", fontsize=11)
```

```
# Horizontal threshold line
plt.axhline(50, color="orange", linestyle=":", label="Safe limit = 50")
plt.legend()
```

The `axvline` / `axhline` pair (vertical / horizontal reference lines) is the quickest way to mark a threshold, a regulatory limit, or a clinical cut-off on any plot.

17.6 Advanced bar charts — stacked and grouped

When comparing ADR counts across drugs **and** severity levels, a simple bar chart runs out of dimensions. Two solutions:

Grouped bars place bars side by side within each category:

```
import matplotlib.pyplot as plt
import numpy as np

drugs = ["Metformin", "Amoxicillin", "Ibuprofen"]
mild = [5, 8, 3]
moderate = [3, 4, 5]
serious = [2, 1, 3]

x = np.arange(len(drugs))
width = 0.25

plt.bar(x - width, mild, width, label="Mild", color="teal")
plt.bar(x, moderate, width, label="Moderate", color="orange")
plt.bar(x + width, serious, width, label="Serious", color="crimson")
plt.xlabel("Drug")
plt.ylabel("ADR count")
plt.title("ADR Severity by Drug (Grouped)")
plt.xticks(x, drugs, rotation=15)
plt.legend()
plt.tight_layout()
plt.show()
```

Stacked bars pile the sub-categories on top of each other — better when the total matters:

```
plt.bar(drugs, mild, label="Mild", color="teal")
plt.bar(drugs, moderate, bottom=mild, label="Moderate", color="orange")
plt.bar(drugs, serious, bottom=[m + mo for m, mo in zip(mild, moderate)],
        label="Serious", color="crimson")
plt.xlabel("Drug")
plt.ylabel("Total ADR count")
plt.title("ADR Severity by Drug (Stacked)")
plt.legend()
plt.tight_layout()
plt.show()
```

Style	Best when	Reading
Grouped	comparing severity counts between drugs	read across
Stacked	the total per drug is the headline	read the total height, then the segments

17.7 Error bars — showing uncertainty

Every measurement has uncertainty. An **error bar** draws a vertical line above and below each data point to visualise that spread — the standard deviation, standard error, or a confidence interval:

```
drugs = ["Formulation A", "Formulation B", "Reference"]
cmax_mean = [52.3, 48.7, 50.1]
cmax_std = [3.2, 2.1, 2.8]

plt.bar(drugs, cmax_mean, yerr=cmax_std, capsize=5,
        color=["teal", "orange", "gray"], edgecolor="black")
plt.xlabel("Formulation")
plt.ylabel("Cmax (mcg/mL) \u00b1 SD")
plt.title("Cmax with Error Bars (n = 3 batches)")
plt.show()
```

The `yerr` parameter accepts a list of standard deviations; `capsize=5` draws the little horizontal caps at the end of each bar. Overlapping error bars suggest the formulations are not significantly different — a visual bioequivalence screen.

Error bars also work on line plots:

```
time = [0, 2, 4, 6, 8, 10]
conc = [0, 18, 25, 22, 16, 10]
conc_err = [0, 2.1, 3.0, 2.5, 1.8, 1.2]

plt.errorbar(time, conc, yerr=conc_err, marker="o", capsize=4,
             color="teal", linewidth=2)
plt.xlabel("Time (hours)")
plt.ylabel("Concentration (mcg/mL)")
plt.title("Concentration-Time with Error Bars")
plt.show()
```

Quick checks 17. (a) Why does an IV curve start high and only decline? (b) What does the AUC represent? (c) Why rotate x-tick labels on a bar chart? (d) In the dissolution example, which formulation reaches 85% faster? (e) Name the three libraries used in the mini PK study and their roles. (f) What does `xy=` specify in `annotate()`? (g) When would you use a stacked bar instead of a grouped bar? (h) What does `yerr` add to a plot?

Chapter 18 — Scientific Interpretation of Plots (Syllabus Point)

18.1 The figure is only step one

Correctly reading and drawing conclusions from a chart is the skill that gives a figure its value in pharmaceutical work. This section provides the interpretation framework the syllabus expects.

18.2 The interpretation reference table

Plot feature	What to look for	Pharmacy meaning
Peak height (Cmax)	higher or earlier peak	faster or more complete absorption
Curve steepness (decline)	steeper fall	faster elimination (higher clearance)
Overlapping curves	similar shape and AUC	may suggest bioequivalence between formulations
Box-plot spread (IQR)	broader box	greater manufacturing inconsistency — a quality concern
Outlier points	dots beyond whiskers	may indicate an assay error *or* a genuine unusual patient/batch
Scatter-plot trend	clear upward (or downward) pattern	suggests a dose-response correlation

18.3 Correlation is not the same as causation

A scatter graph on which dose and response climb in tandem does **not**, on its own, demonstrate that the dose *produces* the response. A confounder such as disease severity, age, or a co-prescribed drug may influence both. Causation demands a controlled trial, not an observed association. This distinction matters daily in pharmacovigilance: an ADR cluster after a drug's launch may reflect increased prescribing, not increased risk.

18.4 Common interpretation mistakes

Mistake	Why it matters	The fix
Reading a bar chart without context	highest count ≠ highest risk	always ask "how many patients were on each drug?"
Ignoring outliers	they may be the most important data points	investigate: assay error or genuine signal?
Over-reading small samples	3 points cannot establish a trend	report sample size alongside every figure

Mistake	Why it matters	The fix
Confusing AUC with Cmax	AUC = total exposure; Cmax = peak level	both matter, but for different clinical questions

Chapter summary. Every plot tells a story: peaks reveal absorption, steepness reveals elimination, overlap suggests bioequivalence, spread reveals manufacturing consistency, outliers warrant scrutiny. Correlation observed on a scatter plot is a starting point for study, not proof of cause. Always contextualise counts (prescription volume), investigate outliers, and report sample sizes.

18.5 Pie charts — proportions of a whole

A **pie chart** shows how a total splits into parts — for example, the proportion of ADR reports by severity band, or a pharmacy budget by category. Matplotlib's `plt.pie()` takes a list of values and optional labels:

```
import matplotlib.pyplot as plt

severity = ["Mild", "Moderate", "Serious"]
counts = [45, 30, 15]

plt.pie(counts, labels=severity, autopct="%1.1f%%",
        colors=["teal", "orange", "crimson"], startangle=90)
plt.title("ADR Reports by Severity")
plt.show()
```

The `autopct="%1.1f%%"` parameter prints the percentage on each slice; `startangle=90` rotates the first slice to the top.

When to use a pie chart: only when there are 3–5 categories and the proportions are the story. With more categories, or when precise values matter, a bar chart is clearer.

Five pie chart examples:

```
# With percentages and a shadow
plt.pie(counts, labels=severity, autopct="%1.0f%%", shadow=True)
```

```
# Explode (pull out) the largest slice
explode = [0.1, 0, 0]  # pull "Mild" out by 10%
plt.pie(counts, labels=severity, explode=explode, autopct="%1.1f%%")
```

```
# Drug budget breakdown
categories = ["Generics", "Brands", "Surgical", "OTC"]
spend = [40, 35, 15, 10]
plt.pie(spend, labels=categories, autopct="%1.0f%%", colors=["teal", "orange",
"gray", "crimson"])
```

```
# Donut chart (add a hole) — a modern pie variant
plt.pie(counts, labels=severity, autopct="%1.1f%%", wedgeprops={"width": 0.6})
plt.text(0, 0, "90\nreports", ha="center", va="center", fontsize=14,
fontweight="bold")
```

```
# Flat (no tilt) — for reports that will be printed in grayscale
plt.pie(counts, labels=severity, autopct="%1.0f%%",
        colors=["#E8F4F1", "#F5C069", "#C0533E"],
        wedgeprops={"edgecolor": "black", "linewidth": 1})
```

18.6 Colour maps and colour bars

When a scatter plot carries a third variable (e.g. dose vs response vs patient age), colour can encode it. A **colour bar** acts as the legend for the colour scale:

```
import matplotlib.pyplot as plt

dose = [50, 100, 150, 200, 250, 300, 350, 400]
response = [8, 18, 25, 33, 38, 44, 50, 55]
age = [34, 58, 45, 28, 62, 51, 40, 55]    # third variable

scatter = plt.scatter(dose, response, c=age, cmap="viridis", s=80,
                      edgecolors="black", linewidth=0.5)
plt.colorbar(scatter, label="Patient Age (years)")
plt.xlabel("Dose (mg)")
plt.ylabel("Response (% reduction)")
plt.title("Dose-Response Coloured by Age")
plt.show()
```

The `c=age` parameter assigns each point a colour based on its age value; `cmap="viridis"` selects the colour map (viridis is colourblind-friendly); `plt.colorbar()` displays the scale.

Quick checks 18. (a) What does a steeper decline on a concentration-time curve suggest? (b) A wide box in a box plot means what? (c) Give one reason why correlation does not prove causation. (d) Why report sample size alongside every figure? (e) When is a pie chart appropriate? (f) What does `c=` encode in a scatter plot?

Unit V — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Introduction to Matplotlib	Ch 16 (§16.1–16.2)
Creating line plots, histograms, scatter plots, box plots	Ch 16 (§16.3–16.6)
Labeling axes, titles, legends	Ch 16 (§16.7)
Pharmaceutical plots: concentration-time, ADR rates, dissolution	Ch 17 (§17.1–17.4)
Scientific interpretation of plots	Ch 18

Twenty-question exercise bank

Short answers (1–8).

1. Name the four core plot types and their use cases.
2. What are the five steps of the plotting workflow?
3. What does the `bins` parameter control in a histogram?
4. In a box plot: what do the box edges, the line inside, and the whiskers represent?
5. What does `alpha=0.5` do?
6. Why is a legend essential when overlaying two datasets?
7. How does an IV concentration-time curve differ from an oral one?
8. What does the AUC represent on a concentration-time curve?

Interpretation (9–14).

9. A box plot of Batch X is much wider than Batch Y. What does this mean?
10. A scatter plot of dose vs response shows a clear upward trend. Can you conclude that higher doses cause better response?
11. A bar chart shows Drug A with 100 ADR reports and Drug B with 20. Is Drug A riskier?
12. Two dissolution profiles have almost identical shapes. What might this suggest?
13. A concentration-time curve drops very steeply after the peak. What pharmacokinetic property does this reflect?
14. An outlier dot appears in a box plot of Cmax values. List two possible explanations.

Write programs (15–20).

15. Plot a line graph of temperature readings over 24 hours with proper labels.
16. Create a histogram of 20 patient ages with `bins=8`.

17. Scatter plot dose (x) against response (y) for 8 data points; add labels.
18. Box plot Cmax values for two batches; label the batches.
19. Plot an oral concentration-time curve (use the §16.1 model) with Cmax annotated.
20. Bar chart ADR counts for five drugs; rotate the labels.

Answers to selected exercises

1. Line (time-series), histogram (distribution), scatter (relationship), box (five-number summary). 2. Plot → xlabel → ylabel → title → show. 3. The number of groups the data is divided into. 4. Q3 & Q1 (edges), median (line), typical range (whiskers). 5. Makes the plotted element 50% transparent. 7. IV starts at its peak and only declines; oral rises to a peak then declines. 8. Total drug exposure over the measured period. 9. Greater manufacturing inconsistency in Batch X. 10. No — association is not causation; confounders may exist. 11. Not necessarily — Drug A may simply have more patients. 12. Bioequivalence between the two formulations. 13. High clearance / rapid elimination. 14. Assay error; genuine unusual patient or batch.

Important points for examination

1. Matplotlib is imported as `import matplotlib.pyplot as plt` — the universal alias.
2. The plotting workflow: choose plot type and pass data → label x-axis → label y-axis → add title → display.
3. Four core plot types: **line** (`plt.plot`) for time-series; **histogram** (`plt.hist`) for frequency distributions; **scatter** (`plt.scatter`) for relationships between two variables; **box plot** (`plt.boxplot`) for five-number summaries (median, quartiles, whiskers, outliers) across groups.
4. Customisation parameters: `color`, `marker`, `linewidth`/`lw`, `linestyle`/`ls`, `alpha`, `label` (for the legend), `s` (scatter size), `bins` (histogram groups), `edgecolor` (histogram bar borders).
5. Every scientific figure must be fully labelled: `xlabel`, `ylabel`, `title`, and `legend()` whenever more than one dataset appears.
6. **Concentration-time curves**: IV administration shows an instant maximum followed by elimination alone; oral administration shows absorption → Cmax at Tmax → elimination; the AUC measures cumulative drug exposure.
7. **ADR rates** are compared with bar charts (`plt.bar`); `xticks(rotation=...)` prevents label overlap; weigh counts against how widely each drug is prescribed.
8. **Dissolution profiles** compare formulation release rates; matching curves hint at bioequivalence; separate curves signal non-interchangeability.
9. **Interpretation**: steeper fall = quicker clearance; broader box = more variability; overlapping curves = possible bioequivalence; outliers warrant scrutiny.

10. **Correlation is not the same as causation** — an association on a scatter plot is a lead to investigate, not a finding to publish as proven.

Sources for this unit: course materials; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 5* notes, matplotlib.org documentation and gallery, MCA R24 lecture notes, and the curated learn.biodockify.com catalog (matplotlib, numpy, pandas). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source. Full credits: see the book's Sources and Acknowledgements page.

Recommended References

(Preferably latest editions.)

1. Weiss, C.J., 2017. *Scientific Computing for Chemists with Python.* Available at: https://weisscharlesj.github.io/SciCompforChemists/notebooks/introduction/intro.html

2. Perkovic, L., 2015. *Introduction to Computing Using Python: An Application Development Focus.* 2nd ed. Hoboken: Wiley.

3. Sweigart, A., 2025. *Automate the Boring Stuff with Python.* 3rd ed. Available at: https://automatetheboringstuff.com/

4. W3Schools, n.d. *Python Tutorial.* Available at: https://www.w3schools.com/python/

5. Datasets for Education and Research. Mentors and students can access healthcare datasets from sources such as Kaggle (healthcare records), government agencies (healthdata.gov, WHO, https://www.data.gov.in/), and clinical trial registries (https://ctri.nic.in/, https://clinicaltrials.gov/). Always use data responsibly.

Appendix A — Quick Reference

The one-page summary of every structure and method covered in this book.

Category	Function / Method	Purpose
Variables	type(x), int(), float(), str()	check and convert types
Operators	+ − // / % *	arithmetic
Operators	== != > < >= <= and or not	comparison, logic
Strings	len(), [i], [a:b], .strip()	measure, address, clean
Strings	.upper(), .lower(), .title()	case methods
Strings	.replace(), .find(), .split()	edit, search, take apart
I/O	input(), print(), f"{x:.1f}"	ask, report, format
If / elif / else	if cond: ... elif: ... else:	branch on conditions
Loops	for x in list: / while cond:	repeat work
Loops	break / continue	exit / skip a round
Functions	def name(params): return	define reusable tools
Lists	.append(), .pop(), .sort()	add, remove, order
Lists	len(), sum(), max(), .copy()	measure, aggregate
Tuples	(a, b) — sealed; a, b = t	fixed records
Dicts	d[key], .get(k, default)	lookup by name
Dicts	d[k]=v, .update(), .pop(k)	add, change, remove
NumPy	np.array(), np.zeros(), np.arange()	create arrays
NumPy	arr.mean(), .sum(), .max()	aggregate
NumPy	arr[arr > limit], np.where()	filter, find positions
Pandas	pd.read_csv(), pd.read_excel()	load files
Pandas	df.head(), .info(), .describe()	inspect
Pandas	.dropna(), .fillna(), .drop_duplicates()	clean
Pandas	df[df['col'] > x], df.loc[], df.iloc[]	filter / select
Pandas	df.groupby('col').mean()	summarise groups
Matplotlib	plt.plot(), .hist(), .scatter()	draw charts
Matplotlib	plt.xlabel(), .title(), .legend()	label charts

Appendix B — Practice Datasets

The syllabus recommends the following free healthcare datasets for tutorial and homework use. Each is publicly accessible; always use data responsibly.

Source	URL	What you will find
Kaggle	kaggle.com/datasets	Healthcare records, drug reviews, PK datasets, ADR reports
healthdata.gov	healthdata.gov	US government health datasets (claims, surveys, outcomes)
WHO	who.int/data/gho	Global Health Observatory: world health statistics
data.gov.in	data.gov.in	Government of India open data (health, drugs, pharma)
CTRI	ctri.nic.in	Clinical Trial Registry of India
clinicaltrials.gov	clinicaltrials.gov	US National Library of Medicine trial registry

Suggested exercises using these datasets:

- Download a Kaggle drug-review dataset (CSV) and load it with `pd.read_csv()`. Use `.info()` and `.describe()` to inspect it.
- Filter the dataset for a specific drug name using `df[df['drugName'] == '...']`.
- Group the reviews by rating and count each group with `df.groupby('rating').count()`.
- Plot the age distribution as a histogram with `plt.hist()`.
- Create a bar chart of the top 5 most-reviewed drugs using `plt.bar()`.

Sources and Acknowledgements

The syllabus follows the Pharmacy Council of India's *BP101T — Basics of Python Programming for Pharmaceutical Sciences (Theory)*, B.Pharm Semester I. The clinical formulas used for teaching illustration (Cockcroft–Gault, Clark and Young rules, Mosteller BSA) are standard published equations; their use here is educational, not clinical guidance.

The author gratefully acknowledges the maintainers of the open resources consulted during preparation — including the curated repositories of learn.biodockify.com (NumPy, Pandas, Matplotlib, RDKit, Biopython and the beginner course collections), W3Schools, exercism/python, and the following teaching notes consulted for topic coverage:

- Yamuna Devi, Jeyabharathi & Vajpayee, *Python Programming* (GCS Publishers)
- MCA R24 lecture notes, Sreenivasa Institute of Technology and Management Studies
- R17 lecture notes, Malla Reddy College of Engineering and Technology
- Noteskarts, *Computer Applications in Pharmacy* Units 1–5
- trekhleb/learn-python, jackfrued/Python-100-Days, joaoventura/full-speed-python, gto76/python-cheatsheet

All prose, examples and pharmacy applications in this book are original. Code outputs were verified by execution. Originality was audited against every consulted source (zero seven-word prose overlaps).

Python is a registered trademark of the Python Software Foundation. Product names are used for identification only.

About the Publisher

BioDockify (biodockify.com) is an AI-powered pharmaceutical research platform offering molecular docking, ADMET prediction, molecular dynamics simulation and QSAR modelling — free, browser-based, with no installation required.

BioDockify Learn (learn.biodockify.com) is the education arm — a curated library of free Python, data science, machine learning, bioinformatics and cheminformatics courses built from open-source GitHub repositories, designed for pharmacy and life-science students.

The platform is led by **Mr. Tajuddin Shaik**, Co-Founder of BioDockify and Associate Professor in the Department of Pharmacology at Malla Reddy College of Pharmacy, who directs the scientific vision behind BioDockify's AI-driven drug discovery tools and educational content.

BioDockify

Building the Future of AI-Driven Pharmaceutical Research

learn.biodockify.com

Python for Pharmacists

The complete BP101T companion text — from hello world to drug discovery data.

- 18 chapters across 5 units, following the PCI syllabus exactly
- 355 machine-verified code examples — every output is real
- Pharmacy-first: BMI, Cockcroft–Gault, PK curves, ADR reports, dissolution
- 100 exercise questions with answer keys
- Zero-assumed-prior-knowledge — starts from installation



learn.biodockify.com

Published by the BioDockify Team

Basics of Python Programming for Pharmaceutical Sciences